

classes

January 16, 2006
15:42

Contents

1	Introduction	1
2	Class Attribute Descriptions	3
3	Class definitions	3
4	Element definitions	3
5	Element class attribute descriptions	4
6	Species definitions	5
7	Species class attribute descriptions	6
8	Particle definitions	7
9	Particle class attribute descriptions	9
10	Location definitions	11
11	Location class attribute descriptions	12
12	Flight definitions	13
13	Flight class attribute descriptions	15
14	Reaction definitions	15
15	Reaction class attribute descriptions	17
16	Materials definitions	18
17	Materials class attribute descriptions	19
18	Plasma-Material Interaction (PMI) definitions	21
19	PMI class attribute descriptions	22
20	Background definitions	23
21	Background class attribute descriptions	25

22 Source definitions	26
23 Source class attribute descriptions	28
24 Zone definitions	31
25 Zone class attribute descriptions	33
26 Sector definitions	34
27 Sector class attribute descriptions	39
28 Detector definitions	46
29 Detector class attribute descriptions	47
30 Tally definitions	49
31 Tally class attribute descriptions	54
32 Output definitions	58
33 Output class attribute descriptions	60
34 Local arrays and machinery to allow compression of scoring data	62
35 Description of compression algorithm and data structures	64
36 Random definitions	66
37 Cross section definitions	67
38 Cross section class attribute descriptions	70
39 Rate calculation definitions	72
40 Rate calculation class attribute descriptions	74
41 Plasma-Material Interaction (PMI) format definitions	75
42 PMI format class attribute descriptions	79
43 Problem definitions	81
44 Problem class attribute descriptions	86
45 Reaction data definitions	89
46 Reaction data class attribute descriptions	92
47 Plasma-Material Interaction (PMI) data definitions	95
48 PMI data class attribute descriptions	98
49 String definitions	102
50 Macros for defining and manipulating arrays	102

51 Description of array definition macros	116
52 Macros used to read and write netCDF files	118
53 Description of netCDF Macros	122
54 File names for DEGAS 2	123
55 Description of file name usage in DEGAS 2	125
56 Specification for a 2-D Geometry	125
57 Two-D geometry specification class	126
58 INDEX	127

1 Introduction

\$Id: classes.web,v 1.23 1999/03/25 20:50:15 dstotler Exp \$

This file provides a detailed description of the classes and associated common variables in the code. It is mainly intended to be read by people working on the code directly, including the authors, but may be accessible to the casual user as well. Note that because of its extensive length, the file may not be up-to-date with respect to the code. However, there should not ever be any fundamental discrepancies.

We define the various classes here and the methods to access the components. Each class has a two-letter abbreviation which is used to prefix the accessor functions. Ideally, each class has 5 standard methods:

CL_args(inst) is how to pass a instance of a class to a subroutine. *CL_dummy(inst)* is the dummy argument for a subroutine. *CL_common* will declare common variable for this class. *CL_decl(inst)* is how to declare a member of a class in a subroutine. *CL_copy(inst1, inst2)* is how to copy one instance of a class to another instance of the same class. *CL_check(inst)* will check the consistency of the instance.

This file is intended to document the “accessor functions” (usually macros), representing common arrays as well as few-line code, most used in DEGAS 2. Many of the classes below also contain descriptions of the less frequently used arrays and variables which make up the “internal” description of the class. In a genuine object-oriented language, these would correspond to “public” and “private” methods, respectively.

The instance above is designated in one of two ways.

For some classes, the information is held in global common blocks. In this case, the instance is denoted by an integer variable which is a pointer into the common block arrays. Examples of these classes are elements, species, and reactions.

For the other classes, the instance is tagged with an arbitrary string. The various components are obtained by constructing variable names from that string.

In both cases, the instance can be an array reference and an array of instances can be constructed in a natural manner; e.g., *CL_args(inst₃)* or *CL_decl(inst_{0:5})*.

These combine in a natural way. Thus if we are dealing with flight *f*, then *fl_current(f)* is the current particle being tracked. Its species is *pt_sp(fl_current(f))*. The first component of this species is *sp_el(pt_sp(fl_current(f)), 1)*, and so on.

Because DEGAS 2 is written in a non-object-oriented language and all of this class structure is implemented in a rather ad hoc way, the above described “ideals” are not adhered to throughout the code. The most familiar offenses are:

1. For those classes in which objects are denoted by a single integer, the *CL_args* and *CL_dummy* functions (which are really null macros) are frequently not used. An effort should be made to use these macros anyway as a method of labeling the variables in an argument list.
2. Again for the classes labeled by an integer, *CL_decl* is often dropped in lieu of a normal integer declaration. In some routines, the reason is that the variable in question is the index of a do-loop, and it is desirable to have its integral nature obvious to the reader. Ideally, one would have set up “conversions” from these integers to the actual class variables.

3. Some classes, e.g., the tallies, must be completely defined at the start of the run. In this case, the *CL_copy* function is not really needed and is implemented in only the most superficial manner.
4. For similar reasons, *CL_check* may not always be as thorough as it could be.

Regarding subscript notation: There is a wide variety of subscript notation in use throughout the code. Since it may never be unified, it is necessary to alert the reader to the possible forms and their interpretation.

- All macros contain argument lists in parentheses: (x, y) . These variables may either be passed into some macro code, or may only represent an abbreviation of a common variable name. For example, *el_name*(x) is a macro meaning *element_name*[x]. It *should* be the case that the order of the indices in these macro arrays corresponds to that used in C; i.e., the rightmost index is the most rapidly varying as the length of the array is traversed in memory. To be certain, one must check the macro definition.
- Direct references to common variables have subscripts in brackets: $[x]$. Through some separate coding (not explicit macros in the class declarations as above) C-like subscripts have been set up in the code so that multiple subscripts may be written as: $[y][x]$. In this case, it is guaranteed that the order of the subscripts is reversed in translating these expressions into compilable FORTRAN. The primary reason for establishing the C-like syntax is that a compact vector notation can be implemented via some well-defined macros.
- Unfortunately, the hybrid syntax $[x, y]$ is also found throughout the code. In this case, the order of the array subscripts matches that of the underlying FORTRAN array (leftmost index is most rapidly varying); usage of this sort should be discouraged.
- Finally, one can write the subscripts in the familiar FORTRAN notation: *reaction_reagent*(i, x). This is discouraged since it may possibly cause a variable reference to be mis-identified by a reader as a macro call. Also, the first two syntax options, macro or C-like, should be used whenever possible to encourage uniformity throughout the code.
- To summarize, the following are equivalent variable references:
 1. *rc_reagent*(x, i), invokes a macro,
 2. *reaction_reagent*[i, x], is the result of processing the macro,
 3. *reaction_reagent*[x][i], is the corresponding C-like syntax,
 4. *reaction_reagent*(i, x), is what the compiler will see.
- Throughout most of the rest of this document and in the “woven” code, these are typeset as (note that FWEAVE *does not* swap the index order in the third case as it should):
 1. *rc_reagent*(x, i), invokes a macro,
 2. *reaction_reagent* _{i, x} , is the result of processing the macro,
 3. *reaction_reagent* _{x, i} , is the corresponding C-like syntax,
 4. *reaction_reagent*(i, x), is what the compiler will see.

- The macro invocations used to define arrays in the header files, *define_var_pk* and *define_varp_pk*, order the subscripts in the FORTRAN way. E.g., the definition of the array used in the previous example is:

```
define_varp_pk(rc, reaction_reagent, INT, reagent_ind, reaction_ind)
```

with i being associated with *reagent_ind* and x with *reaction_ind*.

- But, netCDF is based on C conventions and the array declaration is again turned around in there:
- ```
int reaction_reagent(reaction_ind, reagent_ind) ;
```

The representation of the data in the file is consistent with that:

```
reaction_reagent =
 2, 3,
 2, 5,
 5, 3,
 2, 4,
```

and so on with each line specifying the reagents for a given reaction, and one line appearing for each reaction.

For more details on array definitions and the macros used to carry them out, see the documentation in *array.hweb*.

The only instance in which the macro representation of an array cannot be used is in the invocation of memory allocation arrays. These must contain the names of the actual common variables.

Throughout the code there are a handful of large, multidimensional data objects. Frequently appearing are the arrays containing the reaction and plasma-material interaction data, e.g., *reaction\_handling*, and the arrays containing the tally and output data, e.g., *output\_grp*. Because these dimensions are “ragged”, direct representation as multi-dimensional FORTRAN arrays is not convenient. Instead, a large, one dimensional array is dimensioned, and pointers or base address arrays used to identify the portion of the array corresponding to a particular combination of subscript values. Specific macros are invoked at setup to compute the size of the array and to establish the base addresses. Another macro is used to mimic the multi-dimensional representation for the reader’s benefit. However, the tally and output arrays are treated differently, because of the presence of additional regular structure in them. See the specific section below on the output class for more details.

## 2 Class Attribute Descriptions

Have moved this documentation into the header files themselves (inserted into this file by FWEB) to facilitate updating documentation with code changes.

## 3 Class definitions

[/u/dstotler/degas2/src/classes.hweb]

```
$Id: classes.hweb,v 1.22 2001/11/16 15:37:41 dstotler Exp $
```

## 4 Element definitions

[/u/dstotler/degas2/src/element.hweb]

```
$Id: element.hweb,v 1.9 1999/03/25 20:46:17 dstotler Exp $
```

[/u/dstotler/degas2/src/element.hweb] Information about elements is held in arrays in common blocks. An element is identified by an integer indexing into these arrays.

```
"classes.f" 4.1 ≡
 @f el_decl integer
 @m el_args(x) x
 @m el_dummy(x) x
 @m el_decl(x) integer x
 @m el_copy(x,y) y = x
 @m el_check(x) (x > 0 ∧ x ≤ el_num)
 @m el_name(x) element_name_x
 @m el_sy(x) element_sy_x
 @m el_m(x) element_m_x
 @m el_z(x) element_z_x
 @m el_lookup(sy) string_lookup(sy, element_sy, el_num)
```

[/u/dstotler/degas2/src/element.hweb] Length specifications.

```
"classes.f" 4.2 ≡
 @m el_sy_len 3
 @m el_name_len 16
```

[/u/dstotler/degas2/src/element.hweb] Variable definitions.

```
"classes.f" 4.3 ≡
 package_init(el)
 define_dimen_pk(el, element_symbol_string, el_sy_len)
 define_dimen_pk(el, element_name_string, el_name_len)
 define_var_pk(el, el_num, INT)
 define_dimen_pk(el, element_ind, el_num)
 define_varp_pk(el, element_name, CHAR, element_name_string, element_ind)
 define_varp_pk(el, element_sy, CHAR, element_symbol_string, element_ind)
 define_varp_pk(el, element_z, INT, element_ind)
 define_varp_pk(el, element_m, FLOAT, element_ind)
 define_varlocal_pk(el, element_version, CHAR, string)
 package_end(el)
```

## 5 Element class attribute descriptions

[/u/dstotler/degas2/src/element.hweb]

[/u/dstotler/degas2/src/element.hweb] This defines elements. Prefix is *el*.

The identifier *element* is an integer variable.

*el\_name(element)* Returns the long name (length *el\_name\_len*).

*el\_sy(element)* Returns the symbolic name (length *el\_sy\_len*).

*el\_z(element)* Returns the charge number.

*el\_m(element)* Returns the mass number (in amu).

[/u/dstotler/degas2/src/element.hweb] Routines using elements.

*el\_lookup(symbol)* Returns the integer element number corresponding to the symbolic name *symbol*.

## 6 Species definitions

[/u/dstotler/degas2/src/species.hweb]

\$Id: species.hweb,v 1.11 1999/03/25 20:47:37 dstotler Exp \$

[/u/dstotler/degas2/src/species.hweb] Information about species is held in arrays in common blocks. An species is identified by an integer indexing into these arrays.

```
"classes.f" 6.1 ≡
 @f sp_decl integer
 @m sp_args(x) x
 @m sp_dummy(x) x
 @m sp_decl(x) integer x
 @m sp_copy(x,y) y = x
 @m sp_check(x) (x > 0 ∧ x ≤ sp_num)
 @m sp_name(x) species_name_x
 @m sp_sy(x) species_sy_x
 @m sp_m(x) species_m_x
 @m sp_z(x) species_z_x
 @m sp_ncomp(x) species_ncomp_x
 @m sp_generic(x) species_generic_x
 @m sp_multiplicity(x) species_multiplicity_x
 @m sp_el(x,i) species_el_i_x
 @m sp_count(x,i) species_count_i_x
 @m sp_lookup(sy) string_lookup(sy, species_sy, sp_num)
```

[/u/dstotler/degas2/src/species.hweb] Length specifications.

```
"classes.f" 6.2 ≡
 @m sp_sy_len 8
 @m sp_name_len 32
 @m sp_ncomp_max 10
```

[/u/dstotler/degas2/src/species.hweb] Variable definitions.

```
"classes.f" 6.3 ≡
 package_init(sp)
 define_dimen_pk(sp, species_symbol_string, sp_sy_len)
 define_dimen_pk(sp, species_name_string, sp_name_len)
 define_var_pk(sp, sp_num, INT)
 define_dimen_pk(sp, species_ind, sp_num)
 define_dimen_pk(sp, species_comp_ind, sp_ncomp_max)
 define_varp_pk(sp, species_name, CHAR, species_name_string, species_ind)
 define_varp_pk(sp, species_sy, CHAR, species_symbol_string, species_ind)
 define_varp_pk(sp, species_z, INT, species_ind)
 define_varp_pk(sp, species_m, FLOAT, species_ind)
 define_varp_pk(sp, species_ncomp, INT, species_ind)
 define_varp_pk(sp, species_generic, INT, species_ind)
 define_varp_pk(sp, species_multiplicity, INT, species_ind)
 define_varp_pk(sp, species_el, INT, species_comp_ind, species_ind)
 define_varp_pk(sp, species_count, INT, species_comp_ind, species_ind)
 define_varlocal_pk(sp, species_version, CHAR, string)
 package_end(sp)
```

## 7 Species class attribute descriptions

[/u/dstotler/degas2/src/species.hweb]

[/u/dstotler/degas2/src/species.hweb] Define species. Prefix is *sp*.

The identifier *species* is an integer variable.

*sp\_name(species)* Returns the long name (length *sp\_name\_len*).

*sp\_sy(species)* Returns the symbolic name (length *sp\_sy\_len*).

*sp\_m(species)* Returns the mass (in kg).

*sp\_z(species)* Returns the charge number.

*sp\_ncomp(species)* Returns the number of components in a species (maximum size *sp\_ncomp\_max*).

*sp\_generic(species)* Is the integer species number of the isotopic equivalent to *species* which has been designated in the species input as the archetype or “generic” member of that family.

*sp\_multiplicity(species)* Number of different ways in which the isotopically equivalent elements in *species* could be arranged.

*sp\_el(species, i)* Returns the element for the *i*th component.

*sp\_count(species, i)* Returns the count of the *i*th component.

[/u/dstotler/degas2/src/species.hweb] Routines using species.

*sp\_lookup(symbol)* returns the integer species number corresponding to the symbolic name *symbol*.

## 8 Particle definitions

[/u/dstotler/degas2/src/particle.hweb]

\$Id: particle.hweb,v 1.10 1999/03/25 20:46:47 dstotler Exp \$

[/u/dstotler/degas2/src/particle.hweb] An particle is identified by a string tag.

```
"classes.f" 8.1 ≡
@f pt_decl integer
@f pt_decls integer

@m pt_args(x)
 pt_sp(x), pt_test(x), pt_t(x), pt_w(x), lc_args(pt_loc(x)), pt_v(x)1, pt_type(x), pt_author(x)

@m pt_dummy(x)
 pt_sp(x), pt_test(x), pt_t(x), pt_w(x), lc_dummy(pt_loc(x)), pt_v(x), pt_type(x), pt_author(x)

@m pt_decl(x) sp_decl(pt_sp(x))
 pr_test_decl(pt_test(x))
 real pt_t(x), pt_w(x), pt_v(x)3
 lc_decl(pt_loc(x))
 integer pt_type(x), pt_author(x)

@m pt_decls external particle_track
 logical particle_track

@m pt_copy(x,y) pt_sp(y) = pt_sp(x)
 pt_test(y) = pt_test(x)
 pt_t(y) = pt_t(x)
 pt_w(y) = pt_w(x)
 vc_copy(pt_v(x), pt_v(y))
 lc_copy(pt_loc(x), pt_loc(y))
 pt_type(y) = pt_type(x)
 pt_author(y) = pt_author(x)

@m pt_check(x)
 (sp_check(pt_sp(x)) ∧ pt_w(x) ≥ 0 ∧ lc_check(pt_loc(x))) ∧ pr_test_check(pt_test(x)) ∧ pr_test(pt_test(x)) ≡
 pt_sp(x) ∧ (pt_author(x) > 0 ∧ pt_author(x) ≤ so_type_num + pr_reaction_num + pr_pmi_num)

@m pt_sp(x) pt_sp1(x)
@m pt_sp1(x) species_###x
@m pt_test(x) pt_test1(x)
@m pt_test1(x) test_###x
@m pt_t(x) pt_t1(x)
@m pt_t1(x) time_###x
@m pt_w(x) pt_w1(x)
@m pt_w1(x) weight_###x
@m pt_loc(x) pt_loc1(x)
@m pt_loc1(x) x
@m pt_v(x) pt_v1(x)
@m pt_v1(x) velocity_###x
@m pt_type(x) type_###x
@m pt_author(x) pt_author1(x)
@m pt_author1(x) author_###x
```

[/u/dstotler/degas2/src/particle.hweb] Constant definitions of particle types.

```
"classes.f" 8.2 ≡
@m pt_geometry 0
@m pt_neutral 1
@m pt_ion 2

@m zone_exit 0
@m cell_exit
```

[/u/dstotler/degas2/src/particle.hweb] Some macros to manipulate particle tracks.

```
"classes.f" 8.3 ≡
@m pt_track(tmax, t, x) particle_track(tmax, t, pt_args(x))
@m pt_thru_face(x) lc_thru_face(pt_loc(x))
@m pt_specular(x) call surface_specular(lc_face(pt_loc(x)), lc_x(pt_loc(x))1, pt_v(x)1, pt_v(x)1)
 lc_face(pt_loc(x)) = 0

@m pt_author_so(x, type) pt_author(x) = type

@m pt_author_rc(x, reac) pt_author(x) = so_type_num + reac

@m pt_author_pmi(x, pmi) pt_author(x) = so_type_num + pr_reaction_num + pmi

@m pt_init(x, sy, source) pt_sp(x) = sp_lookup(sy)
 pt_test(x) = pr_test_lookup(pt_sp(x))
 if (sp_m(pt_sp(x)) ≡ 0) then
 pt_type(x) = pt_geometry
 else if (sp_z(pt_sp(x)) ≡ 0) then
 pt_type(x) = pt_neutral
 pt_author_so(x, so_type(source))
 else
 pt_type(x) = pt_ion
 pt_author_so(x, so_type(source))
 end if
 pt_t(x) = zero
 pt_w(x) = one

@m pt_scatter_thru(x, w)
 call surface_reflect(−lc_face(pt_loc(x)), lc_x(pt_loc(x))1, pt_v(x)1, w, pt_v(x)1)
 lc_thru_face(pt_loc(x))
```

## 9 Particle class attribute descriptions

[/u/dstotler/degas2/src/particle.hweb]

[/u/dstotler/degas2/src/particle.hweb] Define particles. Prefix is *pt*.

The identifier *particle* is a string tag.

*pt\_sp(particle)* is the species of the particle.

*pt\_test(particle)* is the test particle index.

*pt\_t(particle)* is the time for this particle.

*pt\_w(particle)* is the statistical weight of the particle.

*pt\_loc(particle)* is the location of the particle (see below).

*pt\_v(particle)* is the velocity of the particle.

*pt\_type(particle)* is the type of tracking to be done for the particle.

*pt\_author(particle)* identifies the process (reaction, plasma-material interaction, or source) responsible for setting the particle's current velocity.

[/u/dstotler/degas2/src/particle.hweb] Routines involving particles.

*pt\_track(tmax, t, particle)* Logical function which tracks *particle* for time *tmax*, returning  $\mathcal{F}$  if a zone boundary is encountered. *pt\_thru\_face(particle)* updates the location of *particle* when it traverses a cell face.

*pt\_specular(particle)* If at a face, executes a specular reflection of *particle*.

*pt\_author\_so(particle, type)* Sets the author of *particle* to source *type*.

*pt\_author\_rc(particle, rc)* Sets the author of *particle* to reaction *rc*.

*pt\_author\_pm(particle, pm)* Sets the author of *particle* to plasma-material interaction *pm*.

*pt\_init(particle, sy, source)* Sets properties for *particle*, of species symbol *sy*, generated by source group *source*.

*pt\_scatter\_thru(particle, v)* Reflects *particle* off surface into velocity *v* relative to surface.

## 10 Location definitions

[/u/dstotler/degas2/src/location.hweb]

\$Id: location.hweb,v 1.10 1999/03/25 20:46:26 dstotler Exp \$

A location is identified by a string tag.

"classes.f" 10 ≡

**@f** *lc\_decl integer*

**@f** *lc\_decls integer*

**@m** *lc\_args(x) lc\_x(x)<sub>1</sub>, lc\_cell(x), lc\_zone(x), lc\_face(x), lc\_cell\_next(x), lc\_zone\_next(x),  
lc\_sector(x), lc\_sector\_next(x)*

**@m** *lc\_dummy(x) lc\_x(x), lc\_cell(x), lc\_zone(x), lc\_face(x), lc\_cell\_next(x), lc\_zone\_next(x),  
lc\_sector(x), lc\_sector\_next(x)*

**@m** *lc\_decl(x) real lc\_x(x)<sub>3</sub>  
integer lc\_cell(x), lc\_zone(x), lc\_face(x), lc\_cell\_next(x), lc\_zone\_next(x), lc\_sector(x),  
lc\_sector\_next(x)*

**@m** *lc\_decls external locate\_point, check\_location  
integer locate\_point  
logical check\_location*

**@m** *lc\_copy(x,y) vc\_copy(lc\_x(x), lc\_x(y))  
lc\_cell(y) = lc\_cell(x)  
lc\_zone(y) = lc\_zone(x)  
lc\_face(y) = lc\_face(x)  
lc\_cell\_next(y) = lc\_cell\_next(x)  
lc\_zone\_next(y) = lc\_zone\_next(x)  
lc\_sector(y) = lc\_sector(x)  
lc\_sector\_next(y) = lc\_sector\_next(x)*

**@m** *lc\_check(x) check\_location(lc\_args(x))*

**@m** *lc\_x(x) lc\_x1(x)*

**@m** *lc\_x1(x) pos\_##x*

**@m** *lc\_cell(x) lc\_cell1(x)*

**@m** *lc\_cell1(x) cell\_##x*

**@m** *lc\_zone(x) lc\_zone1(x)*

**@m** *lc\_zone1(x) zone\_##x*

**@m** *lc\_face(x) lc\_face1(x)*

**@m** *lc\_face1(x) surface\_##x*

**@m** *lc\_cell\_next(x) lc\_cell\_next1(x)*

**@m** *lc\_cell\_next1(x) cell\_next\_##x*

**@m** *lc\_zone\_next(x) lc\_zone\_next1(x)*

**@m** *lc\_zone\_next1(x) zone\_next\_##x*

**@m** *lc\_sector(x) lc\_sector1(x)*

**@m** *lc\_sector1(x) sector\_##x*

**@m** *lc\_sector\_next(x) lc\_sector\_next1(x)*

**@m** *lc\_sector\_next1(x) sector\_next\_##x*

```

[/u/dstotler/degas2/src/location.hweb] Some interface routines

"classes.f" 10.1 ≡
 @m lc_set(x) lc_cell(x) = locate_point(lc_x(x), lc_zone(x))
 lc_face(x) = 0

 @m lc_set_a(x) lc_set(x)
 lc_cell_next(x) = 0
 lc_zone_next(x) = 0
 lc_sector(x) = 0
 lc_sector_next(x) = 0

 @m lc_thru_face(x) if (lc_face(x) ≠ 0) then
 lc_face(x) = 0
 lc_cell(x) = lc_cell_next(x)
 lc_zone(x) = lc_zone_next(x)
 lc_sector(x) = lc_sector_next(x)
 end if

[/u/dstotler/degas2/src/location.hweb] Some constants for the geometry package.

"classes.f" 10.2 ≡
 @m geom_epsilon const(1.0, -8) // A small distance 10 nm.
 @m geom_infinity const(1.0, 16)
 // A large distance—about a light year. Also a long time—about a billion years.
 @m geom_large const(1000.0) // A largish (non-infinite) distance 1 km.
 @m epsilon_angle const(1.0, -10) // A small angle.

```

## 11 Location class attribute descriptions

```

[/u/dstotler/degas2/src/location.hweb]

```

[/u/dstotler/degas2/src/location.hweb] Define a location. Prefix is *lc*.

The identifier *location* is a string tag.

*lc\_x(location)* Is the position.

*lc\_cell(location)* Is the cell number.

*lc\_zone(location)* Is the zone number.

*lc\_face(location)* Is the face of the cell if at a face (else 0).

*lc\_cell\_next(location)* Is the next cell number.

*lc\_zone\_next(location)* Is the next zone number.

*lc\_sector(location)* Is the present sector number if at a sector; else 0.

*lc\_sector\_next(location)* If at or just past a sector adjacent to a second sector, contains the number of the latter; else 0.

[/u/dstotler/degas2/src/location.hweb] Routines using locations.

*lc\_set(location)* Sets other fields of *location* given the position.

*lc\_thru\_face(location)* Updates the fields of *location* for traversal of a cell face.

## 12 Flight definitions

[/u/dstotler/degas2/src/flight.hweb]

\$Id: flight.hweb,v 1.9 1999/03/25 20:58:30 dstotler Exp \$

[/u/dstotler/degas2/src/flight.hweb] A flight is identified by a string tag.

```
"classes.f" 12.1 ≡
 @f fl_decl integer
 @f fl_common integer

 @m fl_args(x) fl_number(x), fl_source(x), pt_args(fl_origin(x)), rn_args(fl_rand(x)),
 pt_args(fl_stack(x)1), fl_pointer(x)

 @m fl_dummy(x) fl_number(x), fl_source(x), pt_dummy(fl_origin(x)), rn_dummy(fl_rand(x)),
 pt_dummy(fl_stack(x)), fl_pointer(x)

 @m fl_decl(x) integer fl_number(x), fl_source(x), fl_pointer(x)
 pt_decl(fl_origin(x))
 rn_decl(fl_rand(x))
 pt_decl(fl_stack(x) fl_stack_max)

 @m fl_common integer fl_temp

 @m fl_copy(x, y) fl_number(y) = fl_number(x)
 fl_source(y) = fl_source(x)
 pt_copy(fl_origin(x), fl_origin(y))
 rn_copy(fl_rand(x), fl_rand(y))
 fl_pointer(x) = fl_pointer(y)
 do fl_temp = 1, fl_pointer(x)
 pt_copy(fl_stack(x) fl_temp, fl_stack(y) fl_temp)
 end do

 @m fl_check(x) (fl_number(x) ≥ 0 ∧ (fl_source(x) > 0 ∧ fl_source(x) ≤ so_grps) ∧ fl_pointer(x) >
 0 ∧ fl_pointer(x) ≤ fl_stack_max)

 @m fl_number(x) number_##x
 @m fl_source(x) source_##x
 @m fl_origin(x) origin_##x
 @m fl_rand(x) x
 @m fl_stack(x) stack_##x
 @m fl_pointer(x) pointer_##x
 @m fl_current(x) fl_stack(x) fl_pointer(x)
```

[/u/dstotler/degas2/src/flight.hweb] Length specifications.

```
"classes.f" 12.2 ≡
 @m fl_stack_max 40
```

[/u/dstotler/degas2/src/flight.hweb] Macros to manipulate flights.

```
"classes.f" 12.3 ≡
 @m fl_label(x, num, isource) fl_number(x) = num
 fl_source(x) = isource

 @m fl_init(x, sy) pt_init(fl_origin(x), sy, fl_source(x))
 fl_pointer(x) = 1
 pt_copy(fl_origin(x), fl_current(x))
```

## 13 Flight class attribute descriptions

[/u/dstotler/degas2/src/flight.hweb]

[/u/dstotler/degas2/src/flight.hweb] Define a flight. Prefix is *fl*.

The identifier *flight* is a string tag.

*fl\_number(flight)* Is the flight number.

*fl\_source(flight)* Is the number of the source which gave rise to the flight.

*fl\_origin(flight)* Is the particle initiating the flight.

*fl\_rand(flight)* Is the random number information for the flight.

*fl\_stack(flight)* Is the stack of particles being processed.

*fl\_pointer(flight)* Is the pointer to the current particle (maximum size *fl\_stack\_max*).

*fl\_current(flight)* Is the current particle (same as *fl\_stack(flight)fl\_pointer(flight)*).

[/u/dstotler/degas2/src/flight.hweb] Routines using flights.

*fl\_label(flight, num, isource)* Sets *fl\_number(flight)* to *num* and

*fl\_source(flight)* To source number *isource*.

*fl\_init(flight, sy)* Sets up a current flight containing a particle of species *sy*, using *fl\_origin(flight)* for the other particle properties and *fl\_source(flight)* as the “author”.

## 14 Reaction definitions

[/u/dstotler/degas2/src/reaction.hweb]

\$Id: reaction.hweb,v 1.18 1999/04/20 20:13:04 dstotler Exp \$

[/u/dstotler/degas2/src/reaction.hweb] Information about reactions is held in arrays in common blocks. An reaction is identified by an integer indexing into these arrays.

```
"classes.f" 14.1 ≡
 @f rc_decl integer

 @m rc_generic_no 0
 @m rc_generic_yes 1

 @m rc_args(x) x
 @m rc_dummy(x) x
 @m rc_decl(x) integer x
 @m rc_copy(x,y) y = x
 @m rc_check(x) (x > 0 ∧ x ≤ rc_num)
 @m rc_name(x) reaction_name_x
 @m rc_sy(x) reaction_sy_x
 @m rc_emitter(x) reaction_emitter_x
 @m rc_reagent_num(x) reaction_reagent_num_x
 @m rc_gen(x) reaction_generic_x
 @m rc_product_num(x) reaction_product_num_x
 @m rc_reagent(x,i) reaction_reagent_{i,x}
 @m rc_product(x,i) reaction_product_{i,x}
 @m rc_filename(x) reaction_filename_x
 @m rc_reaction_type(x) reaction_type_x
 @m rc_lookup(sy) string_lookup(sy, reaction_sy, rc_num)
```

[/u/dstotler/degas2/src/reaction.hweb] Length specifications.

```
"classes.f" 14.2 ≡
 @m rc_sy_len 24
 @m rc_name_len 80
 @m rc_type_len 32
 @m rc_reagent_max 2
 @m rc_product_max 4
```

[/u/dstotler/degas2/src/reaction.hweb] Variable definitions.

```
"classes.f" 14.3 ≡
 package_init(rc)
 define_dimen_pk(rc, reaction_symbol_string, rc_sy_len)
 define_dimen_pk(rc, reaction_name_string, rc_name_len)
 define_dimen_pk(rc, reaction_type_string, rc_type_len)
 define_var_pk(rc, rc_num, INT)
 define_dimen_pk(rc, reaction_ind, rc_num)
 define_dimen_pk(rc, reaction_filename_string, FILELEN)
 define_dimen_pk(rc, reagent_ind, rc_reagent_max)
 define_dimen_pk(rc, product_ind, rc_product_max)
 define_varp_pk(rc, reaction_name, CHAR, reaction_name_string, reaction_ind)
 define_varp_pk(rc, reaction_type, CHAR, reaction_type_string, reaction_ind)
 define_varp_pk(rc, reaction_sy, CHAR, reaction_symbol_string, reaction_ind)
 define_varp_pk(rc, reaction_emitter, INT, reaction_ind)
 define_varp_pk(rc, reaction_reagent_num, INT, reaction_ind)
 define_varp_pk(rc, reaction_generic, INT, reaction_ind)
 define_varp_pk(rc, reaction_product_num, INT, reaction_ind)
 define_varp_pk(rc, reaction_reagent, INT, reagent_ind, reaction_ind)
 define_varp_pk(rc, reaction_product, INT, product_ind, reaction_ind)
 define_varp_pk(rc, reaction_filename, CHAR, reaction_filename_string, reaction_ind)
 define_varlocal_pk(rc, reaction_version, CHAR, string)
 package_end(rc)
```

## 15 Reaction class attribute descriptions

[/u/dstotler/degas2/src/reaction.hweb]

[/u/dstotler/degas2/src/reaction.hweb] Define reactions. Prefix is *rc*.

The identifier *reaction* is an integer variable.

*rc\_name(reaction)* Is the name of the reaction (length *rc\_name\_len*).

*rc\_sy(reaction)* Is the symbolic name of the reaction (length *rc\_sy\_len*).

*rc\_emitter(reaction)* Is the reagent (if negative) or product (if positive) number to be associated with a photon emitted from this reaction.

*rc\_reagent\_num(reaction)* Is the number of reagents in the reaction (maximum value *rc\_reagent\_max*).

*rc\_reagent(reaction, i)* Is the *i*th species reagent in the reaction.

*rc\_gen(reaction)* Contains the value *rc\_generic\_yes* is this reaction (and its data) can be applied to species isotopically equivalent to *rc\_reagent(reaction, i)*; the value *rc\_generic\_no* indicates that the reaction is species-specific.

*rc\_product\_num(reaction)* Is the number of products (maximum value *rc\_product\_max*).

*rc\_product(reaction, i)* Is the species of the *i*th product.

*rc\_filename(reaction)* netCDF filename containing the reaction rate and handling data (class *xs*) for the reaction.

*rc\_reaction\_type(reaction)* The reaction type determines which routines will be used to determine the reaction product velocities during run time.

[/u/dstotler/degas2/src/reaction.hweb] Routines using reactions.

*rc\_lookup(symbol)* returns the integer reaction number corresponding to the symbolic name *symbol*.

## 16 Materials definitions

[/u/dstotler/degas2/src/materials.hweb]

\$Id: materials.hweb,v 1.5 1999/03/25 20:46:30 dstotler Exp \$

[/u/dstotler/degas2/src/materials.hweb] Information about materials is held in arrays in common blocks. A material is identified by an integer indexing into these arrays.

```
"classes.f" 16.1 ≡
 @f ma_decl integer
 @m ma_args(x) x
 @m ma_dummy(x) x
 @m ma_decl(x) integer x
 @m ma_copy(x, y) y = x
 @m ma_check(x) (x > 0 ∧ x ≤ ma_num)
 @m ma_name(x) materials_name_x
 @m ma_sy(x) materials_sy_x
 @m ma_lookup(sy) string_lookup(sy, materials_sy, ma_num)
```

[/u/dstotler/degas2/src/materials.hweb] Length specifications.

```
"classes.f" 16.2 ≡
 @m ma_sy_len 8
 @m ma_name_len 32
```

[/u/dstotler/degas2/src/materials.hweb] Variable definitions.

```
"classes.f" 16.3 ≡
 package_init(ma)
 define_dimen_pk(ma, materials_symbol_string, ma_sy_len)
 define_dimen_pk(ma, materials_name_string, ma_name_len)
 define_var_pk(ma, ma_num, INT)
 define_dimen_pk(ma, materials_ind, ma_num)

 define_varp_pk(ma, materials_name, CHAR, materials_name_string, materials_ind)
 define_varp_pk(ma, materials_sy, CHAR, materials_symbol_string, materials_ind)
 define_varlocal_pk(ma, materials_version, CHAR, string)
 package_end(ma)
```

## 17 Materials class attribute descriptions

[/u/dstotler/degas2/src/materials.hweb]

[/u/dstotler/degas2/src/materials.hweb] Define a material. Prefix is *ma*.

The identifier *material* is an integer variable. The properties of the materials are implicitly defined through the physical behavior of the plasma-material interactions. As a result, this class is really just an arbitrary label; a given instance of the class is generated only in conjunction with one or more PMI which refer to it.

*ma\_name(material)* Returns the long name (length *ma\_name\_len*).

*ma\_sy(material)* Returns the symbolic name (length *ma\_sy\_len*).

[/u/dstotler/degas2/src/materials.hweb] Internal variables in the materials class.

*ma\_num* Number of material instances.

[/u/dstotler/degas2/src/materials.hweb] Routines using materials.

*ma\_lookup(symbol)* Returns the integer materials number corresponding to the symbolic name *symbol*.

## 18 Plasma-Material Interaction (PMI) definitions

[/u/dstotler/degas2/src/pmi.hweb]

\$Id: pmi.hweb,v 1.9 1999/03/25 20:46:51 dstotler Exp \$

Information about plasma-material interaction processes is held in arrays in common blocks. A plasma-material interaction process is identified by an integer indexing into these arrays.

```
"classes.f" 18 ≡
 @f pm_decl integer

 @m pm_generic_no 0
 @m pm_generic_yes 1

 @m pm_args(x) x
 @m pm_dummy(x) x
 @m pm_decl(x) integer x
 @m pm_copy(x,y) y = x
 @m pm_check(x) (x > 0 ∧ x ≤ pm_num)
 @m pm_name(x) pmi_name_x
 @m pm_sy(x) pmi_sy_x
 @m pm_reagent(x) pmi_reagent_x
 @m pm_materials(x) pmi_materials_x
 @m pm_gen(x) pmi_generic_x
 @m pm_product_num(x) pmi_product_num_x
 @m pm_product(x,i) pmi_product_i_x
 @m pm_filename(x) pmi_filename_x
 @m pm_pmi_type(x) pmi_type_x
 @m pm_lookup(sy) string_lookup(sy, pmi_sy, pm_num)
```

[/u/dstotler/degas2/src/pmi.hweb] Length specifications.

```
"classes.f" 18.1 ≡
 @m pm_sy_len 24
 @m pm_name_len 80
 @m pm_type_len 32
 @m pm_product_max 4
```

[/u/dstotler/degas2/src/pmi.hweb] Variable definitions.

```
"classes.f" 18.2 ≡
 package_init(pm)

 define_dimen_pk(pm, pmi_symbol_string, pm_sy_len)
 define_dimen_pk(pm, pmi_name_string, pm_name_len)
 define_dimen_pk(pm, pmi_type_string, pm_type_len)
 define_var_pk(pm, pm_num, INT)
 define_var_pk(pm, pm_ignore, INT)
 define_dimen_pk(pm, pmi_ind, pm_num)
 define_dimen_pk(pm, pmi_filename_string, FILELEN)
 define_dimen_pk(pm, pmi_product_ind, pm_product_max)

 define_varp_pk(pm, pmi_name, CHAR, pmi_name_string, pmi_ind)
 define_varp_pk(pm, pmi_type, CHAR, pmi_type_string, pmi_ind)
 define_varp_pk(pm, pmi_sy, CHAR, pmi_symbol_string, pmi_ind)
 define_varp_pk(pm, pmi_reagent, INT, pmi_ind)
 define_varp_pk(pm, pmi_materials, INT, pmi_ind)
 define_varp_pk(pm, pmi_generic, INT, pmi_ind)
@if 0
 define_varp_pk(pm, pmi_bound_reagent, CHAR, pmi_materials_string, pmi_ind)
 define_varp_pk(pm, pmi_bound_product, CHAR, pmi_materials_string, pmi_ind)
@endif
 define_varp_pk(pm, pmi_product_num, INT, pmi_ind)
 define_varp_pk(pm, pmi_product, INT, pmi_product_ind, pmi_ind)
 define_varp_pk(pm, pmi_filename, CHAR, pmi_filename_string, pmi_ind)
 define_varlocal_pk(pm, pmi_version, CHAR, string)
 package_end(pm)
```

## 19 PMI class attribute descriptions

[/u/dstotler/degas2/src/pmi.hweb]

[/u/dstotler/degas2/src/pmi.hweb] Define plasma-material interactions. Prefix is *pm*.

The identifier *pmi* is an integer variable.

*pm\_name(pmi)* Returns the long name (length *pm\_name\_len*).

*pm\_sy(pmi)* Returns the symbolic name (length *pm\_sy\_len*).

*pm\_reagent(pmi)* Returns the species index (class *sp*) of the incident projectile.

*pm\_materials(pmi)* Returns the materials index (class *ma*) of the target material.

*pm\_gen(pmi)* Contains the value *pmi\_generic\_yes* if this *pmi* (and its data) can be applied to species isotopically equivalent to *pm\_reagent(pmi)*; the value *pmi\_generic\_no* indicates that the *pmi* is species-specific.

*pm\_product\_num(pmi)* Number of product particles.

*pm\_product(pmi, i)* Species index (class *sp*) of the *i*th product particle,  $i = 1 \rightarrow pm\_product\_num(pmi)$ .

*pm\_filename(pmi)* Name of netCDF file containing the data (class *pf*) for this *pmi*.

*pm\_pmi\_type(pmi)* Returns the type of plasma-material interaction (length *pm\_type\_len*).

[/u/dstotler/degas2/src/pmi.hweb] Internal variables in the PMI class.

*pm\_num* Number of PMI in the reference database.

[/u/dstotler/degas2/src/pmi.hweb] Routines using PMI.

*pm\_lookup(symbol)* Returns the integer PMI number corresponding to the symbolic name *symbol*.

## 20 Background definitions

[/u/dstotler/degas2/src/background.hweb]

\$Id: background.hweb,v 1.16 2001/11/16 15:36:24 dstotler Exp \$

[/u/dstotler/degas2/src/background.hweb] Information about background species is held in arrays in common blocks. An background is identified by an integer indexing into these arrays.

```
"classes.f" 20.1 ≡
 @f bk_decl integer

 @m bk_args(x) x
 @m bk_dummy(x) x
 @m bk_decl(x) integer x
 @m bk_copy(x, y) y = x
 @m bk_check(x) (x > 0 ∧ x ≤ bk_num)
 @m bk_n(x, i) background_nx, zn_pointer(i)
 @m bk_temp(x, i) background_tempx, zn_pointer(i)
 @m bk_v(x, i) background_vx, zn_pointer(i)

 @m plasma_coords_cartesian 1 // v holds the x, y, and z components.
 @m plasma_coords_cylindrical 2 // v holds the R, φ, and z components.

 @m bk_mx 120 // Dimensions for plasma code arrays
 @m bk_my 120
```

[/u/dstotler/degas2/src/background.hweb] Move velocity from external reference plane to internal position. In cases with some cylindrical symmetry and in which the background velocities are assumed to be symmetric about the Z axis, the velocities need to be explicitly transformed back and forth between the reference plane (zero toroidal angle, or y = 0). The *coord* value *plasma\_coords\_cylindrical* denotes this symmetry option (somewhat unclearly). There may still be cases with no geometry symmetry, *geometry\_symmetry\_none*, that need this transformation, but disallow them for now with an assert. Note that with *geometry\_symmetry\_cylindrical* the velocities are always rotated back to a reference plane (zero toroidal angle) where the cartesian and cylindrical velocities are identical. Hence, the *coord* parameter is used only in cases without symmetry.

```
"classes.f" 20.2 ≡
 @m v_ext_to_int(x, v, v_t, sym, coord)
 if ((sym ≡ geometry_symmetry_cylindrical ∨ sym ≡ geometry_symmetry_cyl_hw ∨ sym ≡
 geometry_symmetry_cyl_section) ∧ (coord ≡ plasma_coords_cylindrical) ∧ (x12 + x22 > zero))
 then
 v_t1 = (v1 * x1 - v2 * x2) / sqrt(x12 + x22)
 v_t2 = (v1 * x2 + v2 * x1) / sqrt(x12 + x22)
 v_t3 = v3
 else
 assert(¬((sym ≡ geometry_symmetry_none) ∧ (coord ≡ plasma_coords_cylindrical)))
 vc_copy(v, v_t)
 end if
```

[/u/dstotler/degas2/src/background.hweb] Move velocity from internal position external reference plane.

```
"classes.f" 20.3 ≡
@m v_int_to_ext(x, v, v_t, sym, coord)
 if ((sym ≡ geometry_symmetry_cylindrical ∨ sym ≡ geometry_symmetry_cyl_hw ∨ sym ≡
 geometry_symmetry_cyl_section) ∧ (coord ≡ plasma_coords_cylindrical) ∧ (x12 + x22 > zero))
 then
 v_t1 = (v1 * x1 + v2 * x2) / sqrt(x12 + x22)
 v_t2 = (−v1 * x2 + v2 * x1) / sqrt(x12 + x22)
 v_t3 = v3
 else
 assert(¬((sym ≡ geometry_symmetry_none) ∧ (coord ≡ plasma_coords_cylindrical)))
 vc_copy(v, v_t)
 end if
```

[/u/dstotler/degas2/src/background.hweb] Variable definitions.

```
"classes.f" 20.4 ≡
 package_init(bk)
 define_var_pk(bk, bk_num, INT)
 define_dimen_pk(bk, background_ind, bk_num)
 define_dimen_pk(bk, bk_plasma_ind, zone_type_num zn_plasma)
 define_var_pk(bk, background_coords, INT)
 define_varp_pk(bk, background_n, FLOAT, background_ind, bk_plasma_ind)
 define_varp_pk(bk, background_v, FLOAT, vector, background_ind, bk_plasma_ind)
 define_varp_pk(bk, background_temp, FLOAT, background_ind, bk_plasma_ind)
 package_end(bk)
```

## 21 Background class attribute descriptions

[/u/dstotler/degas2/src/background.hweb]

[/u/dstotler/degas2/src/background.hweb] Store background properties. Prefix is *bk*.

The identifier *background* is an integer variable.

*bk\_n*(*background*, *i*) Is the density in the *i*th zone.

*bk\_v*(*background*, *i*) Is the vector velocity in the *i*th zone.

*bk\_temp*(*background*, *i*) Is the temperature in the *i*th zone.

*background\_coords* Specifies the coordinate system used to represent *bk\_v*, either *plasma\_coords\_cartesian* or *plasma\_coords\_cylindrical*.

## 22 Source definitions

[/u/dstotler/degas2/src/sources.hweb]

\$Id: sources.hweb,v 1.11 2001/07/16 21:04:18 dstotler Exp \$

[/u/dstotler/degas2/src/sources.hweb] Specification of the sources. At present only surface and volume sources are handled.

```
"classes.f" 22.1 ≡
 @f so_decl integer

 @m so_args(x) x

 @m so_dummy(x) x

 @m so_decl(x) integer x

 @m so_copy(x,y) y = x

 @m so_check(x) (x > 0 ∧ x ≤ so_grps)

 @m so_seg_check(x) (x > 0 ∧ x ≤ so_seg_tot)

 @m so_base(i) source_base_ptr_i
 @m so_nseg(i) source_num_segments_i
 @m so_type(i) source_type_i
 @m so_geom(i) source_geometry_i
 @m so_nflights(i) source_num_flights_i
 @m so_chkpt(i) source_num_checkpoints_i
 @m so_species(i) source_species_i
 @m so_root_sp(i) source_root_species_i
 @m so_tot_curr(i) source_total_current_i
 @m so_wt_norm(i) source_weight_norm_i
 @m so_scale(i) source_scale_factor_i
 @m so_name(i) source_name_i
 @m so_params(i,p) source_parameters_p, i

 @m so_plate 1 // Source types
 @m so_puff 2
 @m so_recomb 3
 @m so_type_num 3
 @m so_name_len 10

 @m so_point 0 // Source geometries
 @m so_line 1
 @m so_surface 2
 @m so_volume 3

 @m so_random 0 // Source sampling schemes
 @m so_direct 1

 @m so_param_unknown 0 // Indices for miscellaneous source parameters
 @m so_param_puff_temp 1
 @m so_param_max 1

 @m so_direct_mult half * (sqrt(const(5.)) - one)
```

[/u/dstotler/degas2/src/sources.hweb] Length of random seed string. Routines in *random.web* talk about seed strings needing to be at least 17 characters long.

```
"classes.f" 22.2 ≡
 @m so_seed_len 20
```

[/u/dstotler/degas2/src/sources.hweb] Standard initialization statements. These should eventually be in a “run control” dialog along with the number of flights, etc.

```
"classes.f" 22.3 ≡
 @m so_set_run_flags so_restart = FALSE
 so_seed_string = '12'
 so_spaced_seeds = FALSE
 so_seed_spacing = 100000000
 so_sampling = so_random
 so_name(so_plate) = 'plate'
 so_name(so_puff) = 'puff'
 so_name(so_recomb) = 'recomb'
 so_rel_wt_min = const(5., -1)
 so_rel_wt_max = const(2.)
 so_wt_norm_min = const(5., -1)
 so_wt_norm_max = const(2.)
```

[/u/dstotler/degas2/src/sources.hweb] Definition of the common blocks.

```
"classes.f" 22.4 ≡
 package_init(so)

 define_var_pk(so, so_grps, INT)
 define_var_pk(so, so_seg_tot, INT)
 define_var_pk(so, so_restart, INT)
 define_var_pk(so, so_spaced_seeds, INT)
 define_var_pk(so, so_seed_spacing, INT)
 define_var_pk(so, so_sampling, INT)
 define_var_pk(so, so_rel_wt_min, FLOAT)
 define_var_pk(so, so_rel_wt_max, FLOAT)
 define_var_pk(so, so_wt_norm_min, FLOAT)
 define_var_pk(so, so_wt_norm_max, FLOAT)
 define_dimen_pk(so, source_grp_ind, so_grps)
 define_dimen_pk(so, source_seg_ind, so_seg_tot)
 define_dimen_pk(so, so_seed_string_ind, so_seed_len)
 define_dimen_pk(so, so_name_ind, so_name_len)
 define_dimen_pk(so, so_type_ind, so_type_num)
 define_dimen_pk(so, so_param_ind, 0, so_param_max)

 define_var_pk(so, so_seed_string, CHAR, so_seed_string_ind)
 define_var_pk(so, source_name, CHAR, so_name_ind, so_type_ind)

 define_var_pk(so, source_base_ptr, INT, source_grp_ind)
 define_var_pk(so, source_num_segments, INT, source_grp_ind)
 define_var_pk(so, source_type, INT, source_grp_ind)
 define_var_pk(so, source_geometry, INT, source_grp_ind)
 define_var_pk(so, source_num_flights, INT, source_grp_ind)
 define_var_pk(so, source_num_checkpoints, INT, source_grp_ind)
 define_var_pk(so, source_species, INT, source_grp_ind)
 define_var_pk(so, source_root_species, INT, source_grp_ind)
 define_var_pk(so, source_parameters, FLOAT, so_param_ind, source_grp_ind)

 define_var_pk(so, source_total_current, FLOAT, source_grp_ind)
 define_var_pk(so, source_weight_norm, FLOAT, source_grp_ind)
 define_var_pk(so, source_scale_factor, FLOAT, source_grp_ind)

 define_var_pk(so, source_segment_ptr, INT, source_seg_ind)
 define_var_pk(so, source_current, FLOAT, source_seg_ind)
 define_var_pk(so, source_segment_rel_wt, FLOAT, source_seg_ind)
 define_var_pk(so, source_segment_prob_alias, FLOAT, source_seg_ind)
 define_var_pk(so, source_segment_ptr_alias, INT, source_seg_ind)
 define_varlocal_pk(so, so_direct_delta, FLOAT)

 package_end(so)
```

## 23 Source class attribute descriptions

[/u/dstotler/degas2/src/sources.hweb]

[/u/dstotler/degas2/src/sources.hweb] Define properties of sources. Prefix is *so*.

Individual source elements or “segments” are sorted into source “groups”. These groups may be aligned with the sector strata, if desired, for external labeling purposes. Both the source group and segment are represented by integer variables. The segment is a general concept meant to account for all possible source geometries. For example, axisymmetric surface and volume sources are presently treated. The segments in this case represent sectors and zones, respectively. All segments in a group are assumed to have the same type, geometry, and species.

Segment-specific data are stored in single 1-D arrays with the beginning of the portion corresponding to a particular group denoted by the *so\_base(igroup)* array.

A number of run control type variables and switches have been collected into this file, starting with the number of flights which sensibly belongs here. These may eventually be moved elsewhere or be assigned nontrivial values as part of a run setup procedure executed just prior to the running of the main DEGAS 2 code, *flighttest*.

*so\_base(igroup)* Pointer in arrays defined over all segments at which the segments of group *igroup* begin.

*so\_nseg(igroup)* Number of segments in group *igroup*.

*so\_type(igroup)* Physical process associated with group *igroup*. Presently treat plate (recycling), gas puff, and recombination.

*so\_geom(igroup)* Geometry of group *igroup*. Presently treat axisymmetric surface and volume sources. Although in general the geometry should be independent of the source type, practically we expect plate and puff sources to have surface geometry and recombination to be volume.

*so\_nflights(igroup)* Number of flights to be used for group *igroup* during the run. These may be set independently of the source current for each group. The variance in the observable parameters of interest due to a particular group should also be used to choose the number of flights.

*so\_chkpt(igroup)* Number of checkpoints which should be written out for group *igroup*. A value of 0 will result in no checkpoint files.

*so\_species(igroup)* Species index (species class) of group *igroup*.

*so\_root\_sp(igroup)* Root species that gives rise to the actual species sourced. In the cases of plate and recombination (and, perhaps, puff) sources, this is a background species providing the distribution function for the source.

*so\_tot\_curr(igroup)* Total current (particles per second) of all segments in group *igroup*.

*so\_wt\_norm(igroup)* Factor used to renormalize the total weight flown for group *igroup* to adjust for nonunity values of *source\_segment\_rel\_wt*.

*so\_scale(igroup)* Arbitrary scaling factor for the total current of group *igroup*. Setting this to a value  $\neq 1$  has the same impact as rescaling all of the segment currents by this factor. This provides additional flexibility for combining source groups since it can be applied after the run has been completed.

*so\_params(igroup, iparam)* Set of miscellaneous (real) parameters used to describe group *igroup*. The initial entry into this array is the gas puff temperature.

[/u/dstotler/degas2/src/sources.hweb] Internal variables in the sources class.

*so\_grps* Number of source groups in the problem.

*so\_seg\_tot* Total number of source segments in the problem.

*source\_segment\_ptr<sub>iseg</sub>* Integer pointer to the geometry element corresponding to segment *iseg*.

*source\_current<sub>iseg</sub>* Current (in particles per second) associated with segment *iseg*.

*source\_segment\_rel\_wt<sub>iseg</sub>* Arbitrary multiplier on the weight of flights launched from segment *iseg*. Nonunity values allow for stratified source sampling. Biasing of the results is eliminated by dividing the probability for selecting *iseg* by this factor. The factor *so\_wt\_norm* is also required for this purpose.

*source\_segment\_prob\_alias<sub>iseg</sub>* “Probability alias” for segment *iseg*. In sampling from this source group, a random number is compared with entries in this array. For further explanation see the routines *sample\_sources* and *set\_prob\_alias*. This is computed as part of the setup process from the source current distribution.

*source\_segment\_ptr\_alias<sub>iseg</sub>* “Pointer alias” for segment *iseg*. Used in sampling from this source group. A random number is compared with *source\_segment\_prob\_alias*. This leads either directly to an entry in *source\_segment\_ptr* or to an entry in this array which serves as an indirect pointer to the same set of geometry elements. For further explanation see the routines *sample\_sources* and *set\_prob\_alias*. This is computed as part of the setup process from the source current distribution.

*so\_name(itype)* Name of each of the source types stored as a string for output purposes.

[/u/dstotler/degas2/src/sources.hweb] Run control parameters. The macro *so\_set\_run\_flags* assigns all of these at once to the values noted above. The macro is presently invoked in *boxgen.web* and *readbackground.web* (twice). Again, these parameters may eventually be relocated and / or reset elsewhere.

*so\_restart* If *FALSE*, the code will start a new run. If *TRUE*, the code will read an existing output file and add to the data contained therein.

*so\_seed\_string* String representation of the initial random number seed.

*so\_spaced\_seeds* If *FALSE*, the random number seed will be advanced continuously between source groups (i.e., the original mode of operation for the code). If *TRUE*, the initial seed will be incremented *so\_seed\_spacing* for each subsequent source group.

*so\_seed\_spacing* The number of “flights” between the initial seeds for each source group in the case of *so\_spaced\_seeds* = *TRUE*. The code will check that this number is greater than the number of flights, preventing possible reuse of a portion of the random number sequence.

*so\_sampling* Specifies the scheme used to sample the initial source segment for a flight. The usual, random, Monte Carlo approach is selected by *so\_random*. The scheme selected by *so\_direct* flight number to directly generate an irrational number between 0 and 1 which is then used in the usual fashion in extracting a segment number from the “probability alias” arrays.

*so\_rel\_wt\_min* For iterations with a coupled plasma code only, places a lower bound on the ratio of the updated current in a source segment to the “old” current last used to set the source sampling (probability alias) arrays. Smaller values of the ratio will cause the source sampling arrays to be updated. The test is applied separately to each group.

*so\_rel\_wt\_max* For iterations with a coupled plasma code only, places an upper bound on the ratio of the updated current in a source segment to the “old” current last used to set the source sampling (probability alias) arrays. Larger values of the ratio will cause the source sampling arrays to be updated. The test is applied separately to each group.

*so\_wt\_norm\_min* For iterations with a coupled plasma code only, places a lower bound on the *so\_wt\_norm* for a source group (providing an essentially global check on *source\_segment\_rel\_wt*). Smaller values of *so\_wt\_norm* will cause the source sampling arrays to be updated.

*so\_wt\_norm\_max* For iterations with a coupled plasma code only, places an upper bound on the *so\_wt\_norm* for a source group (providing an essentially global check on *source\_segment\_rel\_wt*). Larger values of *so\_wt\_norm* will cause the source sampling arrays to be updated.

## 24 Zone definitions

[/u/dstotler/degas2/src/zone.hweb]

\$Id: zone.hweb,v 1.17 2000/04/01 17:35:24 dstotler Exp \$

[/u/dstotler/degas2/src/zone.hweb] Information about zones is held in arrays in common blocks. An zone is identified by an integer indexing into these arrays.

```
"classes.f" 24.1 ≡
 @f zn_decl integer
 @m zn_args(x) x
 @m zn_dummy(x) x
 @m zn_decl(x) integer x
 @m zn_copy(x,y) y = x
 @m zn_check(x) (x > 0 ∧ x ≤ zn_num ∧ zn_type(x) > 0 ∧ zn_type(x) ≤ zn_type_max)
 @m zn_type(x) zone_type_x
 @m zn_index(x,i) zone_index_{i,x}
 @m zn_pointer(x) zone_pointer_x
 @m zn_volume(x) zone_volume_x
```

[/u/dstotler/degas2/src/zone.hweb] Length specifications.

```
"classes.f" 24.2 ≡
 @m zn_index_max 4
 @m zi_ix 1 // Indices used with zone_index
 @m zi_iz 2
 @m zi_iy 3 // Much newer, hence, the odd ordering.
 @m zi_ptr 4

 @m zn_type_max 4
 @m zn_undefined 0
 @m zn_vacuum 1
 @m zn_plasma 2
 @m zn_solid 3
 @m zn_exit 4
```

[/u/dstotler/degas2/src/zone.hweb] Variable definitions.

```
"classes.f" 24.3 ≡
 package_init(zn)
 define_var_pk(zn, zn_num, INT)
 define_dimen_pk(zn, zone_type_ind, zn_type_max)
 define_dimen_pk(zn, zone_index_ind, zn_index_max)
 define_dimen_pk(zn, zone_ind, zn_num)
 define_var_pk(zn, zone_type_num, INT, zone_type_ind)
 define_var_pk(zn, zone_type, INT, zone_ind)
 define_var_pk(zn, zone_index, INT, zone_index_ind, zone_ind)
 define_var_pk(zn, zone_index_min, INT, zone_index_ind)
 define_var_pk(zn, zone_index_max, INT, zone_index_ind)
 define_var_pk(zn, zone_pointer, INT, zone_ind)
 define_var_pk(zn, zone_volume, FLOAT, zone_ind)
 define_var_pk(zn, zone_center, FLOAT, vector, zone_ind)
 define_var_pk(zn, zone_min, FLOAT, vector, zone_ind)
 define_var_pk(zn, zone_max, FLOAT, vector, zone_ind)
 define_varlocal_pk(zn, zone_version, CHAR, string)
 package_end(zn)
```

[/u/dstotler/degas2/src/zone.hweb] Setting up zone information.

```
"classes.f" 24.4 ≡
 @m zn_type_set(zone, type) zn_type(zone) = type
 zone_type_num_type = zone_type_num_type + 1
 zn_pointer(zone) = zone_type_num_type
```

## 25 Zone class attribute descriptions

[/u/dstotler/degas2/src/zone.hweb]

[/u/dstotler/degas2/src/zone.hweb] Define a zone. Prefix is *zn*.

The identifier *zone* is an integer variable.

*zn\_type(zone)* Is one of *zn\_vacuum*, *zn\_plasma*, *zn\_solid* indicating the type of zone.

*zn\_index(zone, l)* For  $l = 1, \rightarrow zn\_index\_max$  gives the fluid code indices of the zone. More specifically,  $l = 1$  and  $2$  are for the 2-D fluid codes.  $l = 3$  is an index for the third dimension in nearly symmetric 3-D cases. In these cases, the  $l = 1$  and  $l = 2$  values do not vary in the third dimension; this index allows them to be distinguished. The  $l = 4$  index then serves as a pointer back to a reference zone, the one for which  $zn\_index(zone, zi\_ptr) = zone$ , for that set. This allows, say, 2-D information specified at a single value of the  $l = 3$  index to be mapped to the others along the third dimension.

*zn\_pointer(zone)* Is a pointer into arrays specific for different types of zones: plasma, solid, etc.

*zn\_volume(zone)* Is the volume of the zone.

[/u/dstotler/degas2/src/zone.hweb] Internal variables in the zone class.

*zn\_num* Number of zones in the problem.

*zone\_center<sub>zone</sub>* Is an approximate center for the zone; accuracy depends on the geometry. This is always a Cartesian coordinate. In 2-D cases, the  $y$  coordinate is usually fixed at 0; in some 2-D plane symmetric cases it may have a (constant) nonzero value.

*zone\_min<sub>zone</sub>* Minimum value of the coordinates of the corners of the zone; the minimum is performed separately for each component so that this point is not necessarily part of the zone. In 1-D and 2-D cases, the  $y$  coordinate is always = 0. In planar 3-D cases, it corresponds to the Cartesian  $y$  value. In cylindrical 3-D cases, it contains an azimuthal angle in radians.

*zone\_max<sub>zone</sub>* Maximum value of the coordinates of the corners of the zone; the maximum is performed separately for each component so that this point is not necessarily part of the zone. In 1-D and 2-D cases, the  $y$  coordinate is always = 0. In planar 3-D cases, it corresponds to the Cartesian  $y$  value. In cylindrical 3-D cases, it contains an azimuthal angle in radians.

*zone\_index\_min<sub>l</sub>* Minimum value of  $zn\_index(zone, l)$  for  $l = 1, \rightarrow zn\_index\_max$ .

*zone\_index\_max<sub>l</sub>* Maximum value of  $zn\_index(zone, l)$  for  $l = 1, \rightarrow zn\_index\_max$ .

## 26 Sector definitions

[/u/dstotler/degas2/src/sector.hweb]

\$Id: sector.hweb,v 1.10 1999/04/20 20:19:34 dstotler Exp \$

[/u/dstotler/degas2/src/sector.hweb] A stratum is identified by a positive integer.

```
"classes.f" 26.1 ≡
@f sc_decl integer

@m sc_args(x) x

@m sc_dummy(x) x

@m sc_decl(x) integer x

@m sc_plasma_decl(x) integer x
@m sc_target_decl(x) integer x
@m sc_wall_decl(x) integer x

@m sc_copy(x,y) y = x

@m sc_check(x) (x > 0 ∧ x ≤ nsectors)

@m sc_compare(sect, zone, face, stratum, seg)
((zone ≡ sector_zonesect) ∧ (face ≡ sector_surfacesect) ∧ (stratum ≡ stratasect) ∧ (seg ≡
sector_strata_segmentsect))

@m sc_plasma_check(x) (x > 0 ∧ x ≤ sc_plasma_num)
@m sc_vacuum_check(x) (x > 0 ∧ x ≤ sc_vacuum_num)
@m sc_target_check(x) (x > 0 ∧ x ≤ sc_target_num)
@m sc_wall_check(x) (x > 0 ∧ x ≤ sc_wall_num)
@m sc_exit_check(x) (x > 0 ∧ x ≤ sc_exit_num)
@m sc_diag_check(x,g) (x > 0 ∧ x ≤ diagnostic_num_sectorsg)

// Define these to work around the [0] element.

@m diag_lookup(name) string_lookup(name, diagnostic_grp_name1, sc_diagnostic_grps)

@m plasma_lookup(sector) int_lookup(sector, plasma_sector1, sc_plasma_num)
@m target_lookup(sector) int_lookup(sector, target_sector1, sc_target_num)
@m wall_lookup(sector) int_lookup(sector, wall_sector1, sc_wall_num)

@m sc_unknown 0 // Sector types
@m sc_vacuum 1
@m sc_plasma 2
@m sc_target 3
@m sc_wall 4
@m sc_exit 5
@m sc_diagnostic(grp) 6 + grp - 1 // I guess this will work as long as new ones go in before here
@m sc_type_max 17

@m sc_diag_unknown 0 // Independent variables for diagnostic groups
@m sc_diag_energy 1
@m sc_diag_angle 2

@m sc_diag_spacing_unknown 0
@m sc_diag_spacing_linear 1
@m sc_diag_spacing_log 2

@m sc_diag_name_len 40

@m diagnostic_sector(i,grp) diagnostic_sector_tabdiagnostic_grp_basegrp+i-1 // Used anywhere?
```

[/u/dstotler/degas2/src/sector.hweb] Define the dimensions and variables used by the geometry routines. Note that the dimensions in this file begin at 0. However, the 0 index isn't really used. They are defined this way to accomodate netcdf's wish to say that a zero length dimension is unlimited. Namely, until sectors are in wide use, *nsectors*, *sc\_vacuum\_num*, etc. will = 0 for many cases. If the corresponding dimensions began at 1, the *length* of the dimension would be computed to be = 0; hence, netcdf would think we meant these dimensions be unlimited. We would then get an error for having more than one such dimension in the file. By instead starting them at 0, the dimensions have a length of at least 1.

```
"classes.f" 26.2 ≡
 package_init(sc)
 define_var_pk(sc, nsectors, INT)
 define_dimen_pk(sc, sector_ind, 0, nsectors)
 define_dimen_pk(sc, sector_neg_pos_ind, sc_neg, sc_pos)
 define_dimen_pk(sc, sector_type_ind, sc_type_max)

 /* These two provide external code labels for each sector, analogous to zone_index. E.g., strata
 would correspond to DEGAS walls and sector_strata_segment to wall segments. */
 define_var_pk(sc, strata, INT, sector_ind)
 define_var_pk(sc, sector_strata_segment, INT, sector_ind)

 define_var_pk(sc, sectors, INT, sector_ind)
 define_var_pk(sc, sector_zone, INT, sector_ind)
 define_var_pk(sc, sector_surface, INT, sector_ind)
 define_var_pk(sc, sector_points, FLOAT, vector, sector_neg_pos_ind, sector_ind)
 define_var_pk(sc, sector_type_pointer, INT, sector_type_ind, sector_ind)

 /* Subclasses; first vacuum */
 define_var_pk(sc, sc_vacuum_num, INT)
 define_dimen_pk(sc, vacuum_ind, 0, sc_vacuum_num)
 define_var_pk(sc, vacuum_sector, INT, vacuum_ind) // Properties: ?

 /* plasma subclass */
 define_var_pk(sc, sc_plasma_num, INT)
 define_dimen_pk(sc, plasma_ind, 0, sc_plasma_num)
 define_var_pk(sc, plasma_sector, INT, plasma_ind)
 define_var_pk(sc, plasma_temperature, FLOAT, plasma_ind)
 define_var_pk(sc, plasma_e_ion_delta, FLOAT, plasma_ind)
 define_var_pk(sc, plasma_e_ion_mult, FLOAT, plasma_ind)

 /* target subclass */
 define_var_pk(sc, sc_target_num, INT)
 define_dimen_pk(sc, target_ind, 0, sc_target_num)
 define_var_pk(sc, target_sector, INT, target_ind)
 define_var_pk(sc, target_material, INT, target_ind)
 define_var_pk(sc, target_temperature, FLOAT, target_ind)
 define_var_pk(sc, target_recyc_coef, FLOAT, target_ind)
 define_var_pk(sc, target_e_ion_delta, FLOAT, target_ind)
 define_var_pk(sc, target_e_ion_mult, FLOAT, target_ind)
 // Properties: field-line-angle-of-incidence, for example

 /* wall subclass */
 define_var_pk(sc, sc_wall_num, INT)
 define_dimen_pk(sc, wall_ind, 0, sc_wall_num)
 define_var_pk(sc, wall_sector, INT, wall_ind)
 define_var_pk(sc, wall_material, INT, wall_ind)
 define_var_pk(sc, wall_temperature, FLOAT, wall_ind)
```

```

define_varp_pk(sc, wall_recyc_coef, FLOAT, wall_ind)

/* exit subclass (includes pump) */
define_var_pk(sc, sc_exit_num, INT)
define_dimen_pk(sc, exit_ind, 0, sc_exit_num)
define_varp_pk(sc, exit_sector, INT, exit_ind)
@#if 0
define_varp_pk(sc, exit_albedo, FLOAT, exit_ind)
@#endif

/* diagnostic subclass */
define_var_pk(sc, sc_diagnostic_grps, INT)

define_dimen_pk(sc, sc_diag_name_string, sc_diag_name_len)
define_dimen_pk(sc, diag_grp_ind, 0, sc_diagnostic_grps)

define_var_pk(sc, sc_diag_max_bins, INT)

define_varp_pk(sc, diagnostic_grp_name, CHAR, sc_diag_name_string, diag_grp_ind)
define_varp_pk(sc, diagnostic_num_sectors, INT, diag_grp_ind)

define_varp_pk(sc, diagnostic_var, INT, diag_grp_ind)

define_varp_pk(sc, diagnostic_tab_index, INT, diag_grp_ind)
define_varp_pk(sc, diagnostic_min, FLOAT, diag_grp_ind)
define_varp_pk(sc, diagnostic_delta, FLOAT, diag_grp_ind)
define_varp_pk(sc, diagnostic_spacing, INT, diag_grp_ind)

define_varp_pk(sc, diagnostic_grp_base, INT, diag_grp_ind)
define_var_pk(sc, sc_diag_size, INT)
define_dimen_pk(sc, sc_diag_ind, 0, sc_diag_size - 1)
define_varp_pk(sc, diagnostic_sector_tab, INT, sc_diag_ind)

package_end(sc)

```

[/u/dstotler/degas2/src/sector.hweb] Indices

```
"classes.f" 26.3 ≡
 @m sc_neg 0
 @m sc_pos 1

 /* Constructors (really?) for sector subclasses. Any real reason for not doing these as subroutines?
 */
 @m define_sector_vacuum(sc) sc_vacuum_num = sc_vacuum_num + 1
 var_realloca(vacuum_sector)
 vacuum_sector_sc_vacuum_num = sc
 sector_type_pointer_sc_sc_vacuum = sc_vacuum_num
 @m define_sector_plasma(sc, temp) sc_plasma_num = sc_plasma_num + 1
 var_realloca(plasma_sector)
 var_realloca(plasma_temperature)
 var_realloca(plasma_e_ion_delta)
 var_realloca(plasma_e_ion_mult)
 plasma_sector_sc_plasma_num = sc
 sector_type_pointer_sc_sc_plasma = sc_plasma_num
 plasma_temperature_sc_plasma_num = temp
 plasma_e_ion_delta_sc_plasma_num = zero
 plasma_e_ion_mult_sc_plasma_num = one
 @m define_sector_target(sc, mat, temp, recyc) sc_target_num = sc_target_num + 1
 var_realloca(target_sector)
 var_realloca(target_material)
 var_realloca(target_temperature)
 var_realloca(target_recyc_coef)
 var_realloca(target_e_ion_delta)
 var_realloca(target_e_ion_mult)
 target_sector_sc_target_num = sc
 sector_type_pointer_sc_sc_target = sc_target_num
 target_material_sc_target_num = mat
 target_temperature_sc_target_num = temp
 target_recyc_coef_sc_target_num = recyc
 target_e_ion_delta_sc_target_num = zero // Simple default values,
 target_e_ion_mult_sc_target_num = one // Reset with background
 @m define_sector_wall(sc, mat, temp, recyc) sc_wall_num = sc_wall_num + 1
 var_realloca(wall_sector)
 var_realloca(wall_material)
 var_realloca(wall_temperature)
 var_realloca(wall_recyc_coef)
 wall_sector_sc_wall_num = sc
 sector_type_pointer_sc_sc_wall = sc_wall_num
 wall_material_sc_wall_num = mat
 wall_temperature_sc_wall_num = temp
 wall_recyc_coef_sc_wall_num = recyc
 @m define_sector_exit(sc) sc_exit_num = sc_exit_num + 1
 var_realloca(exit_sector)
 exit_sector_sc_exit_num = sc
 sector_type_pointer_sc_sc_exit = sc_exit_num
 @#if 0 exit_albedo_sc_exit_num = alb
 @#endif
```

## 27 Sector class attribute descriptions

[/u/dstotler/degas2/src/sector.hweb]

[/u/dstotler/degas2/src/sector.hweb] Define a sector. Prefix is *sc*.

The default behavior for a surface in DEGAS 2's tracking algorithm is to stop the flight momentarily (to check for a zone boundary) and then to continue. In some instances, these surfaces need to do more, e.g.,

1. If that surface represents the interface between a plasma or vacuum zone and a material (solid) or exit zone, the quantities describing the flight must be altered to represent the effect of that next zone,
2. If the user wishes to monitor the passing of a flight through a particular surface, the flight's parameters must be noted and used to update the appropriate tallies.

To address these needs, DEGAS 2 utilizes the *sector* concept. A sector is identified by a positive integer *sector*. At a minimum, a sector represents a coupled surface and a zone. More specifically, the surface should be a bounding surface of a cell comprising the zone (the orientation is important; see below). The function used to define sectors takes as an optional argument a second zone. If provided, the function will verify that the input surface is indeed an interface between the two zones. The programmer should use this option whenever possible as a consistency check.

A set of sector subclasses have been established to provide clearly defined arrays for storing sector-related properties. The intention is that the number of these subclasses will change infrequently; with new properties added occasionally. Hence, we have opted, for clarity, to make the subclass and property names explicit rather than store all of this information in some flexible array structure.

The sector subclasses are:

**target** The interface between a plasma zone and a solid zone,

**wall** The interface between a vacuum zone and a solid zone,

**exit** The interface between a vacuum or plasma zone and an exit zone,

**diagnostic** A diagnostic sector.

**plasma** A sector type used in an earlier version of the code, retained only for backward compatibility.

**vacuum** A sector type used in an earlier version of the code, retained only for backward compatibility.

At one point, the elimination of the *plasma* and *vacuum* types was considered; the desired objectives were, at that time, being met with just a single sector at each interface. However, initial work with nearly symmetric 3-D problems made clear the need for maintaining all four types. In these cases, the zone type can vary in the third dimension so that the nature of the interface between adjacent zones can also vary. By having sectors defined on both sides, the interface can be completely characterized.

As an example, consider a vertical pipe inserted into the bottom of an otherwise toroidally symmetric solid annular plate. Particles impinging on the plate from above would strike a solid surface except at the location of the pipe; there, they would travel down into the pipe. The surface that serves as the interface between the upper volume and the surface / pipe entrance would be identical, say,  $z = \text{constant}$ . However, we can correctly track the particle through that interface only if we know about the 3-D structure of the plate / pipe combination. This information would be contained in the zones used to model the plate and pipe, and not the surface between them and the volume above. We can accomplish this just by defining separate sectors for the plate (say, a *wall* sector) and the pipe interior (say, a *vacuum* sector). Furthermore, the gas properties in the volume above may be varying toroidally, leading to a toroidal discretization of that vacuum region. Each of those zones could be associated with a different *vacuum* sector. These sectors could also be used to tally toroidally resolved diagnostics at the vacuum - plate / pipe interface or to define toroidally discretized sources at the plate surface. In a toroidally symmetric (2-D) problem, there is no pipe, and a single sector sufficiently defines the interface. Hence, the earlier approach of using just *wall* and *target* sectors.

The *exit* and *diagnostic* subclasses have not been qualitatively altered since their introduction. However, the never-implemented *albedo* property of the exits has been replaced with the equivalent functionality of the *wall* and *target* recycling coefficients. The *exit* sector (and the associated *zone* type) has been retained; note that particles striking an *exit* sector are killed without undergoing any PMI processing. Another useful feature of *exit* sectors is that a separate default diagnostic group is defined for them.

A *target* sector is assumed to exist at the interface between adjacent plasma and solid zones. The properties *target\_material*, *target\_temperature*, and *target\_recyc\_coef* describe the solid. The properties *plasma\_e\_ion\_delta* and *plasma\_e\_ion\_mult* are used to specify the sheath potential needed to establish a plate source at these sectors. Obviously, the *plasma* sector is defined using the plasma zone, and the *target* sector is defined using the target zone. The sectors' surfaces are of opposite sign, chosen so that a particle leaving the plasma zone goes "out" (in the sense determined by the surface function's gradient) through the *plasma* sector's surface and goes "into" the *target* sector's surface. (Although this also seems obvious, the previous implementation used the opposite convention.)

The *wall* sectors exist at the interface between vacuum and solid zones. They currently differ from the *target* zones only by the absence of the above-mentioned sheath related properties.

The diagnostic subclass (distinct from the separate, but also arbitrarily named, *detector* class) properties permit sectors to record information about sector-crossing flights. Default diagnostic sectors are set up using the declared *target*, *wall*, and *exit* sectors. Associated tallies are defined from these. Related diagnostic sectors with identical properties may be collectively "grouped" together. There may be several groups of diagnostics in a given problem: one to measure particle current to each sector, one to collect an energy spectrum of arriving particles, etc. Each sector (a given surface and zone) may belong to one of the non-diagnostic subclasses (this is only a logical restriction and is unenforced), as well as to one or more diagnostic groups. Sectors may also be defined solely to serve as diagnostic surfaces.

The *strata* and *sector\_strata\_segment* arrays provide labels for coordinating input and output data with external codes. The intention is to use them in the same way as the *zn\_index* for individual zones. For historical reasons, a single stratum is used to for sectors on both sides of a boundary (e.g., plasma-surface interface).

*nsectors* Total number of sectors in the problem.

*strata<sub>sector</sub>* External (arbitrary) label for a group of sectors. In the nomenclature of the original DEGAS, one stratum would correspond to a single wall.

*sector\_strata\_segment<sub>sector</sub>* Another external label intended to enumerate individual sectors in a stratum; the original DEGAS analog would be wall segments.

*sectors<sub>pointer</sub>* Array pointed to by *surface\_sectors* (see *geometry.web*) via the integer index *pointer*. The array *sectors* provides the sector(s) index *sector* associated with a given surface. Need to use a pointer here rather than a direct array with a *surface* index since there may indeed be more than one *sector* associated with a given *surface*. Nonetheless, because of the fact that a sector is defined by a single (and only one) surface, the number of elements in *sectors* is just *nsectors*.

*sector\_zone<sub>sector</sub>* Returns the zone (class *zn*) associated with *sector*.

*sector\_surface<sub>sector</sub>* Returns the surface (see *geomint.hweb*) associated with *sector*.

*sector\_points<sub>i, e, sector</sub>* Returns the *i*th component of the position vector at one end (i.e., *e* = 0 or 1) of *sector*.

*sector\_type\_pointer<sub>sector, type</sub>* Integer index into the list of the *type* subclass corresponding to *sector* (each sector may belong to more than one subclass).

### **vacuum subclass**

*sc\_vacuum\_num* Number of sectors in the vacuum subclass.

*vacuum\_sector<sub>vacuum\_ind</sub>* Returns the sector index corresponding to index *vacuum\_ind* in the vacuum subclass list.

### **plasma subclass**

*sc\_plasma\_num* Number of sectors in the plasma subclass.

*plasma\_sector<sub>plasma\_ind</sub>* Returns the sector index corresponding to index *plasma\_ind* in the plasma subclass list.

*plasma\_temperature<sub>plasma\_ind</sub>* Is no longer used. This was originally intended to provide a means for specifying gas puff temperatures. That task is now handled by the *so\_params* array in the source class.

*plasma\_e\_ion\_delta<sub>plasma\_ind</sub>* Returns the constant contribution ( $\delta$ , in eV) in a two part expression for the incident ion energy following acceleration through the sheath potential,  $E_{\text{ion}} = \delta + \alpha T_e$ .

*plasma\_e\_ion\_mult<sub>plasma\_ind</sub>* Returns the multiplier,  $\alpha$ , on the electron temperature in a two part expression for the incident ion energy following acceleration through the sheath potential,  $E_{\text{ion}} = \delta + \alpha T_e$ .

### **target subclass**

*sc\_target\_num* Number of sectors in the target subclass.

*target\_sector<sub>target\_ind</sub>* Returns the sector index corresponding to index *target\_ind* in the target subclass list.

*target\_material*<sub>*target\_ind*</sub> Integer index into the materials class *ma* to be used to identify the plasma-material interactions for the target sector *target\_ind*.

*target\_temperature*<sub>*target\_ind*</sub> Temperature, in joules, of the target material comprising the target sector *target\_ind*.

*target\_recyc\_coef*<sub>*target\_ind*</sub> Recycling coefficient (1 - absorbed fraction) of the target material comprising the target sector *target\_ind*.

*target\_e\_ion\_delta*<sub>*target\_ind*</sub> Is no longer used. This functionality has been transferred to the *plasma* sectors.

*target\_e\_ion\_mult*<sub>*target\_ind*</sub> Is no longer used. This functionality has been transferred to the *plasma* sectors.

### **wall subclass**

*sc\_wall\_num* Number of sectors in the wall subclass.

*wall\_sector*<sub>*wall\_ind*</sub> Returns the sector index corresponding to index *wall\_ind* in the wall subclass list.

*wall\_material*<sub>*wall\_ind*</sub> Integer index into the materials class *ma* to be used to identify the plasma-material interactions for the wall sector *wall\_ind*.

*wall\_temperature*<sub>*wall\_ind*</sub> Temperature, in joules, of the wall material comprising the wall sector *wall\_ind*.

*wall\_recyc\_coef*<sub>*wall\_ind*</sub> Recycling coefficient (1 - absorbed fraction) of the wall material comprising the wall sector *wall\_ind*.

### **exit subclass**

*sc\_exit\_num* Number of sectors in the exit subclass.

*exit\_sector*<sub>*exit\_ind*</sub> Returns the sector index corresponding to index *exit\_ind* in the exit subclass list.

### **diagnostic subclass**

*sc\_diagnostic\_grps* Number of *groups* of diagnostics in this subclass.

*sc\_diag\_max\_bins* Used in conjunction with *de\_max\_bins* to check the dimension for the bins scoring array, *max\_bins*.

*diagnostic\_grp\_name*<sub>*group\_ind*</sub> Descriptive label for each diagnostic group.

*diagnostic\_num\_sectors*<sub>*group\_ind*</sub> Number of sectors comprising each diagnostic group.

*diagnostic\_var*<sub>*group\_ind*</sub> Independent variable for spectrum-collecting (e.g., *sc\_diag\_energy* or incident angle, *sc\_diag\_angle*) diagnostics.

*diagnostic\_tab\_index*<sub>*group\_ind*</sub> Number of bins for spectrum-collecting diagnostics. For simplicity, only rank 0 (designated by *diagnostic\_tab\_index* = 0) or 1 diagnostics are permitted.

*diagnostic\_min*<sub>*group\_ind*</sub> Minimum value for the independent variable.

*diagnostic\_delta*<sub>*group\_ind*</sub> Increment between independent variable bins.

*diagnostic-spacing*<sub>group\_ind</sub> Spacing method for independent variables. Presently only uniform-linear and uniform-log are treated.

*diagnostic-grp-base*<sub>group\_ind</sub> Index into the list of all sectors for the first member of *groud\_ind* (members of each group are required to be consecutive in this list). This array is used only to set up the *diagnostic\_sector* macro (see below).

*sc\_diag-size* Total number of diagnostic sectors.

*diagnostic-sector-tab*<sub>diagnostic\_ind</sub> Sector number corresponding to the diagnostic number *diagnostic\_ind*. Again, this is used only to set up the *diagnostic\_sector* macro.

[/u/dstotler/degas2/src/sector.hweb] Routines using sectors.

*sc\_check*(*sector*) Verifies that this is a valid sector.

*sc\_compare*(*sect*, *zone*, *face*, *stratum*, *seg*) Compares the input zone number *zone*, surface number *face*, stratum label *stratum*, and segment number *seg* with the corresponding properties of sector *sect*. If all four match identically, this macro evaluates as “true”.

*sc\_plasma\_check*(*p\_sector*) Verifies that *p\_sector* is a valid identifier for a member of the plasma subclass (i.e., *p\_sector* is *not* a sector number).

*sc\_target\_check*(*t\_sector*) Verifies that *t\_sector* is a valid identifier for a member of the target subclass (i.e., *t\_sector* is *not* a sector number).

*sc\_wall\_check*(*w\_sector*) Verifies that *w\_sector* is a valid identifier for a member of the wall subclass (i.e., *w\_sector* is *not* a sector number).

*sc\_exit\_check*(*e\_sector*) Verifies that *e\_sector* is a valid identifier for a member of the exit subclass (i.e., *e\_sector* is *not* a sector number).

*sc\_diag\_check*(*d\_sector*, *grp*) Verifies that *d\_sector* is a valid identifier for a member of the diagnostic subclass in diagnostic group *grp* (i.e., *d\_sector* is *not* a sector number).

*diag\_lookup*(**name**) Returns the integral diagnostic group number associated with **name**.

*plasma\_lookup*(*sector*) Retrieves the index in the plasma subclass list corresponding to sector.

*target\_lookup*(*sector*) Retrieves the index in the target subclass list corresponding to sector.

*wall\_lookup*(*sector*) Retrieves the index in the wall subclass list corresponding to sector.

*sc\_diagnostic*(*group*) Provides the integral sector type (analogous to *sc\_wall*, *sc\_plasma*, etc.) associated with *group*.

*diagnostic\_sector*(*i*, *group*) Sector number corresponding to the *i*th member of diagnostic group *group*.

*define\_sector\_vacuum*(*sector*) Adds *sector* to the vacuum sector subclass.

*define\_sector\_plasma*(*sector*, *temp*) Adds *sector* to the plasma sector subclass. The *temp* property is no longer used, but retained in the macro for compatibility. Default values *plasma\_e\_ion\_delta* = 0 and *plasma\_e\_ion\_mult* = 1 are assigned at this point; they may be explicitly changed.

*define\_sector\_target*(*sector*, *material*, *temp*, *recyc*) Adds *sector* to the target sector subclass. It is assumed to be made of *material* (member of the material class *ma*) at temperature *temp*, with recycling coefficient *recyc*.

*define\_sector\_wall*(*sector*, *material*, *temp*, *recyc*) Adds *sector* to the wall sector subclass. It is assumed to be made of *material* (member of the material class *ma*) at temperature *temp*, with recycling coefficient *recyc*.

*define\_sector\_exit*(*sector*) Adds *sector* to the exit sector subclass.

## 28 Detector definitions

[/u/dstotler/degas2/src/detector.hweb]

\$Id: detector.hweb,v 1.5 1999/07/22 17:53:42 dstotler Exp \$

[/u/dstotler/degas2/src/detector.hweb] Parameters and characteristics of detectors. Some of these arrays represent input data, some are derived. These detectors are specifically “viewing” detectors for which a line-of-sight (or “view”) is required. Surface detectors are intended to be represented by the diagnostic sub-class of sectors.

```
"classes.f" 28.1 ≡
 @f de_decl integer
 @m de_args(x) x
 @m de_dummy(x) x
 @m de_decl(x) integer x
 @m de_lookup(name) string_lookup(name, detector_name_1, de_grps)
```

[/u/dstotler/degas2/src/detector.hweb] Length specifications.

```
"classes.f" 28.2 ≡
 @m de_sy_len 24
 @m de_name_len 40

 @m de_var_unknown 0 // Independent variables
 @m de_var_wavelength 1

 @m de_spacing_unknown 0 // Spacing for independent variables
 @m de_spacing_linear 1
 @m de_spacing_log 2

 @m de_view_start 0
 @m de_view_end 1

 @m de_algorithm_unknown 0
 @m de_algorithm_uniform 1
 @m de_algorithm_circular 2

 @m de_view_pointer(i, grp) de_view_tab de_view_base grp + i - 1
```

[/u/dstotler/degas2/src/detector.hweb] Definition of the common blocks.

```
"classes.f" 28.3 ≡
 package_init(de)

 define_dimen_pk(de, de_symbol_string, de_sy_len)
 define_dimen_pk(de, de_name_string, de_name_len)
 define_dimen_pk(de, de_zone_ind, zn_num)

 define_var_pk(de, de_grps, INT)
 define_dimen_pk(de, de_grp_ind, 0, de_grps)

 define_var_pk(de, de_max_bins, INT)

 define_varp_pk(de, detector_name, CHAR, de_name_string, de_grp_ind)
 define_varp_pk(de, detector_num_views, INT, de_grp_ind)

 define_varp_pk(de, detector_var, INT, de_grp_ind)

 define_varp_pk(de, detector_tab_index, INT, de_grp_ind)
 define_varp_pk(de, detector_min, FLOAT, de_grp_ind)
 define_varp_pk(de, detector_delta, FLOAT, de_grp_ind)
 define_varp_pk(de, detector_spacing, INT, de_grp_ind)

 define_var_pk(de, detector_total_views, INT)
 define_dimen_pk(de, de_tot_view_ind, 0, detector_total_views)

 define_dimen_pk(de, de_start_end_ind, de_view_start, de_view_end)
 define_varp_pk(de, de_view_points, FLOAT, vector, de_start_end_ind, de_tot_view_ind)
 define_varp_pk(de, de_view_algorithm, INT, de_tot_view_ind)
 define_varp_pk(de, de_view_halfwidth, FLOAT, de_tot_view_ind)
 define_varp_pk(de, de_zone_fragments, FLOAT, de_tot_view_ind, de_zone_ind)

 define_varp_pk(de, de_view_base, INT, de_grp_ind)
 define_var_pk(de, de_view_size, INT)
 define_dimen_pk(de, de_view_ind, 0, de_view_size - 1)
 define_varp_pk(de, de_view_tab, INT, de_view_ind)
 package_end(de)
```

## 29 Detector class attribute descriptions

[/u/dstotler/degas2/src/detector.hweb]

[/u/dstotler/degas2/src/detector.hweb] Define detectors. Prefix is *de*.

This class is primarily intended to model light gathering diagnostics of various types, although other uses will probably be found. An individual detector is basically represented by a viewing chord and an angular halfwidth. The signal collected by each detector may (or may not) be broken down into individual bins (e.g., of wavelength).

Each viewing chord, angular halfwidth, and averaging algorithm (i.e., some model for how the physical detector averages the signal over its spot size), together make for a single “view”. One or more related views form a “group”. Each view and group are represented by integers indexing into the arrays below.

*de\_grps* Number of groups which have been defined.

*de\_max\_bins* Used in conjunction with *sc\_diag\_max\_bins* to check the dimension for the bins scoring array, *max\_bins*.

*detector\_name<sub>group\_ind</sub>* Descriptive name for each detector group.

*detector\_num\_views<sub>group\_ind</sub>* Number of views comprising each detector group.

*detector\_var<sub>group\_ind</sub>* Independent variable for spectrum-collecting (e.g., *de\_var\_wavelength*) detector groups.

*detector\_tab\_index<sub>group\_ind</sub>* Number of bins for spectrum-collecting detectors. For simplicity, only rank 0 (designated by *detector\_tab\_index* = 0) or 1 detectors are permitted.

*detector\_min<sub>group\_ind</sub>* Minimum value for the independent variable.

*detector\_delta<sub>group\_ind</sub>* Increment between independent variable bins.

*detector\_spacing<sub>group\_ind</sub>* Spacing method for independent variables. Presently only uniform-linear and uniform-log are treated.

*detector\_total\_views* Total number of views which have been defined.

*de\_view\_points<sub>i, se, view\_ind</sub>* *i*th position coordinate of the start *se* = *de\_view\_start* or end *de\_view\_end* of viewing chord *view\_ind*.

*de\_view\_algorithm<sub>view\_ind</sub>* Algorithm (currently *de\_algorithm\_uniform* or *de\_uniform\_circular*) for averaging over the detector viewing width. See subr. *detector\_view\_setup* for implementation details.

*de\_view\_halfwidth<sub>view\_ind</sub>* Angular half-width (in radians) associated with the viewing chord *view\_ind*.

*de\_zone\_frgs<sub>view\_ind, zone</sub>* The path length of chord *view\_ind* through *zone* divided by the zone volume times  $4\pi$ ; an additional factor accounts for the averaging over a finite halfwidth. This is defined before the run so the chord-integrated contributions can be made from volume scores (during or after the run).

*de\_view\_base<sub>group\_ind</sub>* Index into *de\_view\_tab* for the first viewing chord of *group\_ind*.

*de\_view\_size* Length of *de\_view\_tab*. Note: should be set = 1 if there are no detectors to ensure *de\_view\_tab* can be allocated and written to the netCDF file.

*de\_view\_tab<sub>i</sub>* List coordinating the *de\_grps* groups with their constituent *detector\_total\_views* viewing chords. The *detector\_num\_views* chords for each group are added to this array when the group is defined with the first one appearing at *i* = *detector\_view\_base<sub>group\_ind</sub>*.

[/u/dstotler/degas2/src/detector.hweb] Routines using detectors.

*de\_lookup*(**name**) Returns the integral detector group number associated with **name**.

*de\_view\_pointer*(*i*, *group\_ind*) *i*th viewing chord ( $1 \rightarrow \text{detector\_num\_views}_{\text{group\_ind}}$ ) of group *group\_ind*; i.e., this is the entry into the list of all *detector\_total\_views* viewing chords.

## 30 Tally definitions

[/u/dstotler/degas2/src/tally.hweb]

\$Id: tally.hweb,v 1.11 2001/11/16 15:59:08 dstotler Exp \$

[/u/dstotler/degas2/src/tally.hweb] Scoring can happen at one of four points within degas 2: (a) when a track crosses a sector (e.g., to compute a flux), (b) when a track travels through a zone (e.g., to compute a track length estimator), (c) at a collision event (e.g., to compute a collision estimator), and (d) after the run (to compute a derived score).

Used the word “tally” here instead of “score” since the two-letter prefix for the latter would be too similar to those for sectors and sources.

For further documentation, see the classes.web file.

"classes.f" 30.1  $\equiv$

**@f** *tl\_decl integer*

**@m** *tl\_args(x) x*

**@m** *tl\_dummy(x) x*

**@m** *tl\_decl(x) integer x*

**@m** *tl\_copy(x, y) y = x*

**@m** *tl\_check(x) (x > 0  $\wedge$  x  $\leq$  tl\_num  $\wedge$  tally\_type(x) > 0  $\wedge$  tally\_type(x)  $\leq$  tl\_type\_max)*

[/u/dstotler/degas2/src/tally.hweb] Constant specifications.

```
"classes.f" 30.2 ≡
 @m tl_type_max 3 // Types of different tallies
 @m tl_type_undefined 0
 @m tl_type_sector 1
 @m tl_type_test 2
 @m tl_type_reaction 3

 @m tl_est_max 3 // Different estimators
 @m tl_est_unknown 0
 @m tl_est_track 1
 @m tl_est_collision 2
 @m tl_est_post_process 3

 @m tl_rank_max 5
```

[/u/dstotler/degas2/src/tally.hweb] Indices for the independent variables. These are associated with strings in subroutine *set\_var\_list*. Additions here must be made there as well.

```
"classes.f" 30.3 ≡
 @m tl_index_unknown 0
 @m tl_index_zone 1
 @m tl_index_plasma_zone 2
 @m tl_index_test 3
 @m tl_index_problem_sp 4
 @m tl_index_detector 5
 @m tl_index_test_author 6 // Reaction or PMI responsible for $\vec{v}$
 @m tl_index_reaction 7
 @m tl_index_pmi 8
 @m tl_index_material 9
 @m tl_index_source_group 10 // A source group. Probably won't be used.
 @m tl_index_sector 11
 @m tl_index_strata 12
 @m tl_index_strata_segment 13
 @m tl_index_energy_bin 14
 @m tl_index_angle_bin 15
 @m tl_index_wavelength_bin 16
 @m tl_index_diagnostic 17
 @m tl_index_zone_ind.1 18
 @m tl_index_zone_ind.2 19
 @m tl_index_max 100 // An arbitrary limit

 @m tl_name_length 80 // For tally names
 @m tl_tag_length 40 // For dependent variable names

 @m tl_geom_unknown 0 // Tally geometries
 @m tl_geom_volume 1
 @m tl_geom_surface 2
 @m tl_geom_detector 3
 @m tl_geom_global 4

 @m tl_dep_var_scalar 1 // For tally_dep_var_dim
 @m tl_dep_var_vector 3

 @m tl_scoring_index(t,j,ip) pr_var_problem_sp_index(t+j-1, ip tl_index_problem_sp)

 @m tl_cv_unknown 0 // Tally conversions
 @m tl_cv_scaler_unknown 0
 @m tl_cv_partner_unknown 0

 @m tl_cv_max_conversions 5 // Fixed dimensioning parameters
 @m tl_cv_max_scalars 3
 @m tl_cv_max_partners 3
 @m tl_cv_max_local_data 5

 @m tl_cv_track 1 // Types of conversions
 @m tl_cv_post 2
 @m tl_cv_output 3

 @m tl_cv_scale 1 // Action performed by the conversion
 @m tl_cv_to_external_coords 2
 @m tl_cv_to_internal_coords 3
 @m tl_cv_divide_number 4
```

```
@m tl_cv_test_mass 1 // Scalars and other parameters used
@m tl_cv_problem_sp_mass 2
@m tl_cv_volume 3
@m tl_cv_Pa_per_mTorr 4
@m tl_cv_pos_1 5
@m tl_cv_pos_2 6
@m tl_cv_pos_3 7
@m tl_cv_zone_pos_1 8
@m tl_cv_zone_pos_2 9
@m tl_cv_zone_pos_3 10
@m tl_cv_three_halves 11

@m tl_cv_max_params 11

@m scoring_data_max 200 // Length of scoring_data array
@m max_bins 200
```

[/u/dstotler/degas2/src/tally.hweb] Variable definitions. *tally\_ind* is dimensioned from zero to accommodate netcdf. The 0 index is unused. ??? Start from 1 for now.

```
"classes.f" 30.4 ≡
 package_init(tl)

 define_dimen_pk(tl, tally_type_ind, tl_type_max)
 define_dimen_pk(tl, tally_rank_ind, tl_rank_max)
 define_dimen_pk(tl, tally_est_ind, tl_est_max)
 define_dimen_pk(tl, tally_reac_ind, pr_reaction_dim + so_type_num)
 define_dimen_pk(tl, tally_name_string, tl_name_length)
 define_dimen_pk(tl, tally_tag_string, tl_tag_length)
 define_dimen_pk(tl, tally_cv_ptr_ind, tl_cv_max_conversions)
 define_dimen_pk(tl, tally_cv_scaler_ind, tl_cv_max_scalers)
 define_dimen_pk(tl, tally_cv_partner_ind, tl_cv_max_partners)
 define_dimen_pk(tl, tally_index_ind, 0, tl_index_max)

 define_var_pk(tl, tl_num, INT)
 define_dimen_pk(tl, tally_ind, tl_num)

 define_var_pk(tl, nconversions, INT)
 define_dimen_pk(tl, tally_cv_ind, nconversions)

 define_var_pk(tl, tally_size, INT)

 define_varp_pk(tl, tally_type_num, INT, tally_type_ind)
 define_varp_pk(tl, tally_type, INT, tally_ind)
 define_varp_pk(tl, tally_geometry, INT, tally_ind)
 define_varp_pk(tl, tally_geometry_ptr, INT, tally_ind)
 define_varp_pk(tl, tally_base, INT, tally_ind)
 define_varp_pk(tl, tally_type_base, INT, tally_type_ind)
 define_varp_pk(tl, tally_rank, INT, tally_ind)
 define_varp_pk(tl, tally_dep_var_dim, INT, tally_ind)
 define_varp_pk(tl, tally_indep_var, INT, tally_rank_ind, tally_ind)
 define_varp_pk(tl, tally_tab_index, INT, tally_rank_ind, tally_ind)
 define_varp_pk(tl, tally_name, CHAR, tally_name_string, tally_ind)
 define_varp_pk(tl, tally_dep_var, INT, tally_ind)

 define_varp_pk(tl, tally_est_test, FLOAT, tally_est_ind, tally_ind)
 define_varp_pk(tl, tally_est_reaction, FLOAT, tally_reac_ind, tally_est_ind, tally_ind)

 define_varp_pk(tl, tally_num_conversions, INT, tally_ind)
 define_varp_pk(tl, tally_cv_ptr, INT, tally_cv_ptr_ind, tally_ind)
 define_varp_pk(tl, tally_cv_action, INT, tally_cv_ind)
 define_varp_pk(tl, tally_cv_type, INT, tally_cv_ind)
 define_varp_pk(tl, tally_cv_num_partners, INT, tally_cv_ind)
 define_varp_pk(tl, tally_cv_scalers, INT, tally_cv_scaler_ind, tally_cv_ind)
 define_varp_pk(tl, tally_cv_partners, INT, tally_cv_partner_ind, tally_cv_ind)
 define_var_pk(tl, tally_var_list, CHAR, tally_tag_string, tally_index_ind)

 define_varlocal_pk(tl, tally_version, CHAR, string)
 package_end(tl)
```

[/u/dstotler/degas2/src/tally.hweb] Common blocks definitions.

```
"classes.f" 30.5 ≡
 @f tl_decls integer
 @m tl_decls integer inc
```

[/u/dstotler/degas2/src/tally.hweb] Setting up tally information.

/\* The *ragged\_alloc* macros are similar to those in *pmidata.hweb*. More detailed documentation is given there. \*/

```
"classes.f" 30.6 ≡
 @m tl_set_baseinc(x, y, subs) x##_base##subs## = x##_size
 inc = 1 $DO (I, range_min(ind_##y##_1), range_max(ind_##y##_1))
 {
 inc = inc * y##_subs##_I
 }
 inc = x##_dep_var_dim##subs * inc

 @m tl_set_base_size(x, y, subs) tl_set_baseinc(x, y, subs)
 x##_size = x##_size + inc
```

## 31 Tally class attribute descriptions

[/u/dstotler/degas2/src/tally.hweb]

[/u/dstotler/degas2/src/tally.hweb] Define tallies or scores. Prefix is *tl*.

This class provides the machinery for tabulating the Monte Carlo scores or tallies which form the basis of the DEGAS 2 output. The actual output is controlled within the “output class” (see below).

There are a total of *tl\_num* tallies, each representing an accounting of how a physical quantity, the “dependent variable”, depends on a limited set of problem parameters, the “independent variables”. The rank of each tally is in principle arbitrary. Sums over an individual index could either be carried out after the run, or specified as a separate tally (e.g., in order to obtain an explicit estimate of the relative standard deviation in the reduced tally).

For each tally, an estimate of the mean and the relative standard deviation are provided. The latter is ill-defined for tallies with non-positive tally contributions.

There are three types of tally determined by the dependent variable:

*tl\_type\_sector* A simple event score carried out when a flight crosses a sector. This can be either a material surface or a purely diagnostic sector (see the sector class above).

*tl\_type\_test* A tally based solely on the properties of the test flight itself. The most familiar example is the neutral density.

*tl\_type\_reaction* Tallies associated with reactions and sources. The most important examples are the sources of background particles, momenta, and energy arising from each reaction in the problem. The same set of tallies apply to sources. Because this capability was added later, and for the sake of brevity, the associated variable names refer only to “reactions”.

In general, we envision each tally consisting of additive contributions from one or more estimators. At this time, there are only two run time estimators, *tl\_est\_track* and *tl\_est\_collision*. The latter is really a catch-all category since it includes not only collision based scores of type *tl\_type\_test*, but also event-based scores such as those of type *tl\_type\_sector* and of *tl\_type\_reaction* which can only be carried out when the reaction actually occurs or in the case of sources. The third type of estimator *tl\_est\_post\_process* must be derived from existing tallies. This option is implemented presently for only certain reactions, sources and tallies; no effort is made to propagate the standard deviation values through this process. There is also a null estimator, *tl\_est\_unknown* that provides an easy way to ignore the contribution of, say, a particular reaction to a score.

The relative contributions of each estimator to a tally are controlled by the arrays *tally\_est\_test* (for type *tl\_type\_est*) and *tally\_est\_reaction* (*tl\_type\_reaction*). Since there is only one sensible estimator for *tl\_type\_sector*, no analogous array is carried along for that type. These arrays contain the multiplicative factor associated with each estimator for that tally. *tally\_est\_reaction* has an additional index so that different estimator combinations may be specified for each reaction. **Presently, this multiplicative factor is constrained to be either 0 or 1 with the sum over estimators equalling 1.** Once more detailed optimization is carried out, this constraint will be relaxed.

Certain simple manipulations of the tally data can be effected by *conversions*. The principal examples are scaling a tally by a constant (e.g., species mass or zone volume) and velocity coordinate conversion (may be required for coupling DEGAS 2 to a plasma code). The conversions are implemented within the tally class because of the tight links between the two sets of “objects”. The decision not to establish them as separate classes was motivated in part by the treatment of the geometry transformations. Each tally may have associated with it 0 or more (up to *tl\_cv\_max\_conversions*) conversions. Each conversion maintains pointers back to its tally so that the conversions can be carried out completely either by enumerating the conversions for each tally (as is done during tracking) or vice-versa (the case for post-processing). The converted data replace the original so that dimension reducing conversions could not be done with the present approach.

Conversions are carried out at three points during the run; each is designated as a different conversion “type”: *tl\_cv\_track* for those performed during tracking (the only nontrivial example at present is the conversion of velocities from the internal cartesian coordinates to the external cylindrical system associated with a toroidally symmetric geometry), *tl\_cv\_post* for conversions needed to prepare the neutral particle number and velocity which are input to the post processing estimator, and *tl\_cv\_output* for the bulk of the conversions done just before writing the output data.

Presently there are four “actions” performed by the conversions. Although there are no a priori links between the action and the type of conversion, only certain combinations are used in practice; see *tallysetup.web* for specifics. The actions are: *tl\_cv\_scale* to scale a tally by a constant specified as part of the conversion, *tl\_cv\_divide\_number* to normalize a tally by the number of test particles in a zone; *tl\_cv\_to\_external\_coords* and *tl\_cv\_to\_internal\_coords* do the velocity conversions.

*tl\_num* Total number of distinct tallies in the problem.

*nconversions* Total number of distinct conversions in the problem.

*tally\_size* The total size of the array holding all of the tally data.

*tally\_type\_num*<sub>*type\_ind*</sub> Number of tallies of type *type\_ind*.

*tally\_type*<sub>*tally\_ind*</sub> Type of tally *tally\_ind*.

*tally-geometry*<sub>*tally\_ind*</sub> Geometry of tally *tally\_ind*:

1. *tl\_geom-volume* tallies compiled as a function of zone,

2. *tl\_geom\_surface* *tl\_type\_sector* tallies,
3. *tl\_geom\_detector* tallies computed for a member of the detector class,
4. *tl\_geom\_global* globally integrated tallies (not yet implemented).

*tally-geometry\_ptr*<sub>*tally\_ind*</sub> Points to the geometric group of elements corresponding to tally *tally\_ind*. This currently nontrivial only for *tl\_geom\_surface* (points to a diagnostic group) and *tl\_geom\_detector* (points to a detector group).

*tally-base*<sub>*tally\_ind*</sub> Pointer to the first datum for tally *tally\_ind* in the array holding all of the tally data.

*tally-type-base*<sub>*type\_ind*</sub> Pointer to the first datum of type *type\_ind* in the array holding all of the tally data. Note that the tallies of each type are assumed to be consecutive in that array.

*tally-rank*<sub>*tally\_ind*</sub> Number of independent variables (dimensionality) of tally *tally\_ind*.

*tally-dep-var-dim*<sub>*tally\_ind*</sub> Dimensionality of dependent variables of tally *tally\_ind*; presently either 1 (*tl-dep-var-scalar*) or 3 (*tl-dep-var-vector*).

*tally-indep-var*<sub>*rank\_ind*, *tally\_ind*</sub> Integer index corresponding to the *rank\_ind*th independent variable for tally *tally\_ind*. The list of possible values is lengthy and probably will be subject to considerable change; refer to *tally.hweb* for the current list (these are macros beginning with *tl\_index*; they are matched with strings in subroutine *set\_var\_list*).

*tally-tab-index*<sub>*rank\_ind*, *tally\_ind*</sub> Number of values of the *rank\_ind*th independent variable of tally *tally\_ind*.

*tally-name*<sub>*tally\_ind*</sub> Descriptive name for tally *tally\_ind*; if the tally is for a vector quantity, the name *must* contain the word “vector”.

*tally-dep-var*<sub>*tally\_ind*</sub> Integer identifying the physical quantity being accumulated for tally *tally\_ind*. This is set using a string specified in the argument list to *define\_tally* in *tallysetup*. This string is searched for in the global list of dependent variables, *pr\_var0\_list*, set in subroutine *init\_var0\_list*. The familiar examples are:

1. *mass\_change* source of mass for a background or test species (denoted by the generic term “problem species”) due to the current test species,
2. *momentum\_change\_vector* source of vector momentum for a problem species due to the current test species,
3. *energy\_change* source of energy for a problem species due to the current test species,
4. *mass* mass of test species,
5. *momentum\_vector* momentum vector of the test species,
6. *energy* energy of the test species.

Or, it must be one of the other quantities appearing as reaction handling variables (reactiondata class, *reaction\_handling\_var0*). The most commonly used example are the  $H_\alpha$  emission rates which are named *emission\_rate\_h\_alpha* where *h* = H, D, or T according to the current test species.

*tally-est-test*<sub>*est\_ind*, *tally\_ind*</sub> Multiplier on estimator *est\_ind* for tally *tally\_ind*; makes sense only for tallies of *tl\_type\_test*.

*tally-est-reaction*<sub>*reac\_so*, *est\_ind*, *tally\_ind*</sub> Multiplier on estimator *est\_ind* for tally *tally\_ind* when scoring a reaction or source. The index *reac\_so* is a concatenated list of the problem reactions (first) and the source types. Makes sense only for tallies of *tl\_type\_reaction*.

*tally-num-conversions*<sub>*tally\_ind*</sub> Number of conversions for tally *tally\_ind*; must be  $\leq$  *tl\_cv\_max\_conversions*.

*tally\_cv\_ptr*<sub>*ptr\_ind*, *tally\_ind*</sub> The *ptr\_ind*th conversion for tally *tally\_ind*.

*tally\_cv\_action*<sub>*cv\_ind*</sub> The action to be performed by conversion *cv\_ind*.

*tally\_cv\_type*<sub>*cv\_ind*</sub> Type of conversion *cv\_ind*.

*tally\_cv\_num\_partners*<sub>*cv\_ind*</sub> Number of tallies required to carry out *cv\_ind*, must be at least one and not more than *tl\_cv\_max\_partners*.

*tally\_cv\_scalers*<sub>*scal\_ind*, *cv\_ind*</sub> List of scalers (constants) *scal\_ind* required to carry out *cv\_ind*. There may be 0 to *tl\_cv\_max\_scalers* for each conversion; the number is determined by the action.

*tally\_cv\_partner*<sub>*part\_ind*, *cv\_ind*</sub> The *part\_ind*th partner in the list of partners for conversion *cv\_ind*. The first partner is always the tally being operated on by the conversion. Additional (unmodified) tallies which are required to effect the conversion are included as other “partners”.

[/u/dstotler/degas2/src/tally.hweb] Routines for use with tallies.

*tl\_scoring\_index*(*t*, *j*, *ip*) Provides the integer index of the globally used compiled list of dependent variables, *scoring\_data*, for dimension *j* (*j* < *tally\_dep\_var\_dim<sub>t</sub>*) of tally *t*. *ip* is intended to be the up-to-date *index\_parameters* array which is currently used to access the background species index.

## 32 Output definitions

[/u/dstotler/degas2/src/output.hweb]

\$Id: output.hweb,v 1.9 2000/03/27 21:14:59 dstotler Exp \$

[/u/dstotler/degas2/src/output.hweb] These are the principle output data from DEGAS 2. These are naturally grouped with the tally data, but aren’t needed until the run is complete. Hence, we have collected them separately here.

[/u/dstotler/degas2/src/output.hweb] Indices for moments in the arrays.

```
"classes.f" 32.2 ≡
 @m o_mean 0
 @m o_var 1 // variance

 @m ou_coupling_mass 1 // Rate of background mass exchange
 @m ou_coupling_momentum_1 2 // Rate of background momentum exchange
 @m ou_coupling_momentum_2 3 // (by components)
 @m ou_coupling_momentum_3 4
 @m ou_coupling_energy 5 // Rate of background energy exchange
 @m ou_coupling_max 5

 @m ou_back_max 5 // In lieu of pointer for pr_background_num

 @m out_index_5(ind,t) ind_1 + tally_tab_index_{t,1} * (ind_2 + tally_tab_index_{t,2} * (ind_3 + tally_tab_index_{t,3} *
 (ind_4 + tally_tab_index_{t,4} * ind_5)))

 @m out_array_index(i, ind,t) tally_base_t + i - 1 + tally_dep_var_dim_t * (out_index_5(ind, t))
```

[/u/dstotler/degas2/src/output.hweb] Variable definitions.

```
"classes.f" 32.3 ≡
 package_init(ou)

 define_dimen_pk(ou, output_coupling_ind, ou_coupling_max)
 define_dimen_pk(ou, output_moments_ind, o_mean, o_var)
 define_dimen_pk(ou, ou_back_ind, pr_background_num)

 define_dimen_pk(ou, output_tab_ind, 0, tally_size - 1)
 define_dimen_pk(ou, output_grps_ind, so_grps)
 define_dimen_pk(ou, output_seed_ind, 0, ran_s - 1)

 define_var_pk(ou, output_index_1_min, INT)
 define_var_pk(ou, output_index_1_max, INT)
 define_var_pk(ou, output_index_2_min, INT)
 define_var_pk(ou, output_index_2_max, INT)

 define_dimen_pk(ou, output_ind_1, output_index_1_min, output_index_1_max)
 define_dimen_pk(ou, output_ind_2, output_index_2_min, output_index_2_max)

 define_var_pk(ou, output_checkpoint, INT)

 define_varp_pk(ou, output_all, FLOAT, output_moments_ind, output_tab_ind)
 define_varp_pk(ou, output_grp, FLOAT, output_moments_ind, output_tab_ind, output_grps_ind)

 define_varp_pk(ou, out_post_all, FLOAT, output_moments_ind, output_tab_ind)
 define_varp_pk(ou, out_post_grp, FLOAT, output_moments_ind, output_tab_ind, output_grps_ind)
 define_varp_pk(ou, output_2D_coupling, FLOAT, output_ind_1, output_ind_2, ou_back_ind,
 output_coupling_ind, output_grps_ind)
 define_varp_pk(ou, output_weight_grp, FLOAT, output_grps_ind)
 define_varp_pk(ou, output_num_flights, INT, output_grps_ind)
 define_varp_pk(ou, output_random_seed, INT, output_seed_ind, output_grps_ind)
 define_varlocal_pk(ou, output_version, CHAR, string)
 define_varlocal_pk(ou, output_old_file, INT)

 package_end(ou)
```

## 33    Output class attribute descriptions

[/u/dstotler/degas2/src/output.hweb]

[/u/dstotler/degas2/src/output.hweb] Definition of the data in the main output file. Prefix is *ou*.

This class contains various objects needed to record the results of the DEGAS 2 calculation. Presently, these consist of several related large arrays representing components of the total scores. The existing arrays are explicitly of rank 2 or 3. The first of these, indicated by *mom\_ind* below, is either *o\_mean*, indicating the mean score, or *o\_var*, denoting its variance. The *table\_ind* is the compressed single dimension array containing the ragged, multidimensional scoring data. The *\_grp* arrays have a third index *group\_ind* for the source group.

With the addition of checkpointing and restarting options, the *output\_grp* array no longer contains relative standard deviations in the *o\_var* index. Rather, its contents remain in the “raw” form which allows them to be read back into the code and used for further computation during a restart. For convenience, the relative standard deviations have been copied to the *out\_post* arrays, although *these have not been corrected for post-processed additions to the mean values*. Hence, detailed statistical analyses should be restricted to the *output\_all* and *output\_grp* arrays.

Specific data required for the coupling of DEGAS 2 to two dimensional fluid plasma transport codes are contained in the *output\_2D\_coupling* array. The 2-D spatial indices are determined from the *zn\_index* arrays.

*output\_checkpoint* Is *TRUE* when the data in the output file have been generated from a checkpoint. Output data that have been post-processed (not from a checkpoint) are indicated by *FALSE*.

*output\_all*<sub>*mom\_ind*, *table\_ind*</sub> Sum over all source groups of tallies computed during the DEGAS 2 run.

*output\_grp*<sub>*mom\_ind*, *table\_ind*, *group\_ind*</sub> Tallies computed during the DEGAS 2 run for source group *group\_ind*.

*out\_post\_all*<sub>*mom\_ind*, *table\_ind*</sub> Sum over all source groups of all DEGAS 2 tallies, including post-processed contributions.

*output\_grp*<sub>*mom\_ind*, *table\_ind*, *group\_ind*</sub> All DEGAS 2 tallies for source group *group\_ind*, including post-processed contributions.

*output\_index\_1\_min* Minimum of the first index appearing in the *zn\_index* data passed to DEGAS 2.

*output\_index\_1\_max* Maximum of the first index appearing in the *zn\_index* data passed to DEGAS 2.

*output\_index\_2\_min* Minimum of the second index appearing in the *zn\_index* data passed to DEGAS 2.

*output\_index\_2\_max* Maximum of the second index appearing in the *zn\_index* data passed to DEGAS 2.

*output\_2D\_coupling*<sub>*ix*, *iy*, *back\_ind*, *cpl\_ind*, *grp\_ind*</sub> The *cpl\_ind*th variable (ion number, momentum component, or energy source) associated with background particle *back\_ind* in zone (*ix*, *iy*), as specified by *zone\_index*, for source group *group\_ind*.

*output\_weight\_grp*<sub>*group\_ind*</sub> Total weight of particles flown for source group *group\_ind*.

*output\_num\_flights*<sub>*group\_ind*</sub> Total number of flights flown for source group *group\_ind*. In most cases, *output\_weight\_grp* should be numerically equal to *output\_num\_flights*.

*output\_random\_seed*<sub>*seed\_ind*, *group\_ind*</sub> Random seed (a seed is a set of *ran\_s* integers, enumerated by *seed\_ind*) to be used for source group *group\_ind* if there is a restart. These will all have the same value (namely, the seed at the end of the run) if gaps have not been left in the random number sequence (*so\_spaced\_seeds* = *TRUE*).

[/u/dstotler/degas2/src/output.hweb] Routines using output.

Successive attempts have been made at providing a succinct accessor method to the output arrays. The present approach involves the following macro to specify the single array index into which all of the tallies and their dependencies have been compiled:

*out\_array\_index*(*i*, *ind*, *t*) Index for the *i*th component,  $i < tally\_dep\_var\_dim_t$ , of the tally *t* and *ind*, an array of length *tl\_rank\_max* containing the indices corresponding to a particular set of independent variable values (between 0 and  $tally\_tab\_index_{r,t}-1$  with  $1 \leq r \leq tally\_rank_t$ ).

## 34 Local arrays and machinery to allow compression of scoring data

[/u/dstotler/degas2/src/stat.hweb]

\$Id: stat.hweb,v 1.2 1999/03/25 20:47:43 dstotler Exp \$

[/u/dstotler/degas2/src/stat.hweb] Ideally these variables would be defined locally and passed solely through argument lists. However, to permit the scoring arrays to be as small as possible, we need to be able to enlarge them on the fly (in subroutines several levels below that in which they were initially defined). The only convenient means of doing that is by using the *define\_varp* machinery and passing the data via common where needed.

```
"classes.f" 34.1 ≡
 @m stat_mem_inc_ft 2500
 @m stat_mem_inc_frag 15000
 @m stat_mem_inc_fin 15000
```

[/u/dstotler/degas2/src/stat.hweb] Variable definitions.

"classes.f" 34.2 ≡

```

package_init(sa)

define_dimen_pk(sa, stat_moments_ind, o_mean, o_var)

define_var_pk(sa, stat_comp_flt, INT)
define_var_pk(sa, stat_comp_frag, INT)
define_var_pk(sa, stat_comp_fin, INT)

define_var_pk(sa, stat_dim_flt, INT)
define_var_pk(sa, stat_dim_frag, INT)
define_var_pk(sa, stat_dim_fin, INT)

define_dimen_pk(sa, stat_flt_ind, 0, stat_dim_flt - 1)
define_dimen_pk(sa, stat_frag_ind, 0, stat_dim_frag - 1)
define_dimen_pk(sa, stat_fin_ind, 0, stat_dim_fin - 1)

define_var_pk(sa, stat_pf_dim_flt, INT)
define_var_pk(sa, stat_pf_dim_frag, INT)
define_var_pk(sa, stat_pf_dim_fin, INT)

define_dimen_pk(sa, stat_pf_flt_ind, 0, stat_pf_dim_flt - 1)
define_dimen_pk(sa, stat_pf_frag_ind, 0, stat_pf_dim_frag - 1)
define_dimen_pk(sa, stat_pf_fin_ind, 0, stat_pf_dim_fin - 1)

define_var_pk(sa, stat_ps_dim_flt, INT)
define_var_pk(sa, stat_ps_dim_frag, INT)
define_var_pk(sa, stat_ps_dim_fin, INT)

define_dimen_pk(sa, stat_ps_flt_ind, 0, stat_ps_dim_flt - 1)
define_dimen_pk(sa, stat_ps_frag_ind, 0, stat_ps_dim_frag - 1)
define_dimen_pk(sa, stat_ps_fin_ind, 0, stat_ps_dim_fin - 1)

define_var_pk(sa, stat_wt_dim_frag, INT)
define_var_pk(sa, stat_wt_dim_fin, INT)

define_dimen_pk(sa, stat_wt_frag_ind, 0, stat_wt_dim_frag - 1)
define_dimen_pk(sa, stat_wt_fin_ind, 0, stat_wt_dim_fin - 1)

define_var_pk(sa, stat_size_flt, INT)
define_var_pk(sa, stat_size_frag, INT)
define_var_pk(sa, stat_size_fin, INT)

define_var_pk(sa, stat_wt_tot_flt, FLOAT)
define_var_pk(sa, stat_wt_tot_frag, FLOAT)
define_var_pk(sa, stat_wt_tot_fin, FLOAT)

define_varp_pk(sa, stat_flt, FLOAT, stat_moments_ind, stat_flt_ind)
define_varp_pk(sa, stat_frag, FLOAT, stat_moments_ind, stat_frag_ind)
define_varp_pk(sa, stat_fin, FLOAT, stat_moments_ind, stat_fin_ind)

define_varp_pk(sa, stat_ptr2full_flt, INT, stat_pf_flt_ind)
define_varp_pk(sa, stat_ptr2full_frag, INT, stat_pf_frag_ind)
define_varp_pk(sa, stat_ptr2full_fin, INT, stat_pf_fin_ind)

define_varp_pk(sa, stat_ptr2short_flt, INT, stat_ps_flt_ind)
define_varp_pk(sa, stat_ptr2short_frag, INT, stat_ps_frag_ind)
define_varp_pk(sa, stat_ptr2short_fin, INT, stat_ps_fin_ind)

```

```

/* frag and fin are used as a base (and an increment), but flt is always an increment. The weight of
 each entry in a base depends on the scoring history to that point. It's constant for an increment.
 */
define_varp_pk(sa, stat_wt_frag, FLOAT, stat_wt_frag_ind)
define_varp_pk(sa, stat_wt_fin, FLOAT, stat_wt_fin_ind)
package_end(sa)

```

## 35 Description of compression algorithm and data structures

[/u/dstotler/degas2/src/stat.hweb]

[/u/dstotler/degas2/src/stat.hweb] Define structures for compression of scores. Prefix is *sa*.

This class contains the local arrays which carry the scoring data into the output class arrays. A separate class is used to permit these arrays to be compressed (removing entries which are 0), so that operations on the entire arrays are much quicker than those on the full-sized output arrays. At least at the single-flight level, the latter are filled with zeroes apart from the relatively small section of the problem space covered by that flight.

Much of this class deals with the bookkeeping required for managing the compression and decompression process. Furthermore, almost every variable or array appears three times. There are separate “objects” for each level of the code: *flt*, each flight; *frag*, the “fragment” of the total number of flights assigned to each processor; and *fin*, the final set of scores which will be decompressed into the output arrays. The descriptions below will have an appended *fff*, which is intended to stand for these three strings.

Every array below has a corresponding dimension in order to permit full flexibility in enabling and disabling compression at each level. If the data at each level were always compressed, fewer dimension variables would be required. In practice, it is expected that the *flt* data will always be compressed. Since the output class already contains an uncompressed version of the final data, do not expect to want to compress the *fin* data. Hence, *stat\_fin* is not allocated and is replaced in the code explicitly by corresponding output class arrays. However, the code required for compression is in place and commented out just in case; e.g., if it is decided that the output arrays should be compressed as well. Whether or not it is best to compress the *frag* data remains to be determined. The optimum choice may depend upon the number of processors.

*stat\_comp\_fff* Integer flag (to be set using the *FALSE* and *TRUE* flags) indicating whether or not the data at level *fff* are to be compressed.

*stat\_dim\_fff* Integer dimension of *stat\_fff* array.

*stat\_pf\_dim\_fff* Integer dimension of *stat\_ptr2full\_fff* array.

*stat\_ps\_dim\_fff* Integer dimension of *stat\_ptr2short\_fff* array.

*stat\_wt\_dim\_fff* Integer dimension of *stat\_wt\_fff* array (none for *flt*; see below).

*stat\_size\_fff* Current number of entries in *stat\_fff*. If *fff* is not compressed, this is *tally\_size*.

*stat\_wt\_tot\_fff* Total weight represented by the current contents of *stat\_fff*.

*stat\_fff*[*i<sub>s</sub>*] Is the *i<sub>s</sub>*th nonzero score compiled at level *fff*.

*stat\_ptr2full\_fff*[*i<sub>s</sub>*] Pointer to the entry in the full (i.e., uncompressed) array corresponding to the *i<sub>s</sub>* nonzero score compiled at level *fff*.

*stat\_ptr2short\_fff*[*i<sub>f</sub>*] Pointer from the full (i.e., uncompressed) array index *i<sub>f</sub>* back to its corresponding entry in the compressed array *stat\_fff*. The guess is that it is more efficient to carry around this array than to search *stat\_ptr2full\_fff* with each value of *i<sub>f</sub>* that comes along during the scoring process.

*stat\_wt\_frag*[*i<sub>s</sub>*] Weight associated with the *i<sub>s</sub>* nonzero score in *stat\_fff*. Note that there is no such array for *flt* since the weight is a constant (usually unity) for all entries.

[/u/dstotler/degas2/src/stat.hweb] Macros for Stat class.

*stat\_mem\_inc\_fff* Initial dimension and memory increment for compressed arrays at level *fff*. The values used here have been chosen empirically too minimize the number of run time reallocations.

## 36 Random definitions

[/u/dstotler/degas2/src/random.hweb]

\$Id: random.hweb,v 1.11 1999/08/09 15:18:38 karney Exp \$

[/u/dstotler/degas2/src/random.hweb] A random number sequence is identified by a string tag.

```
"classes.f" 36.1 ≡
 @f rn_decl integer
 @f rn_decls integer
 @f rn_seed_decl integer
 @f single_precision integer

 @m rn_args(x) rn_index(x), rn_array(x)_0
 @m rn_dummy(x) rn_index(x), rn_array(x)
 @m rn_decl(x) integer rn_index(x)
 real rn_array(x)_0:ran_k-1
 @m rn_decls real random
 external random
 @m rn_copy(x,y) rn_index(y) = rn_index(x)
 do ran_temp = 0, ran_k - 1
 rn_array(y)_ran_temp = rn_array(x)_ran_temp
 end do

 @m rn_index(x) ran_index_##x // Accessor routines
 @m rn_array(x) ran_array_##x
```

[/u/dstotler/degas2/src/random.hweb] Length specifications.

```
"classes.f" 36.2 ≡
 @m ran_k 100 // The size of the batch of random numbers
 @m ran_s 8 // The size of the seed.
 @m ran_c 34 // The size of the decimal version of seed.
```

```

[/u/dstotler/degas2/src/random.hweb] Inline calling routines.

/* An inline version of $y = \text{random}(\text{rn_args}(x))$ */

"classes.f" 36.3 ≡
 @m rn_next(y,x) y = random(rn_args(x))
 @m rn_init(seed,x) call random_init(seed, rn_args(x))
 @m rn_seed_copy(x,y) rn_seed_copy1(x, y)
 @m rn_seed_copy1(x,y) $DO(I, 0, ran_s - 1)
 {
 $y_I = x_I$
 }
 @m rn_seed_decl(x) integer x0:ran_s-1
 @m rn_seed_args(x) x0
 @m rn_iso_next(v,x) call random_isodist(v, 1, rn_args(x))
 @m rn_cos_next(v,x) call random_cosdist(v, 1, rn_args(x))
 @m rn_gauss_next(y,x) call random_gauss(y, 1, rn_args(x))

```

## 37 Cross section definitions

```

[/u/dstotler/degas2/src/xsection.hweb]

```

```

$Id: xsection.hweb,v 1.17 2001/07/16 21:09:08 dstotler Exp $

```

Information about crosssections.

[/u/dstotler/degas2/src/xsection.hweb] Constant definitions.

```
"classes.f" 37.1 ≡
 @m xs_spacing_unknown 0 // Types of spacing in TABLES
 @m xs_spacing_linear 1
 @m xs_spacing_log 2

 @m xs_var_unknown 0 // Indices for the independent variables
 @m xs_var_density 1 // Indices for the independent variables
 @m xs_var_energy 2
 @m xs_var_temperature 3
 @m xs_var_sp_energy 4
 @m xs_var_sp_temperature 5
 @m xs_var_zone 6
 @m xs_var_1st_random_number 7
 @m xs_var_v_test_1 8
 @m xs_var_v_test_2 9
 @m xs_var_v_test_3 10
 @m xs_var_v_flow_1 11
 @m xs_var_v_flow_2 12
 @m xs_var_v_flow_3 13
 @m xs_var_m_test 14
 @m xs_var_m_back 15
 @m xs_var_m_emitter 16
 @m xs_var_v_flowb_1 17
 @m xs_var_v_flowb_2 18
 @m xs_var_v_flowb_3 19

 @m xs_max_random 1
 @m xs_max_indep_params 19

 @m xs_unit_string_length 12 // The length of the unit specifications
 @m xs_tag_string_length 40 // The length of variable names and spacings
 @m xs_eval_name_length 40 // Routine used to evaluate FITS

 @m xs_table_rank_max 3 // The biggest rank table
 @m xs_dep_var_max 20 // Maximum number of dependent variables
```

[/u/dstotler/degas2/src/xsection.hweb] Accessor functions for *xs\_data\_tab* to make it look like a 4-dimensional arrays.

```
"classes.f" 37.2 ≡
 @m xs_data_table(i, j, k, n) xs_data_tabi+xs_tab_indexn,1*(j+xs_tab_indexn,2*k)+xs_data_basen
```

[/u/dstotler/degas2/src/xsection.hweb] variable definitions.

```
"classes.f" 37.3 ≡
 package_init(xs)

 define_dimen_pk(xs, rank_ind, xs_table_rank_max)
 define_dimen_pk(xs, rank_ind0, 0, xs_table_rank_max)
 define_dimen_pk(xs, dep_var_ind, xs_dep_var_max)
 define_dimen_pk(xs, unit_string, xs_unit_string_length)
 define_dimen_pk(xs, tag_string, xs_tag_string_length)
 define_dimen_pk(xs, xs_symbol_string, rc_sy_len)
 define_dimen_pk(xs, eval_name, xs_eval_name_length)

 /* The "size" variables get used in the table allocation macros. */
 define_var_pk(xs, xs_data_size, INT)
 define_dimen_pk(xs, xs_data_ind, 0, xs_data_size - 1)

 define_var_pk(xs, xs_num_dep_var, INT)
 define_var_pk(xs, xs_rank, INT, dep_var_ind)
 define_var_pk(xs, xs_tab_index, INT, rank_ind, dep_var_ind)
 define_var_pk(xs, xs_data_base, INT, dep_var_ind)
 define_var_pk(xs, xs_data_inc, INT, dep_var_ind)

 define_var_pk(xs, xs_name, CHAR, xs_symbol_string)
 define_var_pk(xs, xs_spacing, CHAR, tag_string, rank_ind0, dep_var_ind)
 define_var_pk(xs, xs_var, CHAR, tag_string, rank_ind0, dep_var_ind)
 define_var_pk(xs, xs_units, CHAR, unit_string, rank_ind0, dep_var_ind)
 define_var_pk(xs, xs_eval_name, CHAR, eval_name, dep_var_ind)

 define_var_pk(xs, xs_min, FLOAT, rank_ind, dep_var_ind)
 define_var_pk(xs, xs_max, FLOAT, rank_ind, dep_var_ind)
 define_var_pk(xs, xs_mult, FLOAT, rank_ind0, dep_var_ind)

 define_varp_pk(xs, xs_data_tab, FLOAT, xs_data_ind)

 define_varlocal_pk(xs, xsection_version, CHAR, string)

 package_end(xs)
```

[/u/dstotler/degas2/src/xsection.hweb] Common blocks definitions.

```
"classes.f" 37.4 ≡
 @f xs_decls integer
 @m xs_decls integer old_size
```

[/u/dstotler/degas2/src/xsection.hweb] Other macros

/\* The following macros are merely renamed versions from pmiformat.hweb. Any description may be found in that file. \*/

```
"classes.f" 37.5 ≡
 @m set_base(x, inc) $ASSERT(range_min(ind_##x##_base_1) ≡
 range_min(ind_##inc##_1) ∧ range_max(ind_##x##_base_1) ≡
 range_max(ind_##inc##_1)) x##_base range_min(ind_##x##_base_1) = 0 $DO (I,
 range_min(ind_##x##_base_1) + 1, range_max(ind_##x##_base_1))
 {
 x##_base_I = x##_base_{I-1} + inc_{I-1}
 }
 x##_size = x##_base range_max(ind_##x##_base_1) + inc range_max(ind_##inc##_1)

 @#if 1
 @m set_inc(x, y) $DO (J, range_min(ind_##y##_2), range_max(ind_##y##_2)) { x##_inc_J = 1
 $DO (I, range_min(ind_##y##_1), range_max(ind_##y##_1))
 {
 x##_inc_J = x##_inc_J * y_{I, J}
 }
 }
 @#else
 @m set_inc(x, y) do j = range_min(ind_##y##_2), range_max(ind_##y##_2)
 x##_inc_j = 1
 do i = range_min(ind_##y##_1), range_max(ind_##y##_1)
 x##_inc_j = x##_inc_j * y_{i, j}
 end do
 end do
 @#endif

 @m xs_ragged_alloc(x, y) set_inc(x, y)
 set_base(x, x##_inc)
 var_alloc(x##_tab)

 @m xs_ragged_realloc(x, y) old_size = x##_size
 set_inc(x, y)
 set_base(x, x##_inc)
 var_realloc(x##_tab, old_size - 1, x##_size - 1)
```

## 38 Cross section class attribute descriptions

[/u/dstotler/degas2/src/xsection.hweb]

[/u/dstotler/degas2/src/xsection.hweb] Format for cross sections and other atomic physics data. Prefix is *xs*.

The properties associated with “cross sections” (which is a generic designation for not only cross sections, but all related quantities such as reaction rates, momentum transfer rates, etc. presented below. Here, the subscript *j* denotes the index for dependent variables (dimension variable *dep\_var\_ind*), while the nonzero values of the subscript *i* represent the independent variables (dimension variables *rank\_ind* and *rank\_ind0*) needed to determine a particular value of variable *j*. Some of these variables apply exclusively to “fits”, some to “tables”; some apply to both. *Note: fits are not yet implemented.* See also the documentation in *ratecalc.web*.

*xs\_data\_size* Size of one-dimensional array containing all data for this process.

*xs\_num\_dep\_var* Number of dependent variables associated with reaction *xs\_name*.

*xs\_rank<sub>j</sub>* Integer number of independent variables for this particular “cross section”.

*xs\_tab\_index<sub>i, j</sub>* For tabular organizations, this integer provides the number of values used for independent variable *i* and dependent variable *j*.

*xs\_data\_base<sub>j</sub>* Index into 1-D data array for first element associated with *j*.

*xs\_data\_inc<sub>j</sub>* Number of data elements associated with *j*.

*xs\_name* String naming the “cross section”. Same as the reaction name; that is, it should match *rc\_sy(rc\_num)*.

*xs\_spacing<sub>0, j</sub>* String describing the way the data are to be interpolated. Either ‘linear’ or ‘log’; applies to tabular organization only.

*xs\_spacing<sub>i, j</sub>* String telling the method of interpolation to be used for independent variable *i*; applies to tabular organization only.

*xs\_var<sub>0, j</sub>* String which tells more precisely what this quantity is; e.g., ‘rate’, ‘cross\_section’.

*xs\_var<sub>i, j</sub>* String describing independent variable *i*. Examples are ‘density’, ‘energy’, ‘temperature’, ‘specific\_energy’ (i.e., energy per amu), and ‘specific\_temperature’. Note: the “non-specific” forms should only be used for electron-impact processes where there can be no confusion as to the relevant mass.

*xs\_units<sub>0, j</sub>* String providing the units for this quantity; may be in any system.

*xs\_units<sub>i, j</sub>* String providing the units for independent variable *i*.

*xs\_eval\_name<sub>j</sub>* Mechanism for evaluating dependent variable *j*; e.g., *table\_calc*, *table\_eval*, *table\_external*, or *fit* (not implemented yet).

*xs\_min<sub>i, j</sub>* Floating point minimum value to be used for independent variable *i* in a table. It will be combined with the maximum and the number of values to actually define the array of independent variable values. For a fit, provides the lower range of validity for this independent variable.

*xs\_max<sub>i, j</sub>* Floating point maximum value to be used for independent variable *i* in a table. For a fit, provides the upper range of validity for this independent variable.

*xs\_mult<sub>0, j</sub>* Floating point multiplier for this quantity. While it may be used for any necessary purpose, it should provide the conversion factor from *xs\_units<sub>0, j</sub>* to the MKS system.

*xs\_mult<sub>i, j</sub>* Floating point multiplier for independent variable *i*. It should provide the conversion factor from *xs\_units<sub>i</sub>* to the MKS system.

*xs\_data\_tab* One-dimensional array containing all data values.

[/u/dstotler/degas2/src/xsection.hweb] Routines using cross section data.

*xs\_data\_table*(*i*, *j*, *k*, *n*) Multidimensional accessor function to cross section data provided for easier use; *i*, *j*, *k* are the independent variable indices,  $0 \leq i \leq xs\_tab\_index_1$ ,  $n-1$ , etc. for the *n*th dependent variable.

*xs\_ragged\_alloc*(*xs\_data*, *xs\_tab\_index*) Computes sizes and base addresses of the one-dimensional data array *xs\_data\_tab* (*xs\_data* is a root name here) based on the multi-dimensional information provided by *xs\_tab\_index*. There is additional documentation of a similar macro in *pmiformat.hweb*.

*xs\_ragged\_realloc*(*xs\_data*, *xs\_tab\_index*) Makes changes to size of one-dimensional data array *xs\_data\_tab* based on a revised *xs\_tab\_index*. The user must be adding to the end of the table for this to work properly.

## 39 Rate calculation definitions

[/u/dstotler/degas2/src/ratecalc.hweb]

\$Id: ratecalc.hweb,v 1.11 1999/03/25 20:54:13 dstotler Exp \$

Define a few macros for neatness sake.

[/u/dstotler/degas2/src/ratecalc.hweb] Parameters used in integration routines

"classes.f" 39.1 ≡

```
@m integration_acc const(1.0, -7) // Fractional accuracy desired
@m integration_infinity const(1.0, 20) // ∞ for upper bound of integrations; should be > 1/epsilon
@m integ_transform_exp 4 // Exponent for power law variable transformation
@m integ_transform_ratio const(2., -2) // Smaller 1/upper bounds get variable transformation
@m min_bound_ratio const(1.003) // Min. ratio of bounds in find_threshold
@m ev_to_ergs (electron_charge * const(1.0, 7)) // To convert eV to ergs
@m atomic_mass_unit_g (atomic_mass_unit * const(1.0, 3)) // 1 amu in grams
@m max_integ_steps 20 // Maximum number of integration steps
@m num_extrap_pts 3 // Points used to extrapolate estimates
@m max_interp_pts 10 // Maximum number of interp. points
@m msg_length 80 // Length of ALADDIN error messages
@m ra_search_label_length 40 // Length of labels used to identify data
@m ra_data_filename_length 80 // Length of filename containing data
@m ra_label_max 100 // Maximum number of search labels for data
@m ra_unit_string_length 12 // Length of unit specifications
@m ra_fit_coef_max 13000 // Maximum number of fit coefficients
```

[/u/dstotler/degas2/src/ratecalc.hweb] variable definitions

```
"classes.f" 39.2 ≡
 package_init(ra)
 define_dimen_pk(ra, search_label, ra_search_label_length)
 define_dimen_pk(ra, label_num_ind, ra_label_max)
 define_dimen_pk(ra, data_filename, ra_data_filename_length)
 define_dimen_pk(ra, mass_unit_string, ra_unit_string_length)
 define_dimen_pk(ra, ra_dep_var_ind, xs_dep_var_max)
 define_var_pk(ra, ra_mass, FLOAT)
 define_var_pk(ra, ra_mass_units, CHAR, mass_unit_string)
 define_var_pk(ra, ra_num_hier, INT)
 define_var_pk(ra, ra_num_bool_and, INT, ra_dep_var_ind)
 define_var_pk(ra, ra_num_bool_not, INT, ra_dep_var_ind)
 define_var_pk(ra, ra_hier_label, CHAR, search_label, label_num_ind)
 define_var_pk(ra, ra_bool_and_label, CHAR, search_label, label_num_ind, ra_dep_var_ind)
 define_var_pk(ra, ra_bool_not_label, CHAR, search_label, label_num_ind, ra_dep_var_ind)
 define_var_pk(ra, ra_data_filename, CHAR, data_filename)
 package_end(ra)
```

[/u/dstotler/degas2/src/ratecalc.hweb] Common block definitions

/\* ALADDIN common blocks use implicit naming convention; define a macro to take care of the type declarations when ALPCOM.FOR is used. \*/

```
"classes.f" 39.3 ≡
 @f decl_alpcom common
 @m decl_alpcom integer nelnmx, nhlmx, nblmx
 integer leseqn, neln, nhl, nbl, lcmprtr, ncmln, lcfptr, ncfln
 include "../Aladdin/ALPCOM.FOR"
```

[/u/dstotler/degas2/src/ratecalc.hweb] These are statement labels for the input reading routines

```
"classes.f" 39.4 ≡
 @m next_dep_var #:0
 @m end_dep_var #:0
 @m next_line #:0
 @m end_indep_var #:0
 @m react_begin #:0
 @m next_hlab #:0
 @m bool_sect #:0
 @m new_bool_sect #:0
 @m bool_list #:0
 @m next_bool #:0
 @m react_done #:0

 /* And a local common block */
 @f ra_localcommon integer
 @m ra_localcommon real u, up, temperature
 double precision fit_coef(ra_fit_coef_max)
 integer moment_number, num_fit_coef, num_evaluations
 character*xs_eval_name_length leval
 common u, up, temperature, fit_coef, moment_number, num_fit_coef, leval, num_evaluations
```

## 40 Rate calculation class attribute descriptions

[/u/dstotler/degas2/src/ratecalc.hweb]

[/u/dstotler/degas2/src/ratecalc.hweb] Data and local commons used in calculation of atomic physics rates. Prefix is *ra*

The variables in this class specify the information required to retrieve data from external database files. Presently, the database is assumed to be ALADDIN. The specific variables used here reflect that assumption. A single file name and set of hierarchical search labels are used for all of the data to be associated with one particular instance of this class. However, as many as *xs\_dep\_var\_max* different (index *j* below) sets of Boolean search labels may be specified. This allows different quantities (e.g., cross sections and reaction rates) to be included here, but forces them to be closely related. In this way, we hope to ensure overall consistency of the data.

*ra\_mass* Mass to be used in relating energy and velocity when evaluating the argument of the function specified by this instance.

*ra\_mass\_units* Units of *ra\_mass*.

*ra\_num\_hier* Number of search labels in array *ra\_hier\_label*.

*ra\_num\_bool\_and<sub>j</sub>* Number of labels in *ra\_bool\_and\_label*.

*ra\_num\_bool\_not<sub>j</sub>* Number of labels in *ra\_bool\_not\_label*.

*ra\_hier\_label* Array of hierarchical search labels used to identify (e.g., for use with ALADDIN) data.

*ra\_bool\_and\_label<sub>j</sub>* Array of labels used to further identify information associated with dependent variable *j*; in searching the data files, entries which *contain* these strings are selected.

*ra\_bool\_not\_label<sub>j</sub>* Array of labels used to further identify information associated with dependent variable *j*; in searching the data files, entries which *do not contain* these strings are selected.

*ra\_data\_filename* File name from which data will be extracted.

## 41 Plasma-Material Interaction (PMI) format definitions

[/u/dstotler/degas2/src/pmiformat.hweb]

\$Id: pmiformat.hweb,v 1.9 1999/03/25 20:47:00 dstotler Exp \$

[/u/dstotler/degas2/src/pmiformat.hweb] Information about data used to define plasma-material interactions

[/u/dstotler/degas2/src/pmiformat.hweb] Constant definitions.

```
"classes.f" 41.2 ≡
 @m pf_spacing_unknown 0 // Types of spacing of data and
 @m pf_spacing_linear 1 // independent variable arrays
 @m pf_spacing_log 2

 @m pf_var_unknown 0 // Indices for the independent variables
 @m pf_var_energy_in 1 // These double as the indices for the
 @m pf_var_vel_1_in 2 // independent_paramaters array used
 @m pf_var_vel_2_in 3 // in the main code.
 @m pf_var_vel_3_in 4
 @m pf_var_polar_angle_in 5
 @m pf_var_t_wall 6
 @m pf_var_1st_random_number 7 // These must be consecutive!
 @m pf_var_2nd_random_number 8
 @m pf_var_3rd_random_number 9
 @m pf_var_cos_polar_angle_in 10

 @m pf_max_random 3
 @m pf_max_indep_params 10

 @m pf_unit_string_length 12 // The length of the unit specifications
 @m pf_tag_string_length 20 // The length of variable names and spacings
 @m pf_eval_string_length 40 // Routine used to evaluate data

 @m pf_table_rank_max 5 // The biggest rank for data tables
 @m pf_dep_var_max 5 // Maximum number of dependent variables
```

[/u/dstotler/degas2/src/pmiformat.hweb] Accessor functions to make main tables look like multi-dimensional arrays

```
"classes.f" 41.3 ≡
 @m pf_data_table(i, j, k, l, m, n)
 pf_data_tab $i+pf_tab_index_{n,1}*(j+pf_tab_index_{n,2}*(k+pf_tab_index_{n,3}*(l+pf_tab_index_{n,4}*m)))+pf_data_base_n$
```

[/u/dstotler/degas2/src/pmiformat.hweb] variable definitions.

```
"classes.f" 41.4 ≡
 package_init(pf)

 define_dimen_pk(pf, pf_rank_ind, pf_table_rank_max)
 define_dimen_pk(pf, pf_rank_ind0, 0, pf_table_rank_max)
 define_dimen_pk(pf, pf_dep_var_ind, pf_dep_var_max)
 define_dimen_pk(pf, pf_unit_string, pf_unit_string_length)
 define_dimen_pk(pf, pf_tag_string, pf_tag_string_length)
 define_dimen_pk(pf, pf_symbol_string, pm_sy_len)
 define_dimen_pk(pf, pf_eval_string, pf_eval_string_length)

 /* The "size" variables get used in the table allocation macros. */
 define_var_pk(pf, pf_data_size, INT)
 define_dimen_pk(pf, pf_data_ind, 0, pf_data_size - 1)

 define_var_pk(pf, pf_num_dep_var, INT)
 define_var_pk(pf, pf_rank, INT, pf_dep_var_ind)
 define_var_pk(pf, pf_tab_index, INT, pf_rank_ind, pf_dep_var_ind)
 define_var_pk(pf, pf_data_base, INT, pf_dep_var_ind)
 define_var_pk(pf, pf_data_inc, INT, pf_dep_var_ind)

 define_var_pk(pf, pf_name, CHAR, pf_symbol_string)
 define_var_pk(pf, pf_spacing, CHAR, pf_tag_string, pf_rank_ind0, pf_dep_var_ind)
 define_var_pk(pf, pf_var, CHAR, pf_tag_string, pf_rank_ind0, pf_dep_var_ind)
 define_var_pk(pf, pf_units, CHAR, pf_unit_string, pf_rank_ind0, pf_dep_var_ind)
 define_var_pk(pf, pf_eval_name, CHAR, pf_eval_string, pf_dep_var_ind)

 define_var_pk(pf, pf_min, FLOAT, pf_rank_ind, pf_dep_var_ind)
 define_var_pk(pf, pf_max, FLOAT, pf_rank_ind, pf_dep_var_ind)
 define_var_pk(pf, pf_mult, FLOAT, pf_rank_ind0, pf_dep_var_ind)

 define_varp_pk(pf, pf_data_tab, FLOAT, pf_data_ind)
 define_varlocal_pk(pf, pf_version, CHAR, string)
 package_end(pf)
```

[/u/dstotler/degas2/src/pmiformat.hweb] Common blocks definitions.

```
"classes.f" 41.5 ≡
 @f pf_decls integer
 @m pf_decls integer old_size
```

[/u/dstotler/degas2/src/pmiformat.hweb] Other macros

/\* *set\_base*(*x*, *y*) is used to set the array *x\_base*(*i*) which points to the first element of the *i*th slice of the rank 1 array *x\_tab*. The length of each slice is provided by the input array *inc*(*i*) (set up via *set\_inc*). The total size of *x\_tab* is stored in *x\_size*. \*/

"classes.f" 41.6 ≡

```
@m set_base(x, inc) $ASSERT(range_min(ind_##x##_base_1) ≡
 range_min(ind_##inc##_1) ∧ range_max(ind_##x##_base_1) ≡
 range_max(ind_##inc##_1))x##_base range_min(ind_##x##_base_1) = 0 $DO (I,
 range_min(ind_##x##_base_1) + 1, range_max(ind_##x##_base_1))
{
 x##_base_I = x##_base_{I-1} + inc_{I-1}
}
x##_size = x##_base range_max(ind_##x##_base_1) + inc range_max(ind_##inc##_1)

/* From the array y(m, n), set_inc computes the number of elements implied in the mth
dimensional object represented by each n and stores it in x_inc(n). The first loop is over the the
dependent variables; the second is over the rank of each dependent variable. */
@m set_inc(x, y) /*
 $ASSERT(rank##y == 2)
*/
$DO (J, range_min(ind_##y##_2), range_max(ind_##y##_2)) { x##_inc_J = 1
 $DO (I, range_min(ind_##y##_1), range_max(ind_##y##_1))
 {
 x##_inc_J = x##_inc_J * y_{I, J}
 }
}

/* pf_ragged_alloc automatically sets up and allocates memory for “ragged” arrays used to store
the main tabular data. The first argument is intended to be a root name. A family of arrays
sharing that root name are required for this macro. These are x_base (pointer to the starting
point of individual array slices), x_tab (the 1-D array which will actually be allocated), x_size
(scalar which contains the total number of elements), and x_inc (number of elements in a given
array slice; i.e., the increment). The macro set_inc is called to set x_inc. A separate accessor
macro is used to equivalence x_tab to the apparently multi-rank array seen by the user. y
describes the rank and size of each dependent variable; namely, it will be the array pf_tab_index
defined above. x will be pf_data. */
@m pf_ragged_alloc(x, y) set_inc(x, y)
 set_base(x, x##_inc)
 var_alloc(x##_tab)

/* pf_ragged_realloc is exactly like pf_ragged_alloc except for saving the old_size and finishing with
a call to var_realloc. Since x_base etc. are recomputed from scratch, the only requirement on the
change in the array dimensions is that the user add on to the end (rather than insert in between
existing slices) of the table. */
@m pf_ragged_realloc(x, y) old_size = x##_size
 set_inc(x, y)
 set_base(x, x##_inc)
 var_realloc(x##_tab, old_size - 1, x##_size - 1)
```

## 42 PMI format class attribute descriptions

[/u/dstotler/degas2/src/pmiformat.hweb]

[/u/dstotler/degas2/src/pmiformat.hweb] Define structures used to specify plasma-material interaction data. Prefix is *pf*.

This class provides a format for external storage of data which deal with a particular PMI. Here, the subscript  $j$  denotes the index for dependent variables, while the nonzero values of the subscript  $i$  represent the independent variables needed to determine a particular value of variable  $j$ .

*pf\_data\_size* Size of one-dimensional array containing all data for this PMI.

*pf\_num\_dep\_var* Number of dependent variables associated with this PMI.

*pf\_rank<sub>j</sub>* Integer number of independent variables for the  $j$ th dependent variable.

*pf\_tab\_index<sub>j,i</sub>* Integer number of values used for the  $i$ th independent variable of the  $j$ th dependent variable.

*pf\_data\_base<sub>j</sub>* Index in *pf\_data\_tab* at which the data for the  $j$ th dependent variable begin. This array is set by the macros associated with *pf\_ragged\_alloc* and *pf\_ragged\_realloc*.

*pf\_data\_inc<sub>j</sub>* Is the number of elements in *pf\_data\_table* required for the  $j$ th dependent variable. This array is set by the macros associated with *pf\_ragged\_alloc* and *pf\_ragged\_realloc*.

*pf\_name* String (length *pm\_sy\_len*) naming the PMI. Same as the name of the PMI in the *pm* class; that is, it should match *pm\_sy*.

*pf\_spacing<sub>j,i</sub>* String (length *pf\_tag\_string\_length*) describing the way the data are to be interpolated. Examples are 'linear', 'log', and 'nonuniform'; applies to tabular data only.

*pf\_var<sub>j,i</sub>* String (length *pf\_tag\_string\_length*) describing the  $i$ th independent variable of the  $j$ th dependent variable.  $i = 0$  refers to the dependent variable itself. Examples are 'yield', 'energy\_in', etc.; see *pmiformat.hweb* for a complete list.

*pf\_units<sub>j,i</sub>* String (length *pf\_eval\_string\_length*) providing the units for the  $i$ th independent variable  $i$ . String providing the units for this quantity; may be in any system.

*pf\_eval\_name<sub>j</sub>* String (length *pf\_eval\_string\_length*) identifying the means for evaluating the  $j$ th dependent variable. The most prevalent value is "table"; other values must correspond to the name of a fitting subroutine. See *plate.web* and the description of the *pd* class for more details. Note that in some instances, the appropriate dependent "variable" is really a vector (i.e., velocity); in this case, tabular evaluation (which can return only a single scalar) makes no sense.

*pf\_min<sub>j,i</sub>* Floating point minimum value to be used for the  $i$ th independent variable of the  $j$ th dependent variable. It will be combined with the maximum and the number of values to actually define the array of independent variable values.

*pf\_max<sub>j,i</sub>* Floating point maximum value to be used for the  $i$ th independent of the  $j$ th dependent variable.

*pf\_mult<sub>j,i</sub>* Floating point multiplier for the  $i$ th independent variable of the  $j$ th dependent variable.  $i = 0$  refers to the dependent variable itself. It should provide the conversion factor from *pf\_units<sub>j,i</sub>* to the MKS system.

*pf\_data\_tab<sub>macro\_index</sub>* is the actual data object which holds the data. As described above, this is a 1-D array; its size is *pf\_data\_size*, an integer variable set by the macros *pf\_ragged\_alloc* and *pf\_ragged\_realloc*.

[/u/dstotler/degas2/src/pmiformat.hweb] Routines using PMI data.

*pf\_data\_table*(*i*, *j*, *k*, *l*, *m*, *n*) Multidimensional accessor function to the PMI data provided for ease of use; the integers *i* through *m* are the independent variable indices,  $0 \leq i \leq pf\_tab\_index_{n,1}-1$ , etc. where *n* is the dependent variable.

*pf\_ragged\_alloc*(*pf\_data*, *pf\_tab\_index*) Computes sizes and base addresses of the one-dimensional data array *pf\_data\_tab* (*pf\_data* is a root name here) based on the multi-dimensional information provided by *pf\_tab\_index*. There is additional documentation of a similar macro in *pmiformat.hweb*.

*pf\_ragged\_realloc*(*pf\_data*, *pf\_tab\_index*) Makes changes to size of one-dimensional data array *pf\_data\_tab* based on a revised *pf\_tab\_index*. The user must be adding to the end of the table for this to work properly.

## 43 Problem definitions

[/u/dstotler/degas2/src/problem.hweb]

\$Id: problem.hweb,v 1.28 2001/07/16 21:00:01 dstotler Exp \$

[/u/dstotler/degas2/src/problem.hweb] Information about problem particles is held in arrays in common blocks. An problem is identified by an integer indexing into these arrays.

```
"classes.f" 43.1 ≡
 @f pr_test_decl integer
 @f pr_background_decl integer
 @f pr_reaction_decl integer
 @f pr_generic_decl integer
 @f pr_materials_decl integer
 @f pr_pmi_decl integer

 @m pr_test_args(x) x
 @m pr_background_args(x) x
 @m pr_reaction_args(x) x
 @m pr_generic_args(x) x
 @m pr_materials_args(x) x
 @m pr_pmi_args(x) x

 @m pr_test_dummy(x) x
 @m pr_background_dummy(x) x
 @m pr_reaction_dummy(x) x
 @m pr_generic_dummy(x) x
 @m pr_materials_dummy(x) x
 @m pr_pmi_dummy(x) x

 @m pr_test_decl(x) integer x
 @m pr_background_decl(x) integer x
 @m pr_reaction_decl(x) integer x
 @m pr_generic_decl(x) integer x
 @m pr_materials_decl(x) integer x
 @m pr_pmi_decl(x) integer x

 @m pr_test_copy(x,y) y = x
 @m pr_background_copy(x,y) y = x
 @m pr_reaction_copy(x,y) y = x
 @m pr_generic_copy(x,y) y = x
 @m pr_materials_copy(x,y) y = x
 @m pr_pmi_copy(x,y) y = x

 @m pr_test_check(x) (x > 0 ∧ x ≤ pr_test_num)
 @m pr_background_check(x) (x > 0 ∧ x ≤ pr_background_num)
 @m pr_reaction_check(x) (x > 0 ∧ x ≤ pr_reaction_num)
 @m pr_materials_check(x) (x > 0 ∧ x ≤ pr_materials_num)
 @m pr_pmi_check(x) (x > 0 ∧ x ≤ pr_pmi_num)

 @m pr_test(x) problem_test_sp_x
 @m pr_background(x) problem_background_sp_x
 @m pr_reaction(x) problem_rc_x
 @m pr_mat_ref(x) problem_materials_ref_x

 @m pr_rc_num(x) problem_reaction_num_x
 @m pr_ts_rc(x,i) problem_test_reaction_i, x
 @m pr_ts_bk(x,i) problem_test_background_i, x

 @m pr_num_arrangements(i,j) problem_num_arrangements_j, i
 @m pr_ts_prod(i,j,k,l) problem_test_products_l, k, j, i
 @m pr_prod_mult(i,j,k) problem_prod_mult_k, j, i
```

```

@m pr_bk_rc(i) problem_background_reactioni
@m pr_bkrc_rg(i,j) problem_bkrc_reagentsj, i
@m pr_bkrc_prod(i,j) problem_bkrc_productsj, i

@m pr_pm_ref(x) problem_pmi_refx
@m pr_pm_case_num(x) problem_pmi_case_numx
@m pr_pm_cases(x,i) problem_pmi_casesi, x

@m pr_pm_num_arrange(i,j) problem_pmi_num_arrangej, i
@m pr_pm_prod(i,j,k,l) problem_pmi_productsl, k, j, i
@m pr_pm_prod_mult(i,j,k) problem_pmi_prod_multk, j, i

@m pr_test_lookup(sp) problem_species_testsp
@m pr_background_lookup(sp) problem_species_backgroundsp
@m pr_reaction_lookup(rc) int_lookup(rc, problem_rc, pr_reaction_num)

```

[/u/dstotler/degas2/src/problem.hweb] Length specifications.

"classes.f" 43.2 ≡

```

@m pr_reaction_max 8
@m pr_arrangement_max 4
@m pr_pmi_max 8
@m pr_max_equiv 8
@m pr_bkrc_reagent_max 2
@m pr_tag_string_length 40

```

[/u/dstotler/degas2/src/problem.hweb] Pointers to entries in dependent variable list. These are associated with string descriptors in subroutine *init\_var0\_list*. New additions here must added there as well.

```
"classes.f" 43.3 ≡
 @m pr_var_unknown 1
 @m pr_var_mass 2 // test particle data
 @m pr_var_momentum_vector 3
 @m pr_var_momentum_2 4
 @m pr_var_momentum_3 5
 @m pr_var_energy 6
 @m pr_var_angle 7
 @m pr_var_emitter_v_vector 8 // For spectrum construction
 @m pr_var_emitter_v_2 9
 @m pr_var_emitter_v_3 10
 @m pr_var_emitter_vf_Maxwell_vector 11
 @m pr_var_emitter_vf_Maxwell_2 12
 @m pr_var_emitter_vf_Maxwell_3 13
 @m pr_var_emitter_vth_Maxwell 14
 @m pr_var_xz_stress 15 // For Couette flow example

 @m pr_num_diag_vars 5 // Constructed variables begin after
 // these; place other constants above.
 @m pr_var_mass_change 20
 @m pr_var_momentum_change_vector 21 // Exchanges with background species
 @m pr_var_momentum_change_2 22
 @m pr_var_momentum_change_3 23
 @m pr_var_energy_change 24
 @m pr_var_mass_in 25 // Direction dependent for use with
 @m pr_var_momentum_in_vector 26 // PMI and diagnostic sectors.
 @m pr_var_momentum_in_2 27
 @m pr_var_momentum_in_3 28 // "mass" variables are assumed to be
 @m pr_var_energy_in 29 // first in these two lists. The lists
 @m pr_var_mass_out 30 // are pr_num_diag_vars items long.
 @m pr_var_momentum_out_vector 31
 @m pr_var_momentum_out_2 32
 @m pr_var_momentum_out_3 33
 @m pr_var_energy_out 34

 @m pr_num_change_vars 15

@if 0
 @m pr_var_back_index(i, back) i + pr_num_change_vars * back
#endif

 @m pr_problem_sp_back(back) back
 @m pr_problem_sp_test(test) (pr_background_num + test)
 @m pr_var_problem_sp_index(i, psp) i + pr_num_change_vars * psp
```

[/u/dstotler/degas2/src/problem.hweb] Variable definitions.

```
"classes.f" 43.4 ≡
package_init(pr)
define_var_pk(pr, pr_test_num, INT)
define_var_pk(pr, pr_background_num, INT)
define_var_pk(pr, pr_reaction_num, INT)
define_var_pk(pr, pr_reaction_dim, INT)
define_var_pk(pr, pr_bkrc_num, INT)
define_var_pk(pr, pr_bkrc_dim, INT)
define_var_pk(pr, pr_materials_num, INT)
define_var_pk(pr, pr_pmi_num, INT)

define_var_pk(pr, pr_var0_num, INT)

define_dimen_pk(pr, problem_test_ind, pr_test_num)
define_dimen_pk(pr, problem_background_ind, pr_background_num)
define_dimen_pk(pr, problem_reaction_ind, pr_reaction_dim)
define_dimen_pk(pr, problem_reaction_ind0, pr_reaction_max)
define_dimen_pk(pr, problem_bkrc_ind, pr_bkrc_dim)
define_dimen_pk(pr, problem_bkrc_rg_ind, pr_bkrc_reagent_max)
define_dimen_pk(pr, problem_species_ind, sp_num)
define_dimen_pk(pr, problem_materials_ref_ind, ma_num)
define_dimen_pk(pr, problem_materials_sub_ind, pr_materials_num)
define_dimen_pk(pr, problem_pmi_ref_ind, pm_num)
define_dimen_pk(pr, problem_pmi_sub_ind, pr_pmi_num)
define_dimen_pk(pr, problem_pmi_ind0, pr_pmi_max)
define_dimen_pk(pr, problem_product_ind, rc_product_max)
define_dimen_pk(pr, problem_arr_ind, pr_arrangement_max)

define_dimen_pk(pr, pr_var0_list_ind, pr_var0_num)
define_dimen_pk(pr, pr_tag_string, pr_tag_string_length)

define_varp_pk(pr, problem_species_test, INT, problem_species_ind)
define_varp_pk(pr, problem_species_background, INT, problem_species_ind)
define_varp_pk(pr, problem_materials_sub, INT, problem_materials_ref_ind)
define_varp_pk(pr, problem_test_sp, INT, problem_test_ind)
define_varp_pk(pr, problem_background_sp, INT, problem_background_ind)
define_varp_pk(pr, problem_materials_ref, INT, problem_materials_sub_ind)

define_varp_pk(pr, problem_rc, INT, problem_reaction_ind)
define_varp_pk(pr, problem_reaction_num, INT, problem_test_ind)
define_varp_pk(pr, problem_test_reaction, INT, problem_reaction_ind0, problem_test_ind)
define_varp_pk(pr, problem_test_background, INT, problem_reaction_ind0, problem_test_ind)

define_varp_pk(pr, problem_num_arrangements, INT, problem_reaction_ind0, problem_test_ind)
define_varp_pk(pr, problem_test_products, INT, problem_product_ind, problem_arr_ind,
 problem_reaction_ind0, problem_test_ind)
define_varp_pk(pr, problem_prod_mult, FLOAT, problem_arr_ind, problem_reaction_ind0,
 problem_test_ind)

define_varp_pk(pr, problem_background_reaction, INT, problem_bkrc_ind)
define_varp_pk(pr, problem_bkrc_reagents, INT, problem_bkrc_rg_ind, problem_bkrc_ind)
define_varp_pk(pr, problem_bkrc_products, INT, problem_product_ind, problem_bkrc_ind)

define_varp_pk(pr, problem_pmi_ref, INT, problem_pmi_sub_ind)
define_varp_pk(pr, problem_pmi_sub, INT, problem_pmi_ref_ind)
```

```

define_varp_pk(pr, problem_pmi_case_num, INT, problem_test_ind)
define_varp_pk(pr, problem_pmi_cases, INT, problem_pmi_ind0, problem_test_ind)

define_varp_pk(pr, problem_pmi_num_arrange, INT, problem_pmi_ind0, problem_test_ind)
define_varp_pk(pr, problem_pmi_products, INT, problem_product_ind, problem_arr_ind,
 problem_pmi_ind0, problem_test_ind)
define_varp_pk(pr, problem_pmi_prod_mult, FLOAT, problem_arr_ind, problem_pmi_ind0,
 problem_test_ind)

define_varp_pk(pr, pr_var0_list, CHAR, pr_tag_string, pr_var0_list_ind)

define_varlocal_pk(pr, problem_version, CHAR, string)
package_end(pr)

```

[/u/dstotler/degas2/src/problem.hweb] Variables for problemsetup.

```

"classes.f" 43.5 ≡
 package_init(ps)
 define_var_pk(ps, num_generics, INT)
 define_dimen_pk(ps, pr_equiv_ind, pr_max_equiv)
 define_varp_pk(ps, generics, INT, problem_species_ind)
 define_varp_pk(ps, num_equiv, INT, problem_species_ind)
 define_varp_pk(ps, equivalents, INT, pr_equiv_ind, problem_species_ind)
 package_end(ps)

```

## 44 Problem class attribute descriptions

[/u/dstotler/degas2/src/problem.hweb]

[/u/dstotler/degas2/src/problem.hweb] Data structures used to specify the current problem. Prefix is *pr*.

These properties characterize the interactions between the various components of the problem: test particles, background species, reactions, materials, and plasma-material interactions. The integer indices and array values noted below correspond to entries into one of several lists. For clarity, we will associate with each list a different subscript to be used in the subsequent definitions:

1. Reference list of species (class *sp*) (subscript: *s*),
2. Reference list of reactions (class *rc*) (subscript: *r*),
3. List of test species (problem input) (subscript: *t*),
4. List of background species (problem input) (subscript: *b*),
5. List of problem reactions (problem input) (subscript: *pr*),
6. List of problem background reactions (problem input) (subscript: *br*),
7. Reference list of materials (class *ma*) (subscript: *m*),
8. List of problem materials (problem input) (subscript: *ms*),
9. Reference list of plasma-materials interactions (class *pm*) (subscript: *p*),
10. List of problem PMI (problem input) (subscript: *ps*),

The index *i* below refers to an element of each of these lists. Once we've got the subset lists specified in the "problem input", we can identify the reactions and PMI in which particular test species participates. Indices for these sub-subset lists are denoted by *j* below. Reactions between background species only (e.g., recombination) are designated as "background reactions" and are characterized by separate arrays below. Background reactions enter into the problem by having a test species (maybe more than one?) in their list of products and, thus, give rise to a source group (class *so*).

*pr\_test*(*i<sub>t</sub>*) Species index of test species *i<sub>t</sub>*.

*pr\_background*(*i<sub>b</sub>*) Species index of background species *i<sub>b</sub>*.

*pr\_reaction*(*i<sub>pr</sub>*) Index into the reference list of reactions for problem reaction *i<sub>pr</sub>*.

*pr\_mat\_ref*(*i<sub>ms</sub>*) Reference materials index of the *i<sub>ms</sub>* entry in the problem materials subset.

*pr\_rc\_num*(*i<sub>t</sub>*) Number of reactions in which test species *i<sub>t</sub>* participates.

*pr\_ts\_rc*(*i<sub>t</sub>*, *j*) Index of the problem reaction for the *j*th reaction involving test species *i<sub>t</sub>*,  $1 \leq j \leq \text{pr\_rc\_num}(i_t)$ .

*pr\_ts\_bk*(*i<sub>t</sub>*, *j*) Index into the list of (problem) background species for the *j*th reaction involving test species *i<sub>t</sub>*.

*pr\_num\_arrangements*(*i<sub>t</sub>*, *j*) Number of arrangements of products (which may or may not be truly different) for the *j*th reaction involving test species *i<sub>t</sub>*.

*pr\_ts\_prod*(*i<sub>t</sub>*, *j*, *k*, *l*) The *l*th product (equivalent to the *l*th entry in the list of generic species given by *rc\_product*) for the *k*th arrangement of the *j*th reaction involving test species *i<sub>t</sub>*.

*pr\_prod\_mult*( $i_t, j, k$ ) Relative weight or multiplicity of the  $k$ th arrangement of the  $j$ th reaction involving test species  $i_t$ . This quantity is the probability of obtaining arrangement  $k$  in this reaction.

*pr\_bkrc*( $i_{br}$ ) Index into the reference list of reactions for problem background reaction  $i_{br}$ .

*pr\_bkrc\_rg*( $i_{br}, j$ ) Index into the list of (problem) background species for the  $j$ th reagent ( $1 \leq j \leq \text{pr\_bkrc\_reagent\_max}$ ) in the  $i_{br}$ th background reaction.

*pr\_bkrc\_prod*( $i_{br}, j$ ) Index into the list of reference species for the  $j$  product of the  $i_{br}$ th background reaction.

*pr\_pm\_ref*( $i_{ps}$ ) Index into the reference list of PMI for problem PMI  $i_{ps}$ .

*pr\_pm\_case\_num*( $i_t$ ) Number of PMI in which test species  $i_t$  participates.

*pr\_pm\_cases*( $i_t, j$ ) Index of the problem PMI for the  $j$ th reaction involving test species  $i_t$ ,  $1 \leq j \leq \text{pr\_pm\_case\_num}(i_t)$ .

*pr\_pm\_num\_arrange*( $i_t, j$ ) Number of arrangements of products (which may or may not be truly different) for the  $j$ th PMI involving test species  $i_t$ .

*pr\_pm\_prod*( $i_t, j, k, l$ ) The  $l$ th product for the  $k$ th arrangement of the  $j$ th PMI involving test species  $i_t$ .

*pr\_prod\_mult*( $i_t, j, k$ ) Relative weight or multiplicity of the  $k$ th arrangement of the  $j$ th PMI involving test species  $i_t$ . This quantity is the probability of obtaining arrangement  $k$  in this PMI.

[/u/dstotler/degas2/src/problem.hweb] Internal variables for the problem class.

*pr\_test\_num* Number of test species in the problem.

*pr\_background\_num* Number of background species in the problem.

*pr\_reaction\_num* Number of reactions in the problem.

*pr\_bkrc\_num* Number of background reactions in the problem.

*pr\_bkrc\_dim* For dimensioning; is equal to *pr\_bkrc\_num*, except is set to 1 when *pr\_bkrc\_num*=0.

*pr\_materials\_num* Number of materials in the problem.

*pr\_pmi\_num* Number of plasma-material interactions in the problem.

*pr\_var0\_num* Number of entries in *pr\_var0\_list*.

*pr\_materials\_sub*[ $i_m$ ] Problem index of reference material  $i_m$ .

*problem\_pmi\_sub*[ $i_p$ ] Index of the problem PMI corresponding to reference PMI  $i_p$ .

*pr\_var0\_list*[ $j$ ] List of dependent variables in the problem (compiled on the fly),  $1 \leq j \leq \text{pr\_var0\_num}$ .

[/u/dstotler/degas2/src/problem.hweb] Routines using problem class.

Note: these first two lookup macros are frequently used and are thus based on actual internal arrays.

*pr\_test\_lookup*( $i_s$ ) Index into the list of problem test species corresponding to reference species  $i_s$ ; returns 0 if the species  $i_s$  is not a test species.

*pr\_background\_lookup*( $i_s$ ) Index into the list of problem background species corresponding to reference species  $i_s$ ; returns 0 if the species  $i_s$  is not a background species.

*pr\_reaction\_lookup*( $i_r$ ) Index into the list of problem reactions corresponding to reference reaction  $i_r$ .

*pr\_problem\_sp\_back*(*back*) Provides integer index into combined background and test list (problem species list) for background species *back*.

*pr\_problem\_sp\_test*(*test*) Provides integer index into combined background and test list (problem species list) for test species *test*.

*pr\_var\_problem\_sp\_index*( $i, psp$ ) Provides integer index into *pr\_var0.list* for any problem species *psp*. The list of problem species is the combined background and test species list. (The generic exchange variables such as *pr\_var\_mass\_change* are duplicated *pr\_background\_num* + *pr\_test\_num* times).

[/u/dstotler/degas2/src/problem.hweb] Problem setup variables for isotopic variations. Prefix is *ps*.

The following variables are required in a number of the *problemsetup.web* routines, but not in any outside; hence, they have been broken off into this separate package defined within *problem.hweb*.

The mechanism for dealing with isotopic variations involves defining a “generic species”, a single species which represents all of the isotopic combinations equivalent to it. E.g., for the all-important case of hydrogen, the generic archetype for a given molecule is defined (or at least it should be) as the one containing only protium isotopes.

*num\_generics* Total number of “generic species” appearing in the lists of test and background species.

*generics*[ $i_s$ ] Provides the index  $i_g$  (used below) of the “generic species” which is being used to represent (the isotopically equivalent) species  $i_s$  of the reference species list.

*num\_equiv*[ $i_g$ ] Total number of entries in the test and background species lists which are equivalent to “generic species”  $i_g$ .

*equivalents*[ $i_g, j$ ] Provides the index into the reference species list of the  $j$ th particle equivalent to generic species  $i_g$ , where  $1 \leq j \leq \text{num\_equiv}[i_g]$ .

## 45 Reaction data definitions

[/u/dstotler/degas2/src/reactiondata.hweb]

\$Id: reactiondata.hweb,v 1.15 1999/03/25 20:47:17 dstotler Exp \$

[/u/dstotler/degas2/src/reactiondata.hweb] Data for reactions is held in arrays in common blocks.  $rd\_rate(i, j, k, r)$  accesses the reaction rate for the  $r$ th reaction as an array with up to 3 dimensions. “Handling” (i.e., not reaction rate) data is kept in the analogous  $rd\_handling(i, j, k, d, r)$  array. The  $d$  index denotes the  $d$ th dependent variable for the  $r$ th reaction.

"classes.f" 45.1  $\equiv$

**@m**  $rd\_rate(i, j, k, r)$

$reaction\_rate\_tab_{reaction\_rate\_base_r+i+reaction\_rate\_tab\_index_{r,1}*(j+reaction\_rate\_tab\_index_{r,2}*k)}$

**@m**  $rd\_handling(i, j, k, d, r)$

$reaction\_handling\_tab_{reaction\_handling\_base_{r,d}+i+reaction\_handling\_tab\_index_{r,d,1}*(j+reaction\_handling\_tab\_index_{r,d,2}*k)}$

**@m**  $rd\_data\_args(x, subs)$

$x_{\#\#\_eval\_name\#\#subs}, x_{\#\#\_rank\#\#subs}, x_{\#\#\_var\#\#subs_1}, x_{\#\#\_tab\_index\#\#subs_1},$   
 $x_{\#\#\_spacing\#\#subs_0}, x_{\#\#\_min\#\#subs_1}, x_{\#\#\_delta\#\#subs_1}, x_{\#\#\_tab\_x_{\#\#\_base\#\#subs}}$

**@m**  $rd\_dep\_var\_max$  **\$EVAL** ( $xs\_dep\_var\_max - 1$ )

[/u/dstotler/degas2/src/reactiondata.hweb] Variable definitions.

```
"classes.f" 45.2 ≡
package_init(rd)

define_dimen_pk(rd, rd_rank_ind, xs_table_rank_max)
define_dimen_pk(rd, rd_dep_var_ind, rd_dep_var_max)
define_dimen_pk(rd, rd_rank_ind0, 0, xs_table_rank_max)
define_dimen_pk(rd, rd_tag_string, xs_tag_string_length)
define_dimen_pk(rd, rd_eval_string, xs_eval_name_length)

define_var_pk(rd, reaction_rate_size, INT)
define_dimen_pk(rd, rd_table_ind, 0, reaction_rate_size - 1)
define_var_pk(rd, reaction_handling_size, INT)
define_dimen_pk(rd, rd_handling_table_ind, 0, reaction_handling_size - 1)

define_varp_pk(rd, reaction_rate_min, FLOAT, rd_rank_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_delta, FLOAT, rd_rank_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_rank, INT, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_spacing, INT, rd_rank_ind0, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_eval_name, CHAR, rd_eval_string, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_tab_index, INT, rd_rank_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_var, INT, rd_rank_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_num_rand, INT, problem_reaction_ind)

define_varp_pk(rd, reaction_rate_base, INT, problem_reaction_ind)
define_varp_pk(rd, reaction_rate_tab, FLOAT, rd_table_ind)

define_varp_pk(rd, reaction_handling_min, FLOAT, rd_rank_ind, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_delta, FLOAT, rd_rank_ind, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_rank, INT, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_spacing, INT, rd_rank_ind0, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_eval_name, CHAR, rd_eval_string, rd_dep_var_ind,
 problem_reaction_ind)
define_varp_pk(rd, reaction_handling_tab_index, INT, rd_rank_ind, rd_dep_var_ind,
 problem_reaction_ind)
define_varp_pk(rd, reaction_handling_var0, INT, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_var, INT, rd_rank_ind, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_num_rand, INT, problem_reaction_ind)

define_varp_pk(rd, reaction_handling_base, INT, rd_dep_var_ind, problem_reaction_ind)
define_varp_pk(rd, reaction_handling_tab, FLOAT, rd_handling_table_ind)
package_end(rd)
```

[/u/dstotler/degas2/src/reactiondata.hweb] Common blocks definitions.

```
"classes.f" 45.3 ≡
@f rd_decls integer
@m rd_decls integer inc

 /* The ragged_alloc macros are the same as those in pmidata.hweb. More detailed documentation
 is given there. */
@m rd_set_baseinc(x, y, subs) x##_base##subs## = x##_size
 inc = 1 $DO (I, range_min(ind_##y##_1), range_max(ind_##y##_1))
 {
 inc = inc * y##subs##_I
 }

@m rd_ragged_alloc(x) x##_size = 0
 var_alloc(x##_base)
 call init_base(x##_base, array_size(x##_base))

@m rd_ragged_realloc(x, y, subs) rd_set_baseinc(x, y, subs)
 var_realloc(x##_tab, x##_size - 1, x##_size - 1 + inc)
 x##_size = x##_size + inc

@m rd_null_alloc(x) var_realloc(x##_tab, x##_size - 1, x##_size - 1 + 1)
 x##_size = 1
```

## 46 Reaction data class attribute descriptions

[/u/dstotler/degas2/src/reactiondata.hweb]

[/u/dstotler/degas2/src/reactiondata.hweb] Structures to hold atomic physics data for the current problem. Prefix is *rd*.

A given reaction within this class is identified by the integer index *pr\_reac* (“problem reaction”) assigned to it during the problem setup phase. This class collates the “cross section” data (class *xs*) for all of the reactions in one place for the main code to use during the run. The reaction data consist of two very similar subclasses. The first has names which begin with *reaction\_rate*; these arrays are used only to contain the reaction rate for a particular *pr\_reac*. The second subclass has names starting with *reaction\_handling* and is used for the various data needed to process or handle a particular collision, along with scoring. The nature and names of the data objects within the *reaction\_handling* array can vary not only with the reaction, but with the model used to represent a given reaction.

The representation of the data in memory is crucial to efficiency. Since the main data tables here are four or five dimensional objects of very “ragged” structure, we have opted for an internal representation as single dimensional arrays. To simplify coding, macros have been developed to simulate the familiar multi-dimensional array functionality. The downside of this is the addition of bookkeeping arrays (here, the *xxx\_base* arrays), the unwieldy-ness of the macros themselves, and the difficulty in extracting data from the 1-D array during debugging or while perusing the netCDF files. At least this last issue may be addressed by the construction of a small browser code to allow multi-dimensional netCDF data to be viewed and plotted in a straightforward manner. The macros used in *pmidata.hweb* are very similar to these; an attempt at documenting their operation can be found in that file.

*reaction\_rate\_size* Size of the 1-D array for the reaction rates; set by the macros *rd\_ragged\_alloc* and *rd\_ragged\_realloc*.

*reaction\_handling\_size* Size of the 1-D array for the handling data; set by the macros *rd\_ragged\_alloc* and *rd\_ragged\_realloc*.

*reaction\_rate\_min<sub>pr\_reac,i</sub>* Floating point minimum value of the *i*th independent variable of the rate for *pr\_reac*.

*reaction\_rate\_delta<sub>pr\_reac,i</sub>* Floating point increment in the *i*th independent variable of the rate for *pr\_reac*.

*reaction\_rate\_rank<sub>pr\_reac</sub>* Integer number of independent variables in the rate for *pr\_reac*.

*reaction\_rate\_spacing<sub>pr\_reac,i</sub>* Integer identifying the way the *i*th variable of the rate (*i* = 0 refers to the rate itself) are to be interpolated. Examples are *xs\_spacing\_linear* and *xs\_spacing\_log*. Namely, in the latter case the log of the independent variable (or dependent if *i* = 0) is interpolated linearly.

*reaction\_rate\_eval\_name<sub>pr\_reac</sub>* String providing the name of the function which will be used to evaluate the rate of *pr\_reac*. The only currently implemented option is “table”; more generally, the name of a fitting function would be possible.

*reaction\_rate\_tab\_index<sub>pr\_reac,i</sub>* Number of values used for the *i*th independent variable of the rate for *pr\_reac*.

*reaction\_rate\_var<sub>pr\_reac,i</sub>* Integer identifying the *i*th independent variable for the rate. The independent variables are typically temperature, density, or energy; see *xsection.hweb* for a full list.

*reaction\_rate\_num\_rand<sub>pr\_reac</sub>* Integer number of uniform random deviates required to evaluate the rate for *pr\_reac* (will usually be 0 here, however).

*reaction\_rate\_base<sub>pr\_reac</sub>* Index in *reaction\_rate\_tab* at which the data for the rate of *pr\_reac* begins. This array is set by the macros *rd\_ragged\_alloc* and *rd\_ragged\_realloc*.

*reaction\_rate\_tab*<sub>macro\_index</sub> is the actual data object which holds the reaction rate data. As described above, this is a 1-D array of size *reaction\_rate\_size*. These data are normally accessed through the macro interface *rd\_rate* (see below).

*reaction\_handling\_min*<sub>pr\_reac,j,i</sub> Floating point minimum value of the *i*th independent variable for the *j*th dependent variable in *pr\_reac*.

*reaction\_handling\_delta*<sub>pr\_reac,j,i</sub> Floating point increment in the *i*th independent variable for the *j*th dependent variable in *pr\_reac*.

*reaction\_handling\_rank*<sub>pr\_reac,j</sub> Integer number of independent variables for the *j*th dependent variable in *pr\_reac*.

*reaction\_handling\_spacing*<sub>pr\_reac,j,i</sub> Integer identifying the way the *i*th variable for the *j*th dependent variable (*i* = 0 refers to the dependent variable itself) is to be interpolated. Examples are *xs\_spacing\_linear* and *xs\_spacing\_log*. Namely, in the latter case the log of the independent variable (or dependent if *i* = 0) is interpolated linearly.

*reaction\_handling\_eval\_name*<sub>pr\_reac,j</sub> String providing the name of the function which will be used to evaluate the data for the *j*th dependent variable of *pr\_reac*. The only currently implemented option is “table”; more generally, the name of a fitting function would be possible.

*reaction\_handling\_tab\_index*<sub>pr\_reac,j,i</sub> Number of values used for the *i*th independent variable of the *j*th dependent variable for *pr\_reac*.

*reaction\_handling\_var0*<sub>pr\_reac,j</sub> Integer identifying the *j*th dependent variable of *pr\_reac*. This value is a pointer into the global dependent variable list *pr\_var0\_list*.

*reaction\_handling\_var*<sub>pr\_reac,j,i</sub> Integer identifying the *i*th independent variable for the *j*th dependent variable of *pr\_reac*. The independent variables are typically temperature, density, or energy; see *xsection.hweb* for a full list.

*reaction\_handling\_num\_rand*<sub>pr\_reac</sub> Integer number of uniform random deviates required to evaluate all dependent variables of *pr\_reac*. For example, the integral of a distribution may be broken into a number of equal pieces so that it can be sampled by interpolation with a single random number. Since the same random number may be needed for more than one dependent variable (e.g., coupled, multi-dimensional distributions), this variable contains only the total number of random deviates.

*reaction\_handling\_base*<sub>pr\_reac,j</sub> Index in *reaction\_handling\_tab* at which the data for the *j*th dependent variable of *pr\_reac* begin. This array is set by the macros *rd\_ragged\_alloc* and *rd\_ragged\_realloc*.

*reaction\_handling\_tab*<sub>macro\_index</sub> Is the actual data object which holds the handling data. As described above, this is a 1-D array of size *reaction\_handling\_size*. These data are normally accessed through the macro interface *rd\_handling* (see below).

[/u/dstotler/degas2/src/reactiondata.hweb] Routines using reaction data.

*rd\_rate*(*i*, *j*, *k*, *pr\_reac*) Multidimensional accessor function to the reaction rate data provided for easier use; *i*, *j*, *k* are the independent variable indices,  $0 \leq i \leq xs\_tab\_index_{pr\_reac,1}-1$ , etc. for the reaction *pr\_reac*.

*rd\_handling*(*i*, *j*, *k*, *d*, *pr\_reac*) Multidimensional accessor function to the handling data provided for easier use; *i*, *j*, *k* are the independent variable indices,  $0 \leq i \leq xs\_tab\_index_{pr\_reac,d,1}-1$ , etc. for the *d*th dependent variable of reaction *pr\_reac*.

*rd\_data\_args*(*x*, *subs*) This macro provides an argument list of all of the *reaction\_rate* or *reaction\_handling* (use either of these as *x*) arrays for passing down to a subroutine. The *subs* argument provides the *pr\_reac* array index (both cases) and the dependent variable index for *reaction\_handling*. E.g., usage might be: *rd\_data\_args*(*reaction\_rate*, *pr\_reac*) or *rd\_data\_args*(*reaction\_handling*, *pr\_reac*, *d*).

*rd\_ragged\_alloc*(*rd\_data*) Initializes *rd\_data\_size* and allocates space for *rd\_data\_base* (*rd\_data* is a root name here, either *reaction\_rate* or *reaction\_handling*). This macro needs to do nothing more because the reaction data are loaded in one reaction at a time, and the sizes are not known a priori.

*rd\_ragged\_realloc*(*rd\_data*, *rd\_tab\_index*, *subs*) Makes changes to size of one-dimensional data array *rd\_data\_tab* (*rd\_data* is a root name here, either *reaction\_rate* or *reaction\_handling*) based on a revised *rd\_tab\_index*. *subs* provides the *pr\_reac* array index (both cases) and the dependent variable index for *reaction\_handling*. The user must be adding to the end of the table for this to work properly.

## 47 Plasma-Material Interaction (PMI) data definitions

[/u/dstotler/degas2/src/pmidata.hweb]

\$Id: pmidata.hweb,v 1.9 2000/04/01 17:33:05 dstotler Exp \$

[/u/dstotler/degas2/src/pmidata.hweb] Data for Plasma Material Interactions is held in arrays in common blocks. *pd\_yield*(*i*, *j*, *k*, *l*, *m*, *p*) accesses the data for the *p*th PMI as an array with up to 5 dimensions. Data used in handling the PMI are stored in the *pd\_handling*(*i*, *j*, *k*, *l*, *m*, *d*, *p*) array with the *d* subscript denoting the *d*th dependent variable associated with the *p*th PMI.

"classes.f" 47.1 ≡

@m *pd\_yield*(*i*, *j*, *k*, *l*, *m*, *p*)

*pmi\_yield\_tab*<sub>*i*+pmi\_yield\_tab\_index<sub>*p*,1</sub>\*(*j*+pmi\_yield\_tab\_index<sub>*p*,2</sub>\*(*k*+pmi\_yield\_tab\_index<sub>*p*,3</sub>\*(*l*+pmi\_yield\_tab\_index<sub>*p*,4</sub>\**m*))))+pmi\_ye</sub>

@m *pd\_handling*(*i*, *j*, *k*, *l*, *m*, *d*, *p*)

*pmi\_handling\_tab*<sub>*i*+pmi\_handling\_tab\_index<sub>*p*,d,1</sub>\*(*j*+pmi\_handling\_tab\_index<sub>*p*,d,2</sub>\*(*k*+pmi\_handling\_tab\_index<sub>*p*,d,3</sub>\*(*l*+pmi\_handling\_ta</sub>

@m *pd\_data\_args*(*x*, *subs*)

*x##\_eval\_name##\_subs*, *x##\_rank##\_subs*, *x##\_var##\_subs*<sub>1</sub>, *x##\_tab\_index##\_subs*<sub>1</sub>,  
*x##\_spacing##\_subs*<sub>0</sub>, *x##\_min##\_subs*<sub>1</sub>, *x##\_delta##\_subs*<sub>1</sub>, *x##\_tab*<sub>0</sub>, *x##\_base##\_subs*

@m *pd\_dep\_var\_max* \$EVAL (*pf\_dep\_var\_max* − 1)

[/u/dstotler/degas2/src/pmidata.hweb] Variable definitions.

```
"classes.f" 47.2 ≡
 package_init(pd)

 define_dimen_pk(pd, pd_rank_ind, pf_table_rank_max)
 define_dimen_pk(pd, pd_dep_var_ind, pd_dep_var_max)
 define_dimen_pk(pd, pd_rank_ind0, 0, pf_table_rank_max)
 define_dimen_pk(pd, pd_tag_string, pf_tag_string_length)
 define_dimen_pk(pd, pd_eval_string, pf_eval_string_length)

 define_var_pk(pd, pmi_yield_size, INT)
 define_dimen_pk(pd, pd_yield_ind, 0, pmi_yield_size - 1)
 define_var_pk(pd, pmi_handling_size, INT)
 define_dimen_pk(pd, pd_handling_ind, 0, pmi_handling_size - 1)

 define_varp_pk(pd, pmi_yield_min, FLOAT, pd_rank_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_delta, FLOAT, pd_rank_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_rank, INT, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_spacing, INT, pd_rank_ind0, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_eval_name, CHAR, pd_eval_string, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_tab_index, INT, pd_rank_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_var, INT, pd_rank_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_num_rand, INT, problem_pmi_sub_ind)

 define_varp_pk(pd, pmi_yield_base, INT, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_yield_tab, FLOAT, pd_yield_ind)

 define_varp_pk(pd, pmi_handling_min, FLOAT, pd_rank_ind, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_delta, FLOAT, pd_rank_ind, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_rank, INT, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_spacing, INT, pd_rank_ind0, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_eval_name, CHAR, pd_eval_string, pd_dep_var_ind,
 problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_tab_index, INT, pd_rank_ind, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_var0, CHAR, pd_tag_string, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_var, INT, pd_rank_ind, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_num_rand, INT, problem_pmi_sub_ind)

 define_varp_pk(pd, pmi_handling_base, INT, pd_dep_var_ind, problem_pmi_sub_ind)
 define_varp_pk(pd, pmi_handling_tab, FLOAT, pd_handling_ind)
 define_varlocal_pk(pd, pd_inc, INT)
 package_end(pd)
```

[/u/dstotler/degas2/src/pmidata.hweb] Common blocks definitions.

/\* *pd\_ragged\_alloc* and *pd\_ragged\_realloc* are designed to automatically allocate memory for ragged data arrays. To save space, these arrays are packed into 1-D arrays and the multidimensionality is simulated through the macros at the top of this class. Two variables are used along with the main data table in this process. All three should be formed from the same “root” word or phrase; this simplifies the macro argument lists so that only *x* needs to be passed in. The *y* array provides the actual dimensions of the ragged array.

*x\_tab* One-dimensional array which will contain the data.

*x\_size* Integer variable representing the total size of *x\_tab*.

*x\_base* Multidimensional array which provides the “bookmarks” needed to separate the portions of *x\_tab*. The input variable *subs* represents the dependence on variables which the macro doesn’t need to know about (e.g., dependent variable index or reaction number).

*y* Dependence on indices external to this macros is again carried through the variable *subs*. The “rank” index used in the macros gives the size of the data required for this dependent variable in *x\_tab*. The number of nontrivial ( $\neq 1$ ; can’t use 0 as a trivial size because of the need to take products) entries in *y* gives the real dimensionality of this dependent variable. The value of *y* at each of these entries is the number of independent variables for that dimension.

*x\_base* just keeps track of where each dependent variable begins in *x\_tab*. The macro used to simulate the multidimensionality of *x\_tab* uses both *x\_base* and *y* to locate a particular entry. The size required for a dependent variable is just the product of its dimensions which can be computed (*pd\_set\_baseinc*) by looping over its rank and forming the product of the corresponding entries in *y*.

Since *x\_tab* has zero size at the beginning of the code, no initial allocation is required. However, *x\_size* must be initialized and memory for *x\_base* allocated. We do this in the disguise of *pd\_ragged\_alloc* for consistency with other macros which are expanded at the same time. \*/

"classes.f" 47.3  $\equiv$

```
@m pd_set_baseinc(x,y,subs) x##_base##subs = x##_size
 pd_inc = 1 $DO (I, range_min(ind_##y##_1), range_max(ind_##y##_1))
 {
 pd_inc = pd_inc * y##subs_I
 }
```

```
@m pd_ragged_alloc(x) x##_size = 0
 var_alloc(x##_base)
 call init_base(x##_base, array_size(x##_base))
```

```
@m pd_ragged_realloc(x,y,subs) pd_set_baseinc(x, y, subs)
 var_realloc(x##_tab, x##_size - 1, x##_size - 1 + pd_inc)
 x##_size = x##_size + pd_inc
```

/\* Although the end result of these macros is similar to that from those in *pmiformat.hweb*, the approach appears quite different. The reason is that in this case, none of the crucial dimensions are known at compile time. Instead, the base and increments are recomputed, and memory reallocated, not only as new data sets are read in, but also as each dependent variable is handled. \*/

## 48 PMI data class attribute descriptions

[/u/dstotler/degas2/src/pmidata.hweb]

[/u/dstotler/degas2/src/pmidata.hweb] Structures used to hold plasma-material interaction data for the current problem. Prefix is *pd*.

A given PMI is identified by an integer index *pr\_pmi* (“problem PMI”). This is the internal class corresponding to the “PMI format” class *pf*. It contains all of the data required to deal with the PMI in the problem. Not by accident, the elements of this class closely resemble those of the reaction data class *rd*. One difference is that fits have been made available for some of the PMI.

For clarity, here are the assumptions made about the data used to represent the fit and the associated information in this class:

1. *rank* provides the number of independent variables required to evaluate the fit; their identity is stored in the corresponding *var* array just as for tables.
2. The data table passed (the fit coefficients) is considered to be strictly 1-D with *tab\_index*<sub>1</sub> elements.
3. The multiplier *mult* does get applied to the fit coefficients. Hence, *mult*<sub>0</sub> should probably be set to one.
4. No spacing or value information is stored for the independent variables. Since this information is at least likely to be peculiar to each fit and at most irrelevant, it is assumed to reside within the fit function.
5. Eventually, we would like to store the fit function along with the data so that the data could be completely described by a single package, as can now be done with tables.

The PMI data class is divided into two similar subclasses. The first represents a single dependent variable, the “yield” of the PMI (e.g., the reflection coefficient, adsorption rate, sputtering yield, etc.; in effect, the probability of the process taking place). Hence, this sub-class lacks any explicit “dependent variable” dimension in the arrays. The *pmi\_handling* arrays contain the data required to process or “handle” *pr\_pmi*. The nature and names of the data objects within the *pmi\_handling* array can vary not only with the PMI, but with the model used to represent a given reaction.

The representation of the data in memory is crucial to efficiency. Since the main data tables here are six or seven dimensional objects of very “ragged” structure, we have opted for an internal representation as single dimensional arrays. To simplify coding, macros have been developed to simulate the familiar multi-dimensional array functionality. The downside of this is the addition of bookkeeping arrays (here, the *xxx\_base* arrays), the unwieldy-ness of the macros themselves, and the difficulty in extracting data from the 1-D array during debugging or while perusing the netCDF files. At least this last issue may be addressed by the construction of a small browser code to allow multi-dimensional netCDF data to be viewed and plotted in a straightforward manner. The macros are documented in more detail in *pmidata.hweb*.

*pmi\_yield\_size* Size of the 1-D array for the PMI yields; set by the macros *pd\_ragged\_alloc* and *pd\_ragged\_realloc*.

*pmi\_handling\_size* Size of the 1-D array for the handling data; set by the macros *pd\_ragged\_alloc* and *pd\_ragged\_realloc*.

*pmi\_yield\_min*<sub>*pr\_pmi,i*</sub> Floating point minimum value of the *i*th independent variable of the yield for *pr\_pmi*.

*pmi\_yield\_delta*<sub>*pr\_pmi,i*</sub> Floating point increment in the *i*th independent variable of the yield for *pr\_pmi*.

*pmi\_yield\_rank*<sub>*pr\_pmi*</sub> Integer number of independent variables for the yield for *pr\_pmi*.

*pmi\_yield\_spacing*<sub>*pr\_pmi,i*</sub> Integer identifying the way the *i*th variable for the yield of *pr\_pmi* (*i* = 0 refers to the yield itself) is to be interpolated. Examples are *pf\_spacing\_linear* and *pf\_spacing\_log*. Namely, in the log case the log of the independent variable (or dependent if *i* = 0) is interpolated linearly.

*pmi\_yield\_eval\_name*<sub>*pr\_pmi*</sub> String (length *pf\_eval\_string\_length*) providing the name of the function which will be used to evaluate the yield data. The most prevalent value is “table”; other values must correspond to the name of a fitting subroutine. See *plate.web* for more details.

*pmi\_yield\_tab\_index*<sub>*pr\_pmi,i*</sub> Number of values used for the *i*th independent variable of the yield for *pr\_pmi*.

*pmi\_yield\_var*<sub>*pr\_pmi,i*</sub> Integer identifying the *i*th independent variable for the yield. The independent variables are typically incident energy, and incident poloidal angle; see *pmiformat.hweb* for a full list.

*pmi\_yield\_num\_rand*<sub>*pr\_pmi*</sub> Integer number of uniform random variates required to evaluate the yield for *pr\_pmi*. This is provided for compatibility with the *pmi\_handling* variables and will usually be 0 here.

*pmi\_yield\_base*<sub>*pr\_pmi*</sub> Index in *pmi\_yield\_tab* at which the data for the yield of *pr\_pmi* begin. This array is set by the macros *pd\_ragged\_alloc* and *pd\_ragged\_realloc*.

*pmi\_yield\_tab*<sub>*macro\_index*</sub> Is the actual data object which holds the yield data. As described above, this is a 1-D array of size *pmi\_yield\_size*. These data are normally accessed through the macro interface *pd\_yield*.

*pmi\_handling\_min*<sub>*pr\_pmi,j,i*</sub> Floating point minimum value of the *i*th independent variable for the *j*th dependent variable in *pr\_pmi*.

*pmi\_handling\_delta*<sub>*pr\_pmi,j,i*</sub> Floating point increment in the *i*th independent variable for the *j*th dependent variable in *pr\_pmi*.

*pmi\_handling\_rank*<sub>*pr\_pmi,j*</sub> Integer number of independent variables for the *j*th dependent variable in *pr\_pmi*.

*pmi\_handling\_spacing*<sub>*pr\_pmi,j,i*</sub> Integer identifying the way the *i*th variable for the *j*th dependent variable (*i* = 0 refers to the dependent variable itself) is to be interpolated. Examples are *pf\_spacing\_linear* and *pf\_spacing\_log*. Namely, in the log case the log of the independent variable (or dependent if *i* = 0) is interpolated linearly.

*pmi\_handling\_eval\_name*<sub>*pr\_pmi,j*</sub> String (length *pf\_eval\_string\_length*) providing the name of the function or subroutine which will be used to evaluate the data for the *j*th dependent variable in handling *pr\_pmi*. The most prevalent value is “table”; other values must correspond to the name of a fitting subroutine. See *plate.web* for more details. Note that in some instances, the appropriate dependent “variable” is really a vector (i.e., velocity); in this case, tabular evaluation (which can return only a single scalar) makes no sense.

*pmi\_handling\_tab\_index*<sub>*pr\_pmi,j,i*</sub> Number of values used for the *i*th independent variable of the *j*th dependent variable for *pr\_pmi*.

*pmi\_handling\_var0*<sub>*pr\_pmi,j*</sub> String (length *pf\_tag\_string\_length*) identifying the *j*th dependent variable of *pr\_pmi*. A string is used here since the nature of the handling data varies from one PMI to the next.

*pmi\_handling\_var*<sub>*pr\_pmi,j,i*</sub> Integer identifying the *i*th independent variable for the *j*th dependent variable. The independent variables are typically incident energy, and incident poloidal angle; see *pmiformat.hweb* for a full list.

*pmi\_handling\_num\_rand<sub>pr\_pmi</sub>* Integer number of uniform random deviates required to handle a single interaction of *pr\_pmi*. The need for this became apparent when implementing the Bateman format. In this case, the *same random number* is needed to evaluate more than one dependent variable. *pmi\_handling\_num\_rand* lets the code maintain them through all of the required dependent variable evaluations.

*pmi\_handling\_base<sub>pr\_pmi,j</sub>* Index in *pmi\_handling\_tab* at which the data for the *j*th dependent variable of *pr\_pmi* begin. This array is set by the macros *pd\_ragged\_alloc* and *pd\_ragged\_realloc*.

*pmi\_handling\_tab<sub>macro\_index</sub>* Is the actual data object which holds the handling data. As described above, this is a 1-D array of size *pmi\_handling\_size*. These data are normally accessed through the macro interface *pd\_handling*.

[/u/dstotler/degas2/src/pmidata.hweb] Routines using PMI data.

*pd\_yield(k, l, m, n, o, pr\_pmi)* The programmer's representation of the yield data for *pr\_pmi*. *k, l, m, n*, and *o* represent the up-to-five dimensional dependency of the yield on its independent variables. *k* ranges from  $0 \rightarrow \text{pmi\_yield\_tab\_index}_{pr\_pmi,1} - 1$ ; similarly for *l* with  $i=2$ , *m*,  $i=3$ , etc.

*pd\_handling(k, l, m, n, o, j, pr\_pmi)* The programmer's representation of the handling data for *pr\_pmi*. *k, l, m, n*, and *o* represent the up-to-five dimensional dependency of the *j*th dependent variable on its independent variables. *k* ranges from  $0 \rightarrow \text{pmi\_handling\_tab\_index}_{pr\_pmi,j,1} - 1$ ; similarly for *l* with  $i=2$ , *m*,  $i=3$ , etc.

*pd\_data\_args(x, subs)* This macro provides an argument list of all of the *pmi\_yield* or *pmi\_handling* (use either of these as *x*) arrays for passing down to a subroutine. The *subs* argument provides the *pr\_pmi* array index (both cases) and the dependent variable index for *pmi\_handling*. E.g., usage might be: *pd\_data\_args(pmi\_yield, pr\_pmi)* or *pd\_data\_args(pmi\_handling, pr\_pmi,j)*.

*pd\_ragged\_alloc(pd\_data)* Initializes *pd\_data\_size* and allocates space for *pd\_data\_base* (*pd\_data* is a root name here, either *pmi\_yield* or *pmi\_handling*). This macro needs to do nothing more because the PMI data are loaded in one PMI at a time, and the sizes are not known a priori.

*pd\_ragged\_realloc(pd\_data, pd\_tab\_index, subs)* Makes changes to size of one-dimensional data array *pd\_data\_tab* (*pd\_data* is a root name here, either *pmi\_yield* or *pmi\_handling*) based on a revised *pd\_tab\_index*. *subs* provides the *pr\_pmi* array index (both cases) and the dependent variable index for *pmi\_handling*. The user must be adding to the end of the table for this to work properly.

## 49 String definitions

[/u/dstotler/degas2/src/string.hweb]

\$Id: string.hweb,v 1.9 2000/04/01 17:34:47 dstotler Exp \$

Declarations of the string routines.

```
"classes.f" 49 ≡
@f st_decls integer
@m st_decls external string_length, read_string, parse_string, next_token, string_lookup, int_lookup,
 read_real, read_integer, read_int_soft_fail, read_real_soft_fail
 integer string_length, parse_string, string_lookup, int_lookup, read_integer, read_int_soft_fail
 real read_real, read_real_soft_fail
 logical read_string, next_token
```

[/u/dstotler/degas2/src/string.hweb] A string without its trailing blanks

```
"classes.f" 49.1 ≡
@m SP $UNQUOTE('␣')
@m trim(s) sSP(1 : string_length(s))
```

[/u/dstotler/degas2/src/string.hweb] Read text from a string (does nothing)

```
"classes.f" 49.2 ≡
@m read_text(x) x
```

## 50 Macros for defining and manipulating arrays

[/u/dstotler/degas2/src/array.hweb]

[/u/dstotler/degas2/src/array.hweb] Macro definitions.

\$Id: array.hweb,v 1.29 1999/03/25 20:45:21 dstotler Exp \$

```
"classes.f" 50.1 ≡
@m MAX_DIM 5
@m CHAR 0
@m INT 1
@m FLOAT 2
@m COMMA ,
@m paste(x,y) _paste(x, y)
@m _paste(x,y) x##y
```

[/u/dstotler/degas2/src/array.hweb] Declaring dimensions and variables. Each vector that is declared has one or more dimensions which have names. These have to be defined first. Character variables have a name associated with their length. Each of these dimension names will have an upper and lower bound. @ f **define\_dimen** integer tells web to format the variables in **define\_dimen** as integers. The macro **define\_dimen** ( name ,  $\neq$  ) defines the lower and upper bounds of the dimension name **name**. It defines variables by prefixing min and max to the dimension name. If 1 arg. is given, it puts the lower limit (*min\_name*) =1, and upper limit (*max\_name*) = the first (and only) argument in **define\_dimen**. If 2 args. are given, it puts the lower limit as the first arg and the upper limit as the second.

"classes.f" 50.2  $\equiv$

```
@f define_dimen integer

/* #0 is the number of args represented by the dots.(atmost 2) ## pastes the strings on either side
of it after appropriate expansions, hence name is expanded to the variable's name. #n represents
the nth argument represented by the dots. */

@m define_dimen(name,...) $ASSERT (#0 $\equiv$ 1 $\vee$ #0 $\equiv$ 2)$IF (#0 $\equiv$ 1, $\sqcup$ $DEFINE(min_##name $\sqcup$ 1),\
$DEFINE(min_##name $\sqcup$ 1))$IF (#0 $\equiv$ 1, $\sqcup$ $DEFINE(max_##name $\sqcup$ 1),\
$DEFINE(max_##name $\sqcup$ 2))

/* Once the variables min_name and max_name are defined for each dimension name, the macro
range(name) defines the range of the name. range_size(name) defines the size (upper limit -
lower limit +1). */

@m range(name) range_min(name) : range_max(name)
@m rangea(name) range_min(name), range_max(name)
@m range_star(name) range_min(name) : *
@m range_stara(name) range_min(name) : range_min(name)
@m range_min1(name) min_##name
@m range_min(name) (range_min1(name))
@m range_max1(name) max_##name
@m range_max(name) (range_max1(name))
@m range_size(name) (range_max(name) - range_min(name) + 1)

@f define_var integer

/* The macro define_var defines variables of the form ind_x_n which are equated with the args.
to define_var represented by dots. The args. are names of the dimensions of the array x or
name of the length of character variable x. The macro defines a variable type_x (an integer
which tells us whether it is real(2), integer(1) or character(0)). x is substituted by the name of
the array/character variable and n is the no. of the arg. represented by the dots. rank_x is the
no. of args represented by the dots. */

// Suppose we have defined :
// define_dimen(a,0,10)
// define_dimen(b,1,10)
// define_dimen(c,5,20)
// define_var(temp,real,a,b,c)

// we get ind_temp_1 = a, ind_temp_2 =b, ind_temp_3 = c

/* ind_x_m equals the name of the last argument and ind_temp_m =c in the above example. */

@m define_var(x,type,...)
$ASSERT (#0 $\leq$ MAX_DIM $\wedge$ #0 $\geq$ 0)$ASSERT (type $\geq$ CHAR $\wedge$ type $\leq$ FLOAT)$ASSERT (type $\neq$
CHAR $\vee$ #0 $\geq$ 1)$IF ($\sqcup$ type $\equiv$ CHAR, $\sqcup$ $ASSERT(range_min(#1) $\equiv$ 1\
), $\sqcup$)$DEFINE (type_##x $\sqcup$ type)$DEFINE (rank_##x $\sqcup$ #0)$IF (#0 $\geq$ 1,\
 $\sqcup$ $DEFINE(ind_##x##_1 $\sqcup$ #1),)$IF (#0 $\geq$ 2, $\sqcup$ $DEFINE(ind_##x##_2 $\sqcup$ #2),)$IF (#0 $\geq$ 3,\
```

```

 $DEFINE(ind###_3_#3),)$IF (#0_>=_4,_$DEFINE(ind_###_4_#4),)$IF (#0_>=_5,\
 $DEFINE(ind###_5_#5),)$DEFINE (ind_###_m_$IFCASE(#0,0,#1,#2,#3,#4,#5))

@m define_varp(x,type,...)
 $ASSERT((type == CHAR ^ #0 ≥ 2) ∨ (type ≠ CHAR ^ #0 ≥ 1)) define_var(x, type, #.)
 /* Return type declaration (including character size) */
@if FORTRAN77
 @m char_spec(size) *(size)
#else
 @m char_spec(size) (len = size)
#endif

@m var_type(x) $IFCASE (type_##x,_character,_integer,_real)$IF (type_##x==_CHAR,\
 _char_spec(range_max(ind_###_1)),)

@m var_type_star(x)
 $IFCASE (type_##x,_character,_integer,_real)$IF (type_##x==_CHAR,_char_spec(*),)
 /* The explicit array shape, e.g., (0 : 1, 3 : 10) */
@m array_shape(x) $IF (type_##x==_CHAR,_,_$IFCASE(rank_##x,,\
 _(_range(ind_###_2)),_(_range(ind_###_2),\
 range(ind###_3)),_(_range(ind_###_2),range(ind_###_3),\
 range(ind###_4)),_(_range(ind_###_2),range(ind_###_3),\
 range(ind###_4),range(ind_###_5))),_,$IFCASE(rank_##x\
 ,_,_(_range(ind_###_1)),_(_range(ind_###_1),\
 range(ind###_2)),_(_range(ind_###_1),range(ind_###_2),\
 range(ind###_3)),_(_range(ind_###_1),range(ind_###_2),\
 range(ind###_3),range(ind_###_4)),_(_range(ind_###_1),\
 range(ind###_2),range(ind_###_3),range(ind_###_4),range(ind_###_5))))

/* The shape with the last bound unspecified, e.g., (0 : 1, 3 : *) */
@if FORTRAN77
@m array_shape_star(x) $IF (type_##x==_CHAR,_,_$IFCASE(rank_##x,,\
 _(_range_star(ind_###_2)),_(_range(ind_###_2),\
 _range_star(ind_###_3)),_(_range(ind_###_2),range(ind_###_3),\
 _range_star(ind_###_4)),_(_range(ind_###_2),range(ind_###_3),\
 range(ind###_4),range_star(ind_###_5))),_,$IFCASE(rank_##x\
 ,_,_(_range_star(ind_###_1)),_(_range(ind_###_1),\
 _range_star(ind_###_2)),_(_range(ind_###_1),range(ind_###_2),\
 _range_star(ind_###_3)),_(_range(ind_###_1),range(ind_###_2),\
 range(ind###_3),range_star(ind_###_4)),_(_range(ind_###_1\
 1),range(ind###_2),range(ind_###_3),range(ind_###_4),\
 _range_star(ind_###_5))))

/* The shape with the last size set to 1. This is because IBM's xlf doesn't like a pointer array to
have a starred last dimension */
@m array_shape_stara(x) $IF (type_##x==_CHAR,_,_$IFCASE(rank_##x,,\
 _(_range_stara(ind_###_2)),_(_range(ind_###_2),\
 _range_stara(ind_###_3)),_(_range(ind_###_2),range(ind_###_3),\
 _range_stara(ind_###_4)),_(_range(ind_###_2),range(ind_###_3),\
 range(ind###_4),range_stara(ind_###_5))),_,$IFCASE(rank_##x\
 ,_,_(_range_stara(ind_###_1)),_(_range(ind_###_1),\
 _range_stara(ind_###_2)),_(_range(ind_###_1),range(ind_###_2),\
 _range_stara(ind_###_3)),_(_range(ind_###_1),range(ind_###_2),\
 range(ind###_3),range_stara(ind_###_4)),_(_range(ind_###_1\
 1),range(ind###_2),range(ind_###_3),range(ind_###_4),\
 _range_stara(ind_###_5))))

```

```

1),range(ind_##x##_2),range(ind_##x##_3),range(ind_##x##_4),\
range_stara(ind_##x##_5)))
@#endif

/* A Fortran-90 style rank, e.g., (:,:) */
@#if FORTRAN90
@m array_rank(x)
$IF (type_##x##_==_CHAR, , $IFCASE(rank_##x, , , (:), (:, \
:), (:, :, :), (:, :, :, :)), $IFCASE(rank_##x, \
, (:), (:, :, :), (:, :, :, :), (:, :, :, :, :), \
(:, :, :, :, :)))
@#endif

/* Comma separated shape, e.g., 0, 1, 3, y. Character length is the first argument */
@#if FORTRAN90
@m array_arglistc(x, y)
$IF (type_##x##_==_CHAR, , $IFCASE(rank_##x, , , range_max(ind_##\
x##_1), , range_max(ind_##x##_1)_COMMA_range_min(ind_##x##_2)_COMMA_y, \
range_max(ind_##x##_1)_COMMA_rangea(ind_##x##_2)_COMMA_range_min(ind_##\
x##_3)_COMMA_y, , range_max(ind_##x##_1)_COMMA_rangea(ind_##\
x##_2)_COMMA_rangea(ind_##x##_3)_COMMA_range_min(ind_##x##_4)_COMMA_y, \
range_max(ind_##x##_1)_COMMA_rangea(ind_##x##_2)_COMMA_rangea(ind_##\
x##_3)_COMMA_rangea(ind_##x##_4)_COMMA_range_min(ind_##x##_5)_COMMA_y), \
$IFCASE(rank_##x, , , range_min(ind_##x##_1)_COMMA_y, \
rangea(ind_##x##_1)_COMMA_range_min(ind_##x##_2)_COMMA_y, \
rangea(ind_##x##_1)_COMMA_rangea(ind_##x##_2)_COMMA_range_min(ind_##\
x##_3)_COMMA_y, , rangea(ind_##x##_1)_COMMA_rangea(ind_##x##_\
_2)_COMMA_rangea(ind_##x##_3)_COMMA_range_min(ind_##x##_4)_COMMA_y, \
rangea(ind_##x##_1)_COMMA_rangea(ind_##x##_2)_COMMA_rangea(ind_##x##_\
_3)_COMMA_rangea(ind_##x##_4)_COMMA_range_min(ind_##x##_5)_COMMA_y))

@m array_arglist(x) array_arglistc(x, range_max(ind_##x##_m))

@m var_typea(x)
paste($IFCASE (type_##x, c, i, r), $IF (type_##x##_==_CHAR, $EVAL(rank_##x-1), rank_##x))
@#endif

@f declare_var integer

/* declare_var(x) declares the variable x. It may be a character, real or integer and can have 0
to MAX_DIM dimensions. For this to work, the values of ind_x.n and corresponding range must
be known through the define_dimen and define_var macros explained earlier. */

// suppose we have
// define_dimen (len_junk, 10)
// define_dimen (len_junk_array, 32)
// define_dimen (dim_junk_array, 1, 5)
// define_var (junk, CHAR, len_junk)
// define_var (junk_array, CHAR, len_junk_array, dim_junk_array)
// declare_var (junk)
// declare_var (junk_array)
// we get:
// character * 10 junk
// character * 32 junk_array(1 : 5)

// NB the syntax of the $IF statement is :

```

```

/* $IF (expression,true,false). note the position of the commas carefully. They separate the true
and false parts. */

@m declare_var(x) var_type(x)xarray_shape(x)

@f declare_varp integer

/* declare_varp(x) is similar to declare_var(x), but it declares a pointer to an array x . It is
necessary to declare a pointer to an array for dynamic allocation. So the last upper dimension is
not fixed. suppose we have : */

// define_dimen (len_junk_array, 32)
// define_dimen (lower_junk_array, 1)
// define_var (junk_array, CHAR, len_junk_array, lower_junk_array)
// declare_varp (junk_array)
// then we get:
// character * 32 junk_array(1 : *)
// pointer(ptr_junk_array, junk_array)

/* The last statement comes from the macro ptr_decl(x) which is explained later. */

@if FORTRAN77
@if ¬IBM
@m declare_varp(x) ptr_decl(x)

var_type(x)xarray_shape_star(x)
#else
@m declare_varp(x) ptr_decl(x)

var_type(x)xarray_shape_star(x)
#endif
#else
@m declare_varp(x) var_type(x), dimension array_rank(x), pointer :: x
#endif

/* array_size(x) is the total size of an array x. It is given as the product of the ranges of the
dimensions. The range is defined by a macro range_size and is basically - dimU - dimL +1 */

@m array_size(x)
$IFCASE (rank_##x,1,(range_size(ind_##x##_1)),,,,,,(range_size(ind_##x##_\
_1)*range_size(ind_##x##_2)),,,,,,(range_size(ind_##x##_\
_1)*range_size(ind_##x##_2)*range_size(ind_##x##_3)),,,,,,(range_size(ind_##x##_\
_1)*range_size(ind_##x##_2)*range_size(ind_##x##_3)*range_size(ind_##x##_4)),\
,,,,,(range_size(ind_##x##_1)*range_size(ind_##x##_2)*range_size(ind_##x##_3)*\
range_size(ind_##x##_4)*range_size(ind_##x##_5)))

/* array_unit(x) is the size calculated by leaving out the last dimension of the array x. */

@if FORTRAN77
@m array_unit(x)
$IFCASE (rank_##x,,1,,,,,(range_size(ind_##x##_1)),,,,,,(range_size(ind_##x##_\
_1)*range_size(ind_##x##_2)),,,,,,(range_size(ind_##x##_\
_1)*range_size(ind_##x##_2)*range_size(ind_##x##_3)),,,,,,(range_size(ind_##x##_\
_1)*range_size(ind_##x##_2)*range_size(ind_##x##_3)*range_size(ind_##x##_4)))
#endif

/* array_sizev(x, y) gives the size of an array x if its last dimension (upper limit) is y. This can
be used to calculate the new size of an array x if it's last dimension is changed. It's basically the
product of the size calculated by leaving out the last dimension (array_unit(x)) and the new

```

size of the last dimension which is just = (dimnU -dimnL +1) where dimnU is y and dimnL is known. ( lower limit of last dimension given by *range\_min(ind\_x\_m)* \*/

@#if FORTRAN77

@m array\_sizev(x,y) array\_unit(x)\*((y) - range\_min(ind\_###\_m) + 1)

@#endif

/\* var\_beg(x) gives the initial element of an array \*/

```
@m var_beg(x) x$IF (type_##x==_CHAR, , , \
 (range_min(ind_###_2)), (range_min(ind_##\
x##_2), range_min(ind_###_3)), (range_min(ind_##\
range_min(ind_###_3), range_min(ind_###_4)), (range_min(ind_##\
x##_2), range_min(ind_###_3), range_min(ind_###_4), range_min(ind_###_5))), \
 (range_min(ind_###_1)), \
 (range_min(ind_###_1), range_min(ind_###_2)), \
 (range_min(ind_###_1), range_min(ind_###_2), \
range_min(ind_###_3)), (range_min(ind_###_1), \
range_min(ind_###_2), range_min(ind_###_3), range_min(ind_###_4)), \
 (range_min(ind_###_1), range_min(ind_###_2), range_min(ind_###_3), \
range_min(ind_###_4), range_min(ind_###_5))))
```

[/u/dstotler/degas2/src/array.hweb] Array slices for passing to subroutines. I think we only need a full one-dimensional slice of numeric arrays and that's what's given here.

"classes.f" 50.3 ≡

@#if FORTRAN77

@m slice1(x,dims) xdims range\_min(ind\_###\_1)

@#else

@m slice1(x,dims) xdims :

@#endif

[/u/dstotler/degas2/src/array.hweb] Macros to simplify memory allocation

```

/* ptr_decl(x) declares a pointer to an array x and calls it ptr_x where x is the name of the array. */
classes.f" 50.4 ≡
#if FORTRAN77
@f ptr_decl integer
@m ptr_decl(x) pointer(ptr_##x, x)
@m mem_assign(x,p) ptr_#x = p
#else
@m mem_assign(x,p) xASSIGNp
#endif

/* real_alloc(x, n) allocates memory for a real array x of size n .real_realloc(x, on, n) reallocates
memory for an real array x which has a size on but needs a size n. Once the array has been
used, it's memory can be freed using real_free(x, n) */

#if FORTRAN77
@m real_alloc(x,n) ptr_##x = mem_alloc(n)
@m real_realloc(x,on,n) ptr_##x = mem_realloc(ptr_##x, on, n)
@m real_free(x,n) call mem_free(ptr_##x, n)
#endif

/* similar macros for allocating, reallocating and freeing memory for integers. they use the same
function calls, only the size is different, this is handled by using int_mem(n) instead of just
nwhich does the right thing */

#if FORTRAN77
@m int_alloc(x,n) real_alloc(x, int_mem(n))
@m int_realloc(x,on,n) real_realloc(x, int_mem(on), int_mem(n))
@m int_free(x,n) real_free(x, int_mem(n))
#endif

/* similar macros for allocating, reallocating and freeing memory for characters. they use the
same function calls, only the size is different, this is handled by using char_mem(n) instead of
just n which does the right thing */

#if FORTRAN77
@m char_alloc(x,n) real_alloc(x, char_mem(n))
@m char_realloc(x,on,n) real_realloc(x, char_mem(on), char_mem(n))
@m char_free(x,n) real_free(x, char_mem(n))
#endif

/* var_alloc(x), var_realloc(x, on, n) and var_free(x) are generalisations of the above macros
which were meant for a specific type. Here, the macro does the right thing depending on whether
the allocation, reallocation or freeing has to be done for a real, integer or character. It finds out
the size of the array too, using the macro array_size(x). note the use of array_sizev(x, y) to get
the size of x with the last dimension set to n and on */

#if FORTRAN77
@m var_alloc(x) $IFCASE(type_##x,uuuuuuchar_alloc(x,array_size(x)),uuuuuuint_alloc(x,\
array_size(x)),uuuuuu real_alloc(x,array_size(x)))
#else
@m var_alloc(x) xASSIGN paste(mem_alloc_, var_typea(x))(array_arglist(x), #x)
#endif

#if FORTRAN77

```

```

 @m var_realloc(x, on, n) $IFCASE (type_##x, \
 char_realloc(x, array_sizev(x, on), \
 array_sizev(x, n)), \
 int_realloc(x, array_sizev(x, on), array_sizev(x, n)), \
 real_realloc(x, array_sizev(x, on), array_sizev(x, n)))
@#else
 @m var_realloc(x, on, n) xASSIGN paste(mem_realloc_, var_typea(x))(x, array_arglistc(x, on), n, #x)
@#endif

@#if FORTRAN77
 @m var_free(x) $IFCASE (type_##x, \
 char_free(x, array_size(x)), \
 int_free(x, array_size(x)), \
 real_free(x, array_size(x)))
@#else
 @m var_free(x) call paste(mem_free_, var_typea(x))(x, array_arglist(x), #x)
@#endif

```

```

"classes.f" 50.5 ≡
 @m mem_inc 100

 /* mem_size(x) returns the size in the nearest unit of mem_inc for a given size x. eg.. mem_size(1),
 mem_size(2) ... mem_size(100) all return 100, but mem_size(101) ... mem_size(200) = 200 etc.
 */

 @m mem_size(x) (int(((x) + mem_inc - 1) / mem_inc) * mem_inc) // [x/mem_inc] mem_inc

 /* array_sizea(x) gives the size of an array x using the nearest multiple of mem_inc as the
 last range. for eg.. if mem_inc =5, then mem_size(1) to mem_size(5) = 5, mem_size(6) to
 mem_size(10) = 10 etc. now if x is x(1:5), array_sizea(x) is 1 × 5 = 5 and if x is x(1:6), it is 1 ×
 10 = 10. array_sizeb(x) gives the previous size of x using the nearest multiple of mem_inc as the
 last range. it is just the mem_size(x) using the last index minus mem_inc. eg.. if x is x(1:6),
 array_sizeb(x) = 1 × (mem_size(6) - 5) = 1 × (10 - 5) = 5 array_sizec(x, y) gives the size of an
 array x using the nearest multiple of mem_inc as the last range (same as array_sizea(x) so far).
 It uses the upper limit y for the last index instead of the original value. */

 @m last_sizea(x) mem_size(range_size(ind_##x##_m))
 @m last_sizeb(x) (mem_size(range_size(ind_##x##_m)) - mem_inc)
 @m last_sizec(x,y) mem_size((y) - range_min(ind_##x##_m) + 1)

@#if FORTRAN90
 @m last_ubound(x) range_max(ind_##x##_m)
 @m last_ubounda(x) (last_sizea(x) + range_min(ind_##x##_m) - 1)
 @m last_uboundb(x) (last_sizeb(x) + range_min(ind_##x##_m) - 1)
 @m last_uboundc(x,y) (last_sizec(x, y) + range_min(ind_##x##_m) - 1)
@#endif

@#if FORTRAN77
 @m array_sizea(x) array_unit(x) * last_sizea(x)
 @m array_sizeb(x) array_unit(x) * last_sizeb(x)
 @m array_sizec(x,y) array_unit(x) * last_sizec(x, y)
@#endif

 /* var_realloca(x) reallocates the size of an array x if it's growing in increments of 1, but
 increments it in steps of mem_inc when it's size has just exceeded the limit. since the value of
 the range at this point is like mem_inc *n +1 (n is an integer), mem_size (range_size(indx_m)
 gives the next higher multiple of its size. This is the new size. The old size is this value minus
 mem_inc which is what the macros array_sizea(x) and array_sizeb(x) return. */

@#if FORTRAN77
 @m var_realloca(x) if (mod(range_size(ind_##x##_m), mem_inc) ≡ 1) then
 $IFCASE (type_##x, char_realloc(x, array_sizeb(x), array_sizea(x)), \
 int_realloc(x, array_sizeb(x), array_sizea(x)), real_realloc(x, \
 array_sizeb(x), array_sizea(x)))
 end if
@#else
 @m var_realloca(x) if (mod(range_size(ind_##x##_m), mem_inc) ≡ 1) then
 var_realloc(x, last_uboundb(x), last_ubounda(x))
 end if
@#endif

 /* var_reallocb(x) sets the size of the array to the actual size at that time given by array_size ,
 if the range size of the last index is not a multiple of mem_inc (which means the growth has
 saturated) */

```

**@#if** *FORTRAN77*

```

@m var_reallocb(x) if (mod(range_size(ind_##x##_m), mem_inc) $\neq$ 0) then
 $IFCASE (type_##x, \char_realloc(x, array_sizea(x), array_size(x)), \
 \int_realloc(x, array_sizea(x), array_size(x)), \real_realloc(x, \
 array_sizea(x), array_size(x)))
end if

```

**@#else**

```

@m var_reallocb(x) if (mod(range_size(ind_##x##_m), mem_inc) $\neq$ 0) then
 var_realloc(x, last_ubounda(x), last_ubound(x))
end if

```

**@#endif**

*/\* var\_reallocc(*x*, *on*, *n*) reallocates the size of an array *x* if the sizes of *x* corresponding to the old value of upper limit of the last dim. (*on*) rounded off to the nearest multiple of *mem\_inc* and the size of *x* corresponding to the new value of upper limit of the last dim. (*n*) rounded off to the nearest multiple of *mem\_inc* are not the same. \*/*

**@#if** *FORTRAN77*

```

@m var_reallocc(x, on, n)
if (mem_size((on) - range_min(ind_##x##_m) + 1) $\neq$ mem_size((n) - range_min(ind_##x##_m) + 1))
 then
 $IFCASE (type_##x, \char_realloc(x, array_sizec(x, on), array_sizec(x, n)), \
 \int_realloc(x, array_sizec(x, on), array_sizec(x, n)), \real_realloc(x, \
 array_sizec(x, on), array_sizec(x, n)))
end if

```

**@#else**

```

@m var_reallocc(x, on, n)
if (mem_size((on) - range_min(ind_##x##_m) + 1) $\neq$ mem_size((n) - range_min(ind_##x##_m) + 1))
 then
 var_realloc(x, last_uboundc(x, on), last_uboundc(x, n))
end if

```

**@#endif**

[/u/dstotler/degas2/src/array.hweb] Define package functions.

/\* Categories of package definition macros \*/

```
"classes.f" 50.6 ≡
@m PK_DUMMY 0
@m PK_DIMEN 1
@m PK_VAR 2
@m PK_VARP 3
@m PK_VARLOCAL 4
@m PK_CAT_MIN PK_DUMMY
@m PK_CAT_MAX PK_VARLOCAL

@m register(pk,n,x,cat) _register(pk, n, x, cat)
@m _register(pk,n,x,cat) $DEFINE (VAR_##pk##_##n_x)$DEFINE (CAT_##pk##_##n_cat)

@m package_init(pk) $DEFINE (COUNT_##pk_0)$DEFINE (pk##_common_process_pk(pk,\
common))$DEFINE (pk##_module_process_pk(pk,\
module))$DEFINE (pk##_ncdef(fileid)_process_pk(pk,ncdef,\
fileid))$DEFINE (pk##_ncwrite(fileid)_process_pk(pk,\
ncwrite,fileid))$DEFINE (pk##_ncread(fileid)_proces\
s_pk(pk,ncread,fileid))$DEFINE (pk##_ncdecl_process_pk(pk,\
ncdecl))$DEFINE (pk##_pvmput_process_pk(pk,\
pvmput))$DEFINE (pk##_pvmget_process_pk(pk,\
pvmget))$DEFINE (pk##_mpibcast_process_pk(pk,\
mpibcast))$DEFINE (pk##_mpisend(dest,tag)_process_pk(pk,mpisend,dest,\
tag))$DEFINE (pk##_mpireceive(source,tag)_process_pk(pk,mpireceive,source,tag))

@m package_end(pk)

@m define_dimen_pk(pk,name,...)
define_dimen(name, #.) $INCR (COUNT_##pk)register(pk, COUNT_##pk, name, PK_DIMEN)

@m define_dummy_pk(pk,name,type,...)
define_var(name, type, #.) $INCR (COUNT_##pk)register(pk, COUNT_##pk, name, PK_DUMMY)

@m define_var_pk(pk,name,type,...)
define_var(name, type, #.) $INCR (COUNT_##pk)register(pk, COUNT_##pk, name, PK_VAR)

@m define_varp_pk(pk,name,...)
define_varp(name, #.)$INCR (COUNT_##pk)register(pk, COUNT_##pk, name, PK_VARP)

@m define_varlocal_pk(pk,name,type,...) define_var(name, type,
#.) $INCR (COUNT_##pk)register(pk, COUNT_##pk, name, PK_VARLOCAL)
```

[/u/dstotler/degas2/src/array.hweb] Iterating macros. These iterate over all the entities in a package, invoking a macro for entity.

```
"classes.f" 50.7 ≡
@m process_pk(pk, module,...)
 $DEFINE (TEMPCOUNT_0) start_##module(pk, #.)iterate_pk(pk, module, #.)end_##module(pk, #.)
 // The "*" here is significant

@m* iterate_pk(pk, module,...) $INCR (TEMPCOUNT)$IF (TEMPCOUNT_0<=_COUNT_##pk,\
 \#####do_##module(pk,paste(VAR_##pk##_ ,TEMPCOUNT),paste(CAT_##pk##_,\
 TEMPCOUNT),#.)#####iterate_pk(pk,module,#.)#####)

@#if FORTRAN77
@m start_common(pk)
 $DEFINE (COMMON_C_0)$DEFINE (COMMON_I_0)$DEFINE (COMMON_R_0)$DEFINE (COMMON_P_0)

@m end_common(pk)
 $IF (COMMON_C_0>_0,save_/pk##_com_c/;,$IF (COMMON_I_0>_0,save_/pk##_com_i/;,\
)$IF (COMMON_R_0>_0,save_/pk##_com_r/;,$IF (COMMON_P_0>_0,save_/pk##_com_p/;,)

@m do_common(pk, var, cat) $ASSERT (cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE (cat,\
 \#####declare_var(var);#####$IFCASE (paste(type_,var),\
 \#####common_/pk##_com_c/_var;_$INCR(COMMON_C),#####common_/pk##_\
 com_i/_var;_$INCR(COMMON_I),#####common_/pk##_com_r/_var;_$INCR(COMMON_R\
)) ,#####declare_varp(var);common_/pk##_com_p/_paste(ptr_,\
 var);_$INCR(COMMON_P),#####declare_var(var);#####$IFCASE (paste(type\
 _ ,var),#####common_/pk##_com_c/_var;_$INCR(COMMON_C),\
 \#####common_/pk##_com_i/_var;_$INCR(COMMON_I),#####common_/pk##_\
 com_r/_var;_$INCR(COMMON_R)),#####)

@#else
 @m start_common(pk)
 @m end_common(pk) usepk##_mod
 @m do_common(pk, var, cat)

@#endif

@#if FORTRAN90

@#if 0 // This version only outputs module, and end module if there's a body.
 // This break things with MPI on non-MPI builds.
 // This follow version fixes this.
@m start_module(pk) $DEFINE (MODULE_SIZE_0)

@m end_module(pk) $IF (MODULE_SIZE_0>_0, end_module_pk##_mod;,)

@m do_module(pk, var, cat) $ASSERT (cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE (cat,\
 \#####$IF (MODULE_SIZE_0==_0, module_pk##_mod; implicit\
 t_none_f90mod; save;,)#####declare_var(var);#####$INCR(MODULE_SIZE),\
 \#####$IF (MODULE_SIZE_0==_0, module_pk##_mod; implicit_none_f90mod; save;,\
)#####declare_varp(var);#####$INCR(MODULE_SIZE),\
 \#####$IF (MODULE_SIZE_0==_0, module_pk; implicit_none_f90mod; save;,\
)#####declare_var(var);#####$INCR(MODULE_SIZE),#####)

@#else
 @m start_module(pk) modulepk##_mod
 implicit_none_f90mod save

 @m end_module(pk) end modulepk##_mod
```

```
@m do_module(pk, var, cat) $ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 declare_var(var);\
 declare_varp(var);\
 declare_var(var);\
@#endif
@#else
@m start_module(pk)
@m end_module(pk)
@m do_module(pk, var, cat)
@#endif

@m start_ncdef(pk, fileid)
@m do_ncdef(pk, var, cat, fileid)
$ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 dimen_def(fileid,var);\
 var_def(fileid,var);\
 var_def(fileid,var);,\
@m end_ncdef(pk, fileid)
@m start_ncdecl(pk)
@m do_ncdecl(pk, var, cat) $ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 integer_dimid(var);\
 integer_dimid(var);\
 integer_dimid(var);,\
@m end_ncdecl(pk)
@m start_ncwrite(pk, fileid)
@m do_ncwrite(pk, var, cat, fileid)
$ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 var_write(fileid,var);\
 var_write(fileid,var);\
@m end_ncwrite(pk, fileid)
@m start_ncread(pk, fileid)
@m do_ncread(pk, var, cat, fileid)
$ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 dimen_inq(fileid,var);\
 var_inq(fileid,var);\
 var_read(fileid,\
var);\
 var_inq(fileid,var);\
 var_alloc(var);\
 var_read(fileid,var);\
 \
@m end_ncread(pk, fileid)
@m start_pvmput(pk)
@m do_pvmput(pk, var, cat) $ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 pvm_put(var);\
 pvm_put(var);\
 \
@m end_pvmput(pk)
@m start_pvmget(pk)
@m do_pvmget(pk, var, cat)
$ASSERT(cat ≥ PK_CAT_MIN ∧ cat ≤ PK_CAT_MAX)$IFCASE(cat,\
 pvm_get(var);\
 var_alloc(var);\
 pvm_get(var);\
 \
@m end_pvmget(pk)
@m start_mpibroadcast(pk)
```

```
@#if MPI
 @m do_mpibcast(pk, var, cat) $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, \
 ,,,,,,,,,,,,,,,mpi_broadcast(var);,,,,,,,,,,,,,,if(,mpi_rank!=,mp\
 i_root),then;,var_alloc(var);,endif;mpi_broadcast(var);,,,,,,,,,,,,,,)
@#else
 @m do_mpibcast(pk, var, cat) $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, \
 ,,,,,,,,,,,,,,,)
@#endif

@m end_mpibcast(pk)

@m start_mpisend(pk, dest, tag)

@#if MPI
 @m do_mpisend(pk, var, cat, dest, tag)
 $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, ,,,,,,,,,,,,,,, \
 ,,,,,,,,,,,,,,mpi_send(var,dest,tag);, ,,,,,,,,,,,,,,mpi_send(var,dest,tag);, ,,,,,,,,,,,,,,)
@#else
 @m do_mpisend(pk, var, cat, dest, tag)
 $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, ,,,,,,,,,,,,,,, \
 ,,,,,,,,,,,,,,)
@#endif

@m end_mpisend(pk, dest, tag)

@m start_mpireceive(pk, source, tag)

@#if MPI
 @m do_mpireceive(pk, var, cat, source, tag)
 $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, ,,,,,,,,,,,,,,, \
 ,,,,,,,,,,,,,,mpi_receive(var,source,tag);, ,,,,,,,,,,,,,,var_alloc(var);,mpi_receive(var, \
 source,tag);, ,,,,,,,,,,,,,,)
@#else
 @m do_mpireceive(pk, var, cat, source, tag)
 $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, ,,,,,,,,,,,,,,, \
 ,,,,,,,,,,,,,,)
@#endif

@m end_mpireceive(pk, source, tag)

@m do_netcdfa(pk, var, cat)
 $ASSERT($cat \geq PK_CAT_MIN$ $\wedge$ $cat \leq PK_CAT_MAX$)$IFCASE(cat, ,,,,,,,,,,,,,,netcdf0(var);, \
 ,,,,,,,,,,,,,,netcdf1(var);, ,,,,,,,,,,,,,,netcdf2(var);, ,,,,,,,,,,,,,,netcdf3(var);, ,,,,,,,,,,,,,)
```

## 51 Description of array definition macros

[/u/dstotler/degas2/src/array.hweb]

These macros aid in defining array operations. The way the macros are used is outlined below; they are largely invoked in other header files. The primary exceptions to this rule are the pointer-related macros, *var\_alloc*, *var\_realloc*, and *var\_free*. Perhaps the *best* way to see what these macros do is to examine the FORTRAN code produced by the WEB preprocessor.

The macros used most frequently are those associated with “package definition”:

*package\_init(CL)* This would be the first macro executed for the class with abbreviation *CL*. It begins the package definition process by telling WEB to define “variables” (really they are macros the names of which are defined “on-the-fly” using the class abbreviations) like *CL\_common\_b*, *CL\_ncreada*, and *CL\_pvmput*. Then, in the actual WEB code, the programmer uses these phrases to symbolize a complete set of declarations or definitions for this class. This set of definitions, etc. are built up by subsequent macro calls made in the header file as the various dimensions and variables are defined. It is worth going through these macros or “variables” one by one:

*CL\_common\_a* Produces common blocks containing scalar (FORTRAN) variables. This includes a “save” statement for each one. The suffix “\_a” is used to represent scalars; likewise, “\_b” is used for “dimensions” (see below) and “\_c” for arrays. Note that typically one finds in the header file for the class a macro *CL\_common* which concatenates one or more of these macros into a single one.

*CL\_ncdef* Handles the dimensions and variable definitions required for the “definition” phase of creating a new netCDF file.

*CL\_ncwrite* Writes all of the netCDF variables in class *CL* into the netCDF file specified.

*CL\_ncread* Reads all of the netCDF variables in class *CL* from the specified netCDF file into the FORTRAN code.

*CL\_ncdecl* Declares for the FORTRAN compiler the integer identifiers netCDF uses to identify the various dimensions and variables in the class *CL*.

*CL\_pvmput* Associated with PVM...

*CL\_pvmget* Associated with PVM...

*define\_dimen\_pk(CL, dimname,...)* This defines a dimension, i.e., assigns a name, *dimname*, to an integer range. This macro is primarily required only for the netCDF-related macros listed above; the resulting “dimensions” do not appear in the final FORTRAN code except as numbers in these netCDF calls.

*define\_var\_pk(CL, varname, type,...)* Defines a FORTRAN variable *varname*. This can be a scalar, array, or a character variable. *varname* is also added to the corresponding netCDF and PVM macro definitions.

*define\_varp\_pk(CL, varname,...)* Defines a FORTRAN pointer to be associated with *varname*. Again, the macro appends entries to all of the corresponding netCDF and PVM macro definitions. Note that in the case of *CL\_ncread*, the macro *var\_alloc(varname)* (see below) is included. In general, a pointer should be used for variables with dimensions which are not known at compile time.

These “package” macros are largely built up on the other macros defined in this header file. The most significant of these are:

**define\_dimen**(*dimname*, min, max) Defines a dimension range *dimname* as running from min to max; produces no code.

**define\_var**(*array*, *type*, *dimname1*, *dimname2*,...) Defines *array* of type *type* (*CHAR*, *INT*, or *FLOAT*) as having first dimension *dimname1* (as declared with a **define\_dimen** or *define\_dimen\_pk* macro), and so on. The number of dimensions in the argument list is the rank of *array*. This macro is used for scalars as well (rank 0). Again, no code is produced by this macro.

**declare\_var**(*varname*) Once a variable *varname* is defined, this macro is used to actually declare it in the FORTRAN code; the result is a type (including dimensions for arrays and string length for character strings) declaration statement. The variable type is handled automatically by the macro. Note that for character strings, the first dimension provides the length of each element of the array. This makes interaction with netCDF easier where only single character variables are permitted.

**declare\_varp**(*varname*) Same as **declare\_var**, but also uses **ptr\_decl** to actually declare the pointer variable itself (see below).

*array\_size*(*arrayname*) Gives the size of *arrayname*, i.e., the total number of elements in an array.

*array\_unit*(*arrayname*) Number of elements in one “slice” (i.e., one unit of the last array index) of the array *arrayname*.

*array\_sizev*(*arrayname*, *newmax*) Gives the size of the array *arrayname* if the upper limit of the last index were set to *newmax*.

**ptr\_decl**(*arrayname*) Declares a pointer names *ptr\_arrayname* to be associated with the array *arrayname*.

*var\_alloc*(*arrayname*) Allocates space for array *arrayname*; this assumes that all of its dimensions have been defined.

*var\_realloc*(*arrayname*, *oldmax*, *newmax*) Is used when there is an increase in the last dimension of an array *arrayname*; *oldmax* is the old value for the upper limit and *newmax* is the new value.

*var\_free*(*arrayname*) Deallocates the memory associated with the array *arrayname*.

*var\_realloca*(*arrayname*) Used to reallocate space when the last dimension of array *arrayname* grows by one. The allocation will only take place every *mem\_inc* (defined in this file) steps.

*var\_reallocb*(*arrayname*) Once *arrayname* has grown to its final size (via repeated invocations of *var\_realloca*), this macro is used to repack the array in the smallest possible space.

*var\_reallocc*(*arrayname*, *oldmax*, *newmax*) Used to reallocate space when the last dimension of array *arrayname* is growing by steps other than one; *oldmax* and *newmax* are the old and new values of the upper limit of the last dimension, respectively.

If one wishes to understand more completely these macros, it is useful to be familiar with some of the predefined WEB macros. Throughout the macro definitions also be aware of which quantities are the arguments of the macros, which are other macros, and which are strings meant to appear in the final FORTRAN code. The binary operators used in the macros are the same as those used in C; see the section entitled “Expression Evaluation” in the WEB manual if these are unfamiliar to you.

**@f** Denotes a formatting declaration for WEB; affects only the typeset output.

**@m** A macro definition.

**##** Is the concatenation or paste operator. It is typically used to paste a string argument to a macro into a string inside the macro definition. For example, if we used somewhere **ptr\_decl**(*foo*), the definition *ptr\_##x* would be expanded as *ptr\_foo* in the FORTRAN code.

... When an ellipsis appears in an argument list, it means that this macro has a variable number of arguments. The ellipsis may be preceded by specified arguments as well.

**#0** When the macro has a variable number of arguments, ‘**#0**’ expands during WEB processing into the actual number of variable arguments used in this particular invocation of the macro; note that this number does not include any arguments explicitly specified at the beginning of the macro argument list.

**#1** Also for a variable number of arguments, ‘**#n**’ with  $n > 0$  is replaced during WEB processing by the  $n$ -th argument in the actual invocation of the macro. Again, this does not count any explicit arguments in the macro definition.

**\$DEFINE** This WEB macro is typically used to assign a particular value, usually obtained from a macro argument, to a local (i.e., to the preprocessor) variable, the name of a which is also likely constructed using a macro argument. The other macros whose names begin with an underscore such as “**\$IFCASE**” are described in the WEB manual; their usage should be obvious. The most effective use of the “**\$DEFINE**” macro arises in the construction lengthy package declarations such as *pk\_ncdecl* where *pk* represents one of the two letter class designations. Through repeated use of “**\$DEFINE**” and “**\$UNDEF**”, these macros are built up piece by piece during the execution of other macros such as *define\_dimen\_pk* and *define\_var\_pk*.

## 52 Macros used to read and write netCDF files

[/u/dstotler/degas2/src/netcdf.hweb]

Revision 1.4 1993/06/01 22:15:47 karney Redo pre-Albuquerque checkin

Revision 1.4 1993/04/24 23:16:18 karney Pre-Albuquerque checkin

Revision 1.3 1993/03/17 20:35:15 karney Prior to array reorganization

Revision 1.2 1992/08/20 14:54:05 karney Convert to LaTeX/NFSS

[/u/dstotler/degas2/src/netcdf.hweb] Macro definitions from `netcdf.inc`

\$Id: netcdf.hweb,v 1.14 1999/05/25 13:14:09 dstotler Exp \$

```
"classes.f" 52.1 ≡
 @m nccreate nccre // Longer versions of subroutine names
 @m ncopen ncopn
 @m ncredef ncredf
 @m ncendef ncendf
 @m ncclose ncclos
 @m ncattput ncapt
 @m ncattputc ncaptc
 @m ncattget ncagt
 @m ncattgetc ncagtc
 @m ncdimdef ncddef
 @m ncvardef ncvdef
 @m ncvarput ncvpt
 @m ncvarputc ncvptc
 @m ncvarget ncvg
 @m ncvargetc ncvgtc
 @m ncdimid ncdid
 @m ncdiminq ncdinq
 @m ncvarid ncvid
 @m ncvarinq ncvinq
 @m ncvarget ncvg
 @m ncvargetc ncvgtc

 @m NC_CHAR 2
 @m NC_BYTE 1
 @m NC_SHORT 3
 @m NC_LONG 4
 @m NC_FLOAT 5
 @m NC_DOUBLE 6
 @#if defined (NETCDFV2)
 @m NC_CLOBBER 11
 @m NC_NOCLOBBER 15
 @#else
 @m NC_CLOBBER 0
 @m NC_NOCLOBBER 4
 @#endif
 @m NC_WRITE 1
 @m NC_NOWRITE 0
 @m NC_GLOBAL 0
 @m NC_FATAL 1
 @m NC_VERBOSE 2

 @m NC_REAL NC_DOUBLE
 @m NC_INT NC_LONG
 @m MAX_NC_NAME 128
```

[/u/dstotler/degas2/src/netcdf.hweb] Routines to declare the dimensions.

```
"classes.f" 52.2 ≡
@f nc_decls integer
@m nc_decls integer nc_dims(MAX_DIM), nc_corner(MAX_DIM), nc_edge(MAX_DIM), nc_stat,
 nc_size, nc_rank, nc_attr, nc_type
character*MAX_NC_NAME nc_dummy
external ncopen, ncvarid, ncdimid, nccreate, ncvardef, ncdimdef
integer ncopen, ncvarid, ncdimid, nccreate, ncvardef, ncdimdef

@m dimid1(name) name##_id
@m dimid(name) dimid1(name)

@m dimen_def(fileid,name) dimid(name) = ncdimdef(fileid, #name, range_size(name), nc_stat)

@m var_def(fileid,name) $IF (rank_###name_>=1,nc_dims(1)=dimid(ind_###name\
e##_1);,) $IF (rank_###name_>=2,nc_dims(2)=dimid(ind_###name\
e##_2);,) $IF (rank_###name_>=3,nc_dims(3)=dimid(ind_###name##_3);,\
) $IF (rank_###name_>=4,nc_dims(4)=dimid(ind_###name##_4);,) $IF (rank_###name_>=5,\
nc_dims(5)=dimid(ind_###name##_5);,) dimid(name) = ncvardef(fileid, #name,\
$IFCASE (type_###name,NC_CHAR,NC_INT,NC_REAL), rank_###name, nc_dims, nc_stat)

@m var_write(fileid,name)
$IF (rank_###name_>=1,nc_corner(1)=1;nc_edge(1)=range_size(ind_###name##_1);,\
) $IF (rank_###name_>=2,nc_corner(2)=1;nc_edge(2)=range_size(ind_###name##_2);,\
) $IF (rank_###name_>=3,nc_corner(3)=1;nc_edge(3)=range_size(ind_###name##_3);,\
) $IF (rank_###name_>=4,nc_corner(4)=1;nc_edge(4)=range_size(ind_###name##_4);,\
) $IF (rank_###name_>=5,nc_corner(5)=1;nc_edge(5)=range_size(ind_###name##_5\
);,) $IF (type_###name==NC_CHAR,call_ncvarputc(fileid,dimid(name),\
nc_corner,nc_edge,name,array_size(name),nc_stat),call_ncvarput(fileid,\
dimid(name),nc_corner,nc_edge,name,nc_stat))

@m dimen_inq(fileid,name) dimid(name) = ncdimid(fileid, #name, nc_stat)
call ncdiminq(fileid, dimid(name), nc_dummy, nc_size, nc_stat)
assert(nc_size == range_size(name))

@m var_inq(fileid,name) dimid(name) = ncvarid(fileid, #name, nc_stat)
call ncvarinq(fileid, dimid(name), nc_dummy, nc_type, nc_rank, nc_dims, nc_attr, nc_stat)
$IF (rank_###name_>=1,assert(nc_dims(1)==dimid(ind_###name##_1));,\
) $IF (rank_###name_>=2,assert(nc_dims(2)==dimid(ind_###name##_2));,\
) $IF (rank_###name_>=3,assert(nc_dims(3)==dimid(ind_###name##_3));,\
) $IF (rank_###name_>=4,assert(nc_dims(4)==dimid(ind_###name##_4));,\
) $IF (rank_###name_>=5,assert(nc_dims(5)==dimid(ind_###name##_5));,)

@if 0 /* These used to be part of var_inq. Removed to avoid tangle problems. */
assert(nc_rank == rank_###name)
assert(nc_rank >= 0 & nc_rank <= MAX_DIM)
assert($IFCASE (type_###name,NC_CHAR,NC_INT,NC_REAL) == nc_type)
@endif

@m var_read(fileid,name)
$IF (rank_###name_>=1,nc_corner(1)=1;nc_edge(1)=range_size(ind_###name##_1);,\
) $IF (rank_###name_>=2,nc_corner(2)=1;nc_edge(2)=range_size(ind_###name##_2);,\
) $IF (rank_###name_>=3,nc_corner(3)=1;nc_edge(3)=range_size(ind_###name##_3);,\
) $IF (rank_###name_>=4,nc_corner(4)=1;nc_edge(4)=range_size(ind_###name##_4);,\
) $IF (rank_###name_>=5,nc_corner(5)=1;nc_edge(5)=range_size(ind_###name##_5\
);,) $IF (type_###name==NC_CHAR,call_ncvargetc(fileid,dimid(name),\
```

```
nc_corner,nc_edge,name,array_size(name),nc_stat),,,,,,,,,call_ncvarget(fileid,\
dimid(name),nc_corner,nc_edge,name,nc_stat))
```

## 53    Description of netCDF Macros

[/u/dstotler/degas2/src/netcdf.hweb]

The purpose of these macros is to reduce the amount of effort the user has to expend to read and write netCDF files. In particular, since the WEB preprocessor already knows the type and dimensioning information for every variable in the header files, it can generate the required netCDF function calls on the fly during processing. In order to fully comprehend these macros, it is useful to be familiar with the concepts of netCDF files and some of the details about the routines used to deal with them. Perhaps the most helpful information can be found within the “Info” feature of *Emacs* on the PPPL Theory workstations.

Note that the programmer is not likely to call directly any of the macros defined in this file. Rather, one would be more likely to use the netCDF-related package macros defined in `array.hweb`. At the same time, a few of the netCDF routines are simple enough that they are called directly. Let us consider a hypothetical class, `cl`. Here is a typical sequence of subroutine and macro calls used to create and write a new netCDF file:

```
integer fileid \\ With other declarations
cl_ncdecl \\ Declare netCDF id integers
.
.
.
fileid = nccreate('cl.nc',NC_CLOBBER,nc_stat) \\ Enter define mode
call ncattputc(fileid,NC_GLOBAL,'data_version', \\ Put attributes
$ NC_CHAR,string_length(cl_version),
$ cl_version,nc_stat)
cl_ncdef(fileid)
call ncendef(fileid,nc_stat)
cl_ncwrite(fileid) \\ Now in data mode
call ncclose(fileid,nc_stat)
```

The steps used to read an existing file generally look like:

```
integer fileid \\ With other declarations
cl_ncdecl \\ Declare netCDF id integers
.
.
.
fileid = ncopen('cl.nc',NC_NOWRITE,nc_stat)
call ncattgetc(fileid,NC_GLOBAL,'data_version',
$ cl_version,len(cl_version),nc_stat) \\ Get attributes
cl_ncread
call ncclose(fileid,nc_stat)
```

These macros are extremely effective. It is instructive to look at the FORTRAN code generated by the WEB preprocessor to appreciate just how much effort these macros save. For information on these macros, see the documentation associated with the header file `array.hweb`.

Looking more closely now at the macros defined in this header file, we see that they are minimal extensions of the actual netCDF routines.

*dimen\_def(fileid, dimname)* Obtains the size of dimension *dimname* and constructs the line of FORTRAN required to call *ncdimdef*; that routine actually defines the dimension for file *fileid*. Note that *#name* in the macro definition instructs WEB to “stringize” the macro argument *name* (for FORTRAN, place in single quotes) .

*var\_def(fileid, varname)* Checks the rank of the variable *varname*, writes its dimensions to the FORTRAN code via the local array *nc\_dims*, and then assembles the call to *ncvardef*, including the type declaration, which defines *varname* in file *fileid*.

*var\_write(fileid, varname)* First writes out to the FORTRAN code the length of every “edge” (one for each dimension) of the variable *varname*; this is done using the local arrays *nc\_corner* and *nc\_edge*. Then, either the netCDF routine *ncvarputc* (for character strings) or *ncvarput* is called to actually write the data to the file.

*dimen\_inq(fileid, dimname)* Creates two lines of code. The first obtains from the netCDF file the value of the identifier for dimension *dimname*; this will be stored in a local integer variable generated by the *dimid(dimname)* macro. The second line of code obtains the size of the dimension using *ncdiminq*. This is checked against the expected size.

*var\_inq(fileid, varname)* Generates two lines of code. The first obtains from the netCDF file the value of the identifier for variable *varname*; this will be stored in a local integer variable generated by the *dimid(varname)* macro. The second line of code obtains the rank, type, and size of *varname*. Each of these is checked in turn against the expected values.

*var\_read(fileid, varname)* First writes out to the FORTRAN code the length of every “edge” (one for each dimension) of the variable *varname*; this is done using the local arrays *nc\_corner* and *nc\_edge*. Then, either the netCDF routine *ncvargetc* (for character strings) or *ncvarget* is called to actually retrieve the data from the file.

## 54 File names for DEGAS 2

[/u/dstotler/degas2/src/readfilenames.hweb]

```
$Id: readfilenames.hweb,v 1.10 2000/04/01 17:34:20 dstotler Exp $
```

Symbolic names for DEGAS 2 input and output files.

[/u/dstotler/degas2/src/readfilenames.hweb] indices for the array of filenames

```
"classes.f" 54.1 ≡
 @m elementsfile 1
 @m backgroundfile 2
 @m testfile 3 // No longer used
 @m geometryfile 4
 @m problemfile 5
 @m reactionfile 6
 @m speciesfile 7
 @m aladinfile 8 // Not being used
 @m aladoutfile 9 // Not being used
 @m elements_infile 10
 @m problem_infile 11
 @m reaction_infile 12
 @m species_infile 13
 @m materials_infile 14
 @m materialsfile 15
 @m pmi_infile 16
 @m pmifile 17
 @m cramdpblemfile 18 // Not used in public version of code
 @m tallyfile 19
 @m outputfile 20
 @m oldsourcefile 21
 @m tally_infile 22

 @m filenames_max 22 // Largest of the above
```

[/u/dstotler/degas2/src/readfilenames.hweb] common block definitions for filenames

```
"classes.f" 54.2 ≡
 package_init(rf)
@#if 0
 define_var_pk(rf, filenames_max, INT)
@#endif
 define_dimen_pk(rf, filenames_size, FILELEN)
 define_dimen_pk(rf, filenames_ind, filenames_max)
 define_varp_pk(rf, filenames_array, CHAR, filenames_size, filenames_ind)
 package_end(rf)
```

## 55 Description of file name usage in DEGAS 2

[/u/dstotler/degas2/src/readfilenames.hweb]

The names of most input and output files used by DEGAS 2 are specified in a single file, currently called *degas2.in*. The filenames given therein are associated with the symbolic names which appear in the rest of the code. These symbolic names should be self-describing; more details about each of these files is given in the User's Manual. The corresponding user-specified filenames (which can be an absolute or relative UNIX path name) are stored in the *filenames\_array* array. The indices in that array corresponding to each symbolic name are given by the above macros. The entries in the *degas2.in* file are of the form:

```
symbolic_name actual_path_name
```

The entries can be in any order. Comments begin with a “#” and be on a separate line or at the end of a line. Spaces and blank lines are ignored. Unneeded filenames (see comments at the top of this file) do not have to be included. If you forget to specify a needed filename, DEGAS 2 will should give an error like:

```
Assertion failed (pmimatread.web): tempfile!=char_undef
```

An example *degas2.in* file is distributed in the *src* directory of DEGAS 2. Other examples are included in *examples* directories. The *degas2.in* file *must* be in your current “run” directory (e.g., **degas2/SUN**). Files referred to by *degas2.in* in the **data** directory should not be modified by the user. If changes need to be made, the user should make copies of the files in the “run” directory or elsewhere. The user is advised to choose filenames which will aid in the tracking of run variants.

To add new files to the above list:

1. Create a suggestive symbolic file name,
2. Assign the next integer to it in the above macro list,
3. Increment *filenames\_max*,
4. Make corresponding changes to *readfilenames.web* (see the introduction there).

## 56 Specification for a 2-D Geometry

[/u/dstotler/degas2/src/geometry2d.hweb]

```
$Id: geometry2d.hweb,v 1.4 2002/01/28 16:55:53 dstotler Exp $
```

[/u/dstotler/degas2/src/geometry2d.hweb] Parameters

```
"classes.f" 56.1 ≡
 @m g2_num_points 350
 @m g2_xz 2
 @m g2_x 1
 @m g2_z 2
```

[/u/dstotler/degas2/src/geometry2d.hweb] Variable Definitions

```
"classes.f" 56.2 ≡
 package_init(g2)
 define_dimen_pk(g2, g2_points_ind0, 0, g2_num_points - 1)
 define_dimen_pk(g2, g2_points_ind, g2_num_points)
 define_dimen_pk(g2, g2_xz_ind, g2_xz)
 define_var_pk(g2, g2_num_polygons, INT)
 define_dimen_pk(g2, g2_poly_ind, g2_num_polygons)
 define_var_pk(g2, g2_polygon_xz, FLOAT, g2_xz_ind, g2_points_ind0, g2_poly_ind)
 define_var_pk(g2, g2_polygon_segment, INT, g2_points_ind0, g2_poly_ind)
 define_var_pk(g2, g2_polygon_points, INT, g2_poly_ind)
 define_var_pk(g2, g2_polygon_zone, INT, g2_poly_ind)
 define_var_pk(g2, g2_polygon_stratum, INT, g2_poly_ind)
 // This is the minimum; fill out the rest later
 package_end(g2)
```

## 57 Two-D geometry specification class

[/u/dstotler/degas2/src/geometry2d.hweb]

[/u/dstotler/degas2/src/geometry2d.hweb] Define a 2-D geometry. Prefix is g2.

*g2\_num\_points* Maximum number of points in a polygon or wall. Have chosen to fix this rather than dealing with two variable dimensions.

*g2\_xz* Just an index to represent  $x$  and  $z$ .

*g2\_polygon\_xz<sub>poly\_ind,points\_ind,xz\_ind</sub>*  $x$  ( $xz\_ind = 1$ ),  $z$  ( $xz\_ind = 2$ ) coordinates of polygon *poly\_ind*. The first point ( $points\_ind = 0$ ) and last ( $points\_ind = g2\_polygon\_points_{poly\_ind}$ ) must coincide. All polygons used to specify this geometry are contained in this array.

*g2\_polygon\_segment<sub>poly\_ind,points\_ind</sub>* A label for each point and segment of a polygon. For polygons specified directly on input, just corresponds to the number of the point. For triangles generated from polygons, the original segment number is retained during the decomposition process and assigned to this array so that the edges of the new triangles can be related to the original polygon segments.

*g2\_polygon\_points<sub>poly\_ind</sub>* Actual number of distinct (i.e., not counting the duplicate first / last) points in polygon *poly\_ind*.

*g2\_polygon\_zone<sub>poly\_ind</sub>* Zone number to be associated with polygon *poly\_ind*.

*g2\_polygon\_stratum<sub>poly\_ind</sub>* Stratum number specified on input with either this polygon or its parent polygon (if decomposed into triangles).

## 58 INDEX

*\_base*: 30.6, 37.5, 41.6, 45.1, 45.3, 47.1, 47.3.  
*\_base\_1*: 37.5, 41.6.  
*\_delta*: 45.1, 47.1.  
*\_dep\_var\_dim*: 30.6.  
*\_eval\_name*: 45.1, 47.1.  
*\_grp*: 33.1.  
*\_id*: 52.2.  
*\_inc*: 37.5, 41.6.  
*\_m*: 50.2, 50.5.  
*\_min*: 45.1, 47.1.  
*\_mod*: 50.7.  
*\_paste*: 50.1.  
*\_rank*: 45.1, 47.1.  
*\_register*: 50.6.  
*\_size*: 30.6, 37.5, 41.6, 45.3, 47.3.  
*\_spacing*: 45.1, 47.1.  
*\_tab*: 37.5, 41.6, 45.1, 45.3, 47.1, 47.3.  
*\_tab\_index*: 45.1, 47.1.  
*\_var*: 45.1, 47.1.  
*\_1*: 30.6, 37.5, 41.6, 45.3, 47.3, 50.3.  
*\_2*: 37.5, 41.6.  
*aladinfile*: 54.1.  
*aladoutfile*: 54.1.  
*alb*: 26.3.  
*albedo*: 27.1.  
*array*: 1, 51.  
*array\_arglist*: 50.2, 50.4.  
*array\_arglistc*: 50.2, 50.4.  
*array\_rank*: 50.2.  
*array\_shape*: 50.2.  
*array\_shape\_star*: 50.2.  
*array\_shape\_stara*: 50.2.  
*array\_size*: 45.3, 47.3, 50.2, 50.4, 50.5, 51.  
*array\_sizea*: 50.5.  
*array\_sizeb*: 50.5.  
*array\_sizec*: 50.5.  
*array\_sizev*: 50.2, 50.4, 51.  
*array\_unit*: 50.2, 50.5, 51.  
*arrayname*: 51.  
*assert*: 20.2, 20.3, 52.2.  
*ASSIGN*: 50.4.  
*atomic\_mass\_unit*: 39.1.  
*atomic\_mass\_unit\_g*: 39.1.  
*author\_*: 8.1.  
*back*: 43.3, 44.3.  
*back\_ind*: 33.1.  
*background*: 21.1.  
*background\_coords*: 20.4, 21.1.

*background\_ind*: 20.4.  
*background\_n*: 20.1, 20.4.  
*background\_temp*: 20.1, 20.4.  
*background\_v*: 20.1, 20.4.  
*backgroundfile*: 54.1.  
*bk*: 20.4, 21.1.  
*bk\_args*: 20.1.  
*bk\_check*: 20.1.  
*bk\_copy*: 20.1.  
**bk\_decl**: 20.1.  
*bk\_dummy*: 20.1.  
*bk\_mx*: 20.1.  
*bk\_my*: 20.1.  
*bk\_n*: 20.1, 21.1.  
*bk\_num*: 20.1, 20.4.  
*bk\_plasma\_ind*: 20.4.  
*bk\_temp*: 20.1, 21.1.  
*bk\_v*: 20.1, 21.1.  
*bool\_list*: 39.4.  
*bool\_sect*: 39.4.  
*boxgen*: 23.3.  
*cat*: 50.6, 50.7.  
*cell*: 10.  
*cell\_exit*: 8.2.  
*cell\_next*: 10.  
*CHAR*: 4.3, 6.3, 14.3, 16.3, 18.2, 22.4, 24.3, 26.2,  
           28.3, 30.4, 32.3, 37.3, 39.2, 41.4, 43.4, 45.2, 47.2,  
           50.1, 50.2, 51, 54.2.  
*char\_alloc*: 50.4.  
*char\_free*: 50.4.  
*char\_mem*: 50.4.  
*char\_realloc*: 50.4.  
*char\_spec*: 50.2.  
*check\_location*: 10.  
*CL*: 51.  
*CL\_args*: 1.  
*CL\_check*: 1.  
*CL\_common*: 1, 51.  
*CL\_common\_a*: 51.  
*CL\_common\_b*: 51.  
*CL\_copy*: 1.  
*CL\_decl*: 1.  
*CL\_dummy*: 1.  
*CL\_ncdecl*: 51.  
*CL\_ncdef*: 51.  
*CL\_ncread*: 51.  
*CL\_ncreada*: 51.  
*CL\_ncwrite*: 51.  
*CL\_pvmget*: 51.  
*CL\_pvmput*: 51.  
*COMMA*: 50.1.  
**common**: 39.3.

- const*: 10.2, 22.1, 22.3, 39.1.
- coord*: 20.2, 20.3.
- COUNT*\_: 50.6.
- cpl\_ind*: 33.1.
- cramdproblemfile*: 54.1.
- cv\_ind*: 31.1.
- d\_sector*: 27.2.
- data\_filename*: 39.2.
- de*: 28.3, 29.1.
- de\_algorithm\_circular*: 28.2.
- de\_algorithm\_uniform*: 28.2, 29.1.
- de\_algorithm\_unknown*: 28.2.
- de\_args*: 28.1.
- de\_decl**: 28.1.
- de\_dummy*: 28.1.
- de\_grp\_ind*: 28.3.
- de\_grps*: 28.1, 28.3, 29.1.
- de\_lookup*: 28.1, 29.2.
- de\_max\_bins*: 27.1, 28.3, 29.1.
- de\_name\_len*: 28.2, 28.3.
- de\_name\_string*: 28.3.
- de\_spacing\_linear*: 28.2.
- de\_spacing\_log*: 28.2.
- de\_spacing\_unknown*: 28.2.
- de\_start\_end\_ind*: 28.3.
- de\_sy\_len*: 28.2, 28.3.
- de\_symbol\_string*: 28.3.
- de\_tot\_view\_ind*: 28.3.
- de\_uniform\_circular*: 29.1.
- de\_var\_unknown*: 28.2.
- de\_var\_wavelength*: 28.2, 29.1.
- de\_view\_algorithm*: 28.3, 29.1.
- de\_view\_base*: 28.2, 28.3, 29.1.
- de\_view\_end*: 28.2, 28.3, 29.1.
- de\_view\_halfwidth*: 28.3, 29.1.
- de\_view\_ind*: 28.3.
- de\_view\_pointer*: 28.2, 29.2.
- de\_view\_points*: 28.3, 29.1.
- de\_view\_size*: 28.3, 29.1.
- de\_view\_start*: 28.2, 28.3, 29.1.
- de\_view\_tab*: 28.2, 28.3, 29.1.
- de\_zone\_frags*: 28.3, 29.1.
- de\_zone\_ind*: 28.3.
- decl\_alpcom**: 39.3.
- declare\_var**: 50.2.
- declare\_varp**: 50.2.
- define\_dimen**: 50.2.
- define\_dimen\_pk*: 4.3, 6.3, 14.3, 16.3, 18.2, 20.4, 22.4, 24.3, 26.2, 28.3, 30.4, 32.3, 34.2, 37.3, 39.2, 41.4, 43.4, 43.5, 45.2, 47.2, 50.6, 51, 54.2, 56.2.
- define\_dummy\_pk*: 50.6.
- define\_sector\_exit*: 26.3, 27.2.
- define\_sector\_plasma*: 26.3, 27.2.
- define\_sector\_target*: 26.3, 27.2.
- define\_sector\_vacuum*: 26.3, 27.2.
- define\_sector\_wall*: 26.3, 27.2.
- define\_tally*: 31.1.
- define\_var**: 50.2.
- define\_var\_pk*: 1, 4.3, 6.3, 14.3, 16.3, 18.2, 20.4, 22.4, 24.3, 26.2, 28.3, 30.4, 32.3, 34.2, 37.3, 39.2, 41.4, 43.4, 43.5, 45.2, 47.2, 50.6, 51, 54.2, 56.2.
- define\_varlocal\_pk*: 4.3, 6.3, 14.3, 16.3, 18.2, 22.4, 24.3, 30.4, 32.3, 37.3, 41.4, 43.4, 47.2, 50.6.
- define\_varp*: 34.1, 50.2, 50.6.
- define\_varp\_pk*: 1, 4.3, 6.3, 14.3, 16.3, 18.2, 20.4, 22.4, 24.3, 26.2, 28.3, 30.4, 32.3, 34.2, 37.3, 41.4, 43.4, 43.5, 45.2, 47.2, 50.6, 51, 54.2, 56.2.
- degas2*: 55.
- dep\_var\_ind*: 37.3, 38.1.
- dest*: 50.7.
- detector*: 27.1.
- detector\_delta*: 28.3, 29.1.
- detector\_min*: 28.3, 29.1.
- detector\_name*: 28.1, 28.3, 29.1.
- detector\_num\_views*: 28.3, 29.1, 29.2.
- detector\_spacing*: 28.3, 29.1.
- detector\_tab\_index*: 28.3, 29.1.
- detector\_total\_views*: 28.3, 29.1, 29.2.
- detector\_var*: 28.3, 29.1.
- detector\_view\_base*: 29.1.
- detector\_view\_setup*: 29.1.
- diag\_grp\_ind*: 26.2.
- diag\_lookup*: 26.1, 27.2.
- diagnostic*: 26.2, 27.1.
- diagnostic\_delta*: 26.2, 27.1.
- diagnostic\_grp\_base*: 26.1, 26.2, 27.1.
- diagnostic\_grp\_name*: 26.1, 26.2, 27.1.
- diagnostic\_ind*: 27.1.
- diagnostic\_min*: 26.2, 27.1.
- diagnostic\_num\_sectors*: 26.1, 26.2, 27.1.
- diagnostic\_sector*: 26.1, 27.1, 27.2.
- diagnostic\_sector\_tab*: 26.1, 26.2, 27.1.
- diagnostic\_spacing*: 26.2, 27.1.
- diagnostic\_tab\_index*: 26.2, 27.1.
- diagnostic\_var*: 26.2, 27.1.
- dim\_junk\_array*: 50.2.
- dimen\_def*: 52.2, 53.
- dimen\_inq*: 52.2, 53.
- dimid*: 52.2, 53.
- dimid1*: 52.2.
- dimname*: 51, 53.
- dimname1*: 51.
- dimname2*: 51.
- dims*: 50.3.

- do\_common*: [50.7](#).
- do\_module*: [50.7](#).
- do\_mpibcast*: [50.7](#).
- do\_mpireceive*: [50.7](#).
- do\_mpisend*: [50.7](#).
- do\_ncdecl*: [50.7](#).
- do\_ncdef*: [50.7](#).
- do\_ncread*: [50.7](#).
- do\_ncwrite*: [50.7](#).
- do\_netcdfa*: [50.7](#).
- do\_pvmget*: [50.7](#).
- do\_pvmput*: [50.7](#).
- e\_sector*: [27.2](#).
- el*: [4.3](#), [5.1](#).
- el\_args*: [4.1](#).
- el\_check*: [4.1](#).
- el\_copy*: [4.1](#).
- el\_decl**: [4.1](#).
- el\_dummy*: [4.1](#).
- el\_lookup*: [4.1](#), [5.2](#).
- el\_m*: [4.1](#), [5.1](#).
- el\_name*: [1](#), [4.1](#), [5.1](#).
- el\_name.len*: [4.2](#), [4.3](#), [5.1](#).
- el\_num*: [4.1](#), [4.3](#).
- el\_sy*: [4.1](#), [5.1](#).
- el\_sy.len*: [4.2](#), [4.3](#), [5.1](#).
- el\_z*: [4.1](#), [5.1](#).
- electron\_charge*: [39.1](#).
- element*: [5.1](#).
- element\_ind*: [4.3](#).
- element\_m*: [4.1](#), [4.3](#).
- element\_name*: [1](#), [4.1](#), [4.3](#).
- element\_name\_string*: [4.3](#).
- element\_sy*: [4.1](#), [4.3](#).
- element\_symbol\_string*: [4.3](#).
- element\_version*: [4.3](#).
- element\_z*: [4.1](#), [4.3](#).
- elements\_infile*: [54.1](#).
- elementsfile*: [54.1](#).
- emission\_rate\_h\_alpha*: [31.1](#).
- end*: [50.7](#).
- end\_common*: [50.7](#).
- end\_dep\_var*: [39.4](#).
- end\_indep\_var*: [39.4](#).
- end\_module*: [50.7](#).
- end\_mpibcast*: [50.7](#).
- end\_mpireceive*: [50.7](#).
- end\_mpisend*: [50.7](#).
- end\_ncdecl*: [50.7](#).
- end\_ncdef*: [50.7](#).
- end\_ncread*: [50.7](#).
- end\_ncwrite*: [50.7](#).
- end\_pvmget*: [50.7](#).
- end\_pvmput*: [50.7](#).
- energy*: [31.1](#).
- energy\_change*: [31.1](#).
- epsilon*: [39.1](#).
- epsilon\_angle*: [10.2](#).
- equivalents*: [43.5](#), [44.4](#).
- est\_ind*: [31.1](#).
- ev\_to\_ergs*: [39.1](#).
- eval\_name*: [37.3](#).
- examples*: [55](#).
- exit*: [26.2](#), [27.1](#).
- exit\_albedo*: [26.2](#), [26.3](#).
- exit\_ind*: [26.2](#), [27.1](#).
- exit\_sector*: [26.2](#), [26.3](#), [27.1](#).
- face*: [26.1](#), [27.2](#).
- FALSE*: [22.3](#), [23.3](#), [33.1](#), [35.1](#).
- fff*: [35.1](#), [35.2](#).
- fileid*: [50.7](#), [52.2](#), [53](#).
- FILELEN*: [14.3](#), [18.2](#), [54.2](#).
- filenames\_array*: [54.2](#), [55](#).
- filenames\_ind*: [54.2](#).
- filenames\_max*: [54.1](#), [54.2](#), [55](#).
- filenames\_size*: [54.2](#).
- fin*: [34.2](#), [35.1](#).
- find\_threshold*: [39.1](#).
- fit*: [38.1](#).
- fit\_coef*: [39.4](#).
- fl*: [13.1](#).
- fl\_args*: [12.1](#).
- fl\_check*: [12.1](#).
- fl\_common**: [12.1](#).
- fl\_copy*: [12.1](#).
- fl\_current*: [1](#), [12.1](#), [12.3](#), [13.1](#).
- fl\_decl**: [12.1](#).
- fl\_dummy*: [12.1](#).
- fl\_init*: [12.3](#), [13.2](#).
- fl\_label*: [12.3](#), [13.2](#).
- fl\_number*: [12.1](#), [12.3](#), [13.1](#), [13.2](#).
- fl\_origin*: [12.1](#), [12.3](#), [13.1](#), [13.2](#).
- fl\_pointer*: [12.1](#), [12.3](#), [13.1](#).
- fl\_rand*: [12.1](#), [13.1](#).
- fl\_source*: [12.1](#), [12.3](#), [13.1](#), [13.2](#).
- fl\_stack*: [12.1](#), [13.1](#).
- fl\_stack\_max*: [12.1](#), [12.2](#), [13.1](#).
- fl\_temp*: [12.1](#).
- flight*: [13.1](#), [13.2](#).
- flighttest*: [23.1](#).
- FLOAT*: [4.3](#), [6.3](#), [20.4](#), [22.4](#), [24.3](#), [26.2](#), [28.3](#), [30.4](#),  
[32.3](#), [34.2](#), [37.3](#), [39.2](#), [41.4](#), [43.4](#), [45.2](#), [47.2](#), [50.1](#),  
[50.2](#), [51](#), [56.2](#).
- flt*: [34.2](#), [35.1](#).

- foo*: 51.
- FORTTRAN77*: 50.2, 50.4, 50.5, 50.7.
- FORTTRAN90*: 50.2, 50.5, 50.7.
- frag*: 34.2, 35.1.
- generics*: 43.5, 44.4.
- geom\_epsilon*: [10.2](#).
- geom\_infinity*: [10.2](#).
- geom\_large*: [10.2](#).
- geometry*: 27.1.
- geometry\_symmetry\_cyl.hw*: 20.2, 20.3.
- geometry\_symmetry\_cyl.section*: 20.2, 20.3.
- geometry\_symmetry\_cylindrical*: 20.2, 20.3.
- geometry\_symmetry\_none*: 20.2, 20.3.
- geometry\_symmetry\_oned*: 20.
- geometryfile*: [54.1](#).
- geomint*: 27.1.
- groud\_ind*: 27.1.
- group*: 27.2.
- group\_ind*: 27.1, 29.1, 29.2, 33.1.
- grp*: 26.1, 27.2, 28.2.
- grp\_ind*: 33.1.
- g2*: 56.2.
- g2\_num\_points*: [56.1](#), 56.2, 57.1.
- g2\_num\_polygons*: 56.2.
- g2\_points\_ind*: 56.2.
- g2\_points\_ind0*: 56.2.
- g2\_poly\_ind*: 56.2.
- g2\_polygon\_points*: 56.2, 57.1.
- g2\_polygon\_segment*: 56.2, 57.1.
- g2\_polygon\_stratum*: 56.2, 57.1.
- g2\_polygon\_xz*: 56.2, 57.1.
- g2\_polygon\_zone*: 56.2, 57.1.
- g2\_x*: [56.1](#).
- g2\_xz*: [56.1](#), 56.2, 57.1.
- g2\_xz\_ind*: 56.2.
- g2\_z*: [56.1](#).
- half*: 22.1.
- hweb*: 1, 27.1, 30.6, 31.1, 38.2, 42.1, 42.2, 44.4, 45.3, 46.1, 48.1.
- IBM*: 50.2.
- igroup*: 23.1.
- implicit\_none\_f90mod*: 50.7.
- in*: 55.
- inc*: [30.5](#), 30.6, 37.5, 41.6, [45.3](#).
- include*: 39.3.
- ind*: 32.2, 33.2.
- ind\_*: 30.6, 37.5, 41.6, 45.3, 47.3, 50.2, 50.3, 50.5.
- ind\_temp\_m*: 50.2.
- ind\_temp\_1*: 50.2.
- ind\_temp\_2*: 50.2.
- ind\_temp\_3*: 50.2.
- ind\_x.m*: 50.2.
- ind\_x.n*: 50.2.
- independent\_paramaters*: 41.2.
- index\_parameters*: 31.2.
- indx\_m*: 50.5.
- init\_base*: 45.3, 47.3.
- init\_var0\_list*: 31.1, 43.3.
- inst*: 1.
- inst1*: 1.
- inst2*: 1.
- int*: 50.5.
- INT*: 4.3, 6.3, 14.3, 16.3, 18.2, 20.4, 22.4, 24.3, 26.2, 28.3, 30.4, 32.3, 34.2, 37.3, 39.2, 41.4, 43.4, 43.5, 45.2, 47.2, [50.1](#), 51, 54.2, 56.2.
- int\_alloc*: [50.4](#).
- int\_free*: [50.4](#).
- int\_lookup*: 26.1, 43.1, [49](#).
- int\_mem*: 50.4.
- int\_realloc*: [50.4](#).
- integ\_transform\_exp*: [39.1](#).
- integ\_transform\_ratio*: [39.1](#).
- integer**: 4.1, 6.1, 8.1, 10, 12.1, 14.1, 16.1, 18, 20.1, 22.1, 24.1, 26.1, 28.1, 30.1, 30.5, 36.1, 37.4, 39.4, 41.5, 43.1, 45.3, 49, 50.2, 50.4, 52.2.
- integration\_acc*: [39.1](#).
- integration\_infinity*: [39.1](#).
- ip*: 30.3, 31.2.
- iparam*: 23.1.
- iseg*: 23.2.
- isource*: 12.3, 13.2.
- iterate\_pk*: [50.7](#).
- itype*: 23.2.
- ix*: 33.1.
- iy*: 33.1.
- junk*: 50.2.
- junk\_array*: 50.2.
- junk\_arrray*: 50.2.
- label\_num\_ind*: 39.2.
- last\_sizea*: [50.5](#).
- last\_sizeb*: [50.5](#).
- last\_sizec*: [50.5](#).
- last\_ubound*: [50.5](#).
- last\_ubounda*: [50.5](#).
- last\_uboundb*: [50.5](#).
- last\_uboundc*: [50.5](#).
- lc*: 11.1.
- lc\_args*: 8.1, [10](#).
- lc\_cell*: [10](#), 10.1, 11.1.
- lc\_cell\_next*: [10](#), 10.1, 11.1.
- lc\_cell\_next1*: [10](#).
- lc\_cell1*: [10](#).

- lc\_check*: 8.1, [10](#).
- lc\_copy*: 8.1, [10](#).
- lc\_decl**: 8.1, [10](#).
- lc\_decls**: [10](#).
- lc\_dummy*: 8.1, [10](#).
- lc\_face*: 8.3, [10](#), 10.1, 11.1.
- lc\_face1*: [10](#).
- lc\_sector*: [10](#), 10.1, 11.1.
- lc\_sector\_next*: [10](#), 10.1, 11.1.
- lc\_sector\_next1*: [10](#).
- lc\_sector1*: [10](#).
- lc\_set*: [10.1](#), 11.2.
- lc\_set\_a*: [10.1](#).
- lc\_thru\_face*: 8.3, [10.1](#), 11.2.
- lc\_x*: 8.3, [10](#), 10.1, 11.1.
- lc\_x1*: [10](#).
- lc\_zone*: [10](#), 10.1, 11.1.
- lc\_zone\_next*: [10](#), 10.1, 11.1.
- lc\_zone\_next1*: [10](#).
- lc\_zone1*: [10](#).
- lcfptr*: [39.3](#).
- lcmptr*: [39.3](#).
- len*: 50.2.
- len\_junk*: 50.2.
- len\_junk\_array*: 50.2.
- leseqn*: [39.3](#).
- leval*: [39.4](#).
- locate\_point*: [10](#), 10.1.
- location*: 11.1, 11.2.
- lower\_junk\_array*: 50.2.
- ma*: 16.3, 17.1, 19.1, 27.1, 27.2, 44.1.
- ma\_args*: [16.1](#).
- ma\_check*: [16.1](#).
- ma\_copy*: [16.1](#).
- ma\_decl**: [16.1](#).
- ma\_dummy*: [16.1](#).
- ma\_lookup*: [16.1](#), 17.3.
- ma\_name*: [16.1](#), 17.1.
- ma\_name.len*: [16.2](#), 16.3, 17.1.
- ma\_num*: 16.1, 16.3, 17.2, 43.4.
- ma\_sy*: [16.1](#), 17.1.
- ma\_sy.len*: [16.2](#), 16.3, 17.1.
- macro\_index*: 42.1, 46.1, 48.1.
- mass*: 31.1.
- mass\_change*: 31.1.
- mass\_unit\_string*: 39.2.
- mat*: 26.3.
- material*: 17.1, 27.2.
- materials\_ind*: 16.3.
- materials\_infile*: [54.1](#).
- materials\_name*: 16.1, 16.3.
- materials\_name\_string*: 16.3.
- materials\_sy*: 16.1, 16.3.
- materials\_symbol\_string*: 16.3.
- materials\_version*: 16.3.
- materialsfile*: [54.1](#).
- max*: 50.2, 51.
- max\_*: 50.2.
- max\_bins*: 27.1, 29.1, [30.3](#).
- MAX\_DIM*: [50.1](#), 50.2, 52.2.
- max\_integ\_steps*: [39.1](#).
- max\_interp\_pts*: [39.1](#).
- max\_name*: 50.2.
- MAX\_NC\_NAME*: [52.1](#), 52.2.
- mem\_alloc*: 50.4.
- mem\_alloc\_*: 50.4.
- mem\_assign*: [50.4](#).
- mem\_free*: 50.4.
- mem\_free\_*: 50.4.
- mem\_inc*: [50.5](#), 51.
- mem\_realloc*: 50.4.
- mem\_realloc\_*: 50.4.
- mem\_size*: [50.5](#).
- min*: 50.2, 51.
- min\_*: 50.2.
- min\_bound\_ratio*: [39.1](#).
- min\_name*: 50.2.
- mod*: 50.5.
- module*: 50.7.
- mom\_ind*: 33.1.
- moment\_number*: [39.4](#).
- momentum\_change\_vector*: 31.1.
- momentum\_vector*: 31.1.
- MPI*: 50.7.
- msg\_length*: [39.1](#).
- mult*: 48.1.
- name**: 26.1, 27.2, 28.1, 29.2, 50.2, 50.6, 52.2, 53.
- nbl*: [39.3](#).
- nblmx*: [39.3](#).
- nc\_attr*: [52.2](#).
- NC\_BYTE*: [52.1](#).
- NC\_CHAR*: [52.1](#).
- NC\_CLOBBER*: [52.1](#).
- nc\_corner*: [52.2](#), 53.
- nc\_decls**: [52.2](#).
- nc\_dims*: [52.2](#), 53.
- NC\_DOUBLE*: [52.1](#).
- nc\_dummy*: [52.2](#).
- nc\_edge*: [52.2](#), 53.
- NC\_FATAL*: [52.1](#).
- NC\_FLOAT*: [52.1](#).
- NC\_GLOBAL*: [52.1](#).
- NC\_INT*: [52.1](#).
- NC\_LONG*: [52.1](#).

- NC\_NOCLOBBER*: [52.1](#).  
*NC\_NOWRITE*: [52.1](#).  
*nc\_rank*: [52.2](#).  
*NC\_REAL*: [52.1](#).  
*NC\_SHORT*: [52.1](#).  
*nc\_size*: [52.2](#).  
*nc\_stat*: [52.2](#).  
*nc\_type*: [52.2](#).  
*NC\_VERBOSE*: [52.1](#).  
*NC\_WRITE*: [52.1](#).  
*ncagt*: [52.1](#).  
*ncagtc*: [52.1](#).  
*ncapt*: [52.1](#).  
*ncaptc*: [52.1](#).  
*ncattget*: [52.1](#).  
*ncattgetc*: [52.1](#).  
*ncattput*: [52.1](#).  
*ncattputc*: [52.1](#).  
*ncclos*: [52.1](#).  
*ncclose*: [52.1](#).  
*nccre*: [52.1](#).  
*nccreate*: [52.1](#), [52.2](#).  
*ncddef*: [52.1](#).  
*ncdid*: [52.1](#).  
*ncdimdef*: [52.1](#), [52.2](#), [53](#).  
*ncdimid*: [52.1](#), [52.2](#).  
*ncdiminq*: [52.1](#), [52.2](#), [53](#).  
*ncding*: [52.1](#).  
*ncendef*: [52.1](#).  
*ncendf*: [52.1](#).  
*ncfln*: [39.3](#).  
*ncmln*: [39.3](#).  
*nconversions*: [30.4](#), [31.1](#).  
*ncopen*: [52.1](#), [52.2](#).  
*ncopn*: [52.1](#).  
*ncredef*: [52.1](#).  
*ncredf*: [52.1](#).  
*ncvardef*: [52.1](#), [52.2](#), [53](#).  
*ncvarget*: [52.1](#), [53](#).  
*ncvargetc*: [52.1](#), [53](#).  
*ncvarid*: [52.1](#), [52.2](#).  
*ncvarinq*: [52.1](#), [52.2](#).  
*ncvarput*: [52.1](#), [53](#).  
*ncvarputc*: [52.1](#), [53](#).  
*ncvdef*: [52.1](#).  
*ncvgt*: [52.1](#).  
*ncvgtc*: [52.1](#).  
*ncvid*: [52.1](#).  
*ncvinq*: [52.1](#).  
*ncvpt*: [52.1](#).  
*ncvptc*: [52.1](#).  
*neln*: [39.3](#).  
*nelnmx*: [39.3](#).  
*NETCDFV2*: [52.1](#).  
*new\_bool\_sect*: [39.4](#).  
*newmax*: [51](#).  
*next\_bool*: [39.4](#).  
*next\_dep\_var*: [39.4](#).  
*next\_hlab*: [39.4](#).  
*next\_line*: [39.4](#).  
*next\_token*: [49](#).  
*nhl*: [39.3](#).  
*nhlmx*: [39.3](#).  
*nsectors*: [26.1](#), [26.2](#), [27.1](#).  
*num*: [12.3](#), [13.2](#).  
*num\_equiv*: [43.5](#), [44.4](#).  
*num\_evaluations*: [39.4](#).  
*num\_extrap\_pts*: [39.1](#).  
*num\_fit\_coef*: [39.4](#).  
*num\_generics*: [43.5](#), [44.4](#).  
*number\_*: [12.1](#).  
*o\_mean*: [32.2](#), [32.3](#), [33.1](#), [34.2](#).  
*o\_var*: [32.2](#), [32.3](#), [33.1](#), [34.2](#).  
*old\_size*: [37.4](#), [37.5](#), [41.5](#), [41.6](#).  
*oldmax*: [51](#).  
*oldsourcefile*: [54.1](#).  
*on*: [50.4](#), [50.5](#).  
*one*: [8.3](#), [22.1](#), [26.3](#).  
*origin\_*: [12.1](#).  
*ou*: [32.3](#), [33.1](#).  
*ou\_back\_ind*: [32.3](#).  
*ou\_back\_max*: [32.2](#).  
*ou\_coupling\_energy*: [32.2](#).  
*ou\_coupling\_mass*: [32.2](#).  
*ou\_coupling\_max*: [32.2](#), [32.3](#).  
*ou\_coupling\_momentum\_1*: [32.2](#).  
*ou\_coupling\_momentum\_2*: [32.2](#).  
*ou\_coupling\_momentum\_3*: [32.2](#).  
*out\_array\_index*: [32.2](#), [33.2](#).  
*out\_index\_5*: [32.2](#).  
*out\_post*: [33.1](#).  
*out\_post\_all*: [32.3](#), [33.1](#).  
*out\_post\_grp*: [32.3](#).  
*output\_all*: [32.3](#), [33.1](#).  
*output\_checkpoint*: [32.3](#), [33.1](#).  
*output\_coupling\_ind*: [32.3](#).  
*output\_grp*: [1](#), [32.3](#), [33.1](#).  
*output\_grps\_ind*: [32.3](#).  
*output\_ind\_1*: [32.3](#).  
*output\_ind\_2*: [32.3](#).  
*output\_index\_1\_max*: [32.3](#), [33.1](#).  
*output\_index\_1\_min*: [32.3](#), [33.1](#).  
*output\_index\_2\_max*: [32.3](#), [33.1](#).  
*output\_index\_2\_min*: [32.3](#), [33.1](#).

- output\_moments\_ind*: 32.3.
- output\_num\_flights*: 32.3, 33.1.
- output\_old\_file*: 32.3.
- output\_random\_seed*: 32.3, 33.1.
- output\_seed\_ind*: 32.3.
- output\_tab\_ind*: 32.3.
- output\_version*: 32.3.
- output\_weight\_grp*: 32.3, 33.1.
- output\_2D\_coupling*: 32.3, 33.1.
- outputfile*: [54.1](#).
- p\_sector*: 27.2.
- package\_end*: 4.3, 6.3, 14.3, 16.3, 18.2, 20.4, 22.4, 24.3, 26.2, 28.3, 30.4, 32.3, 34.2, 37.3, 39.2, 41.4, 43.4, 43.5, 45.2, 47.2, [50.6](#), 54.2, 56.2.
- package\_init*: 4.3, 6.3, 14.3, 16.3, 18.2, 20.4, 22.4, 24.3, 26.2, 28.3, 30.4, 32.3, 34.2, 37.3, 39.2, 41.4, 43.4, 43.5, 45.2, 47.2, [50.6](#), 51, 54.2, 56.2.
- parse\_string*: [49](#).
- part\_ind*: 31.1.
- particle*: 9.1, 9.2.
- particle\_track*: [8.1](#), 8.3.
- paste*: [50.1](#), 50.2, 50.4.
- pd*: 42.1, 47.2, 48.1.
- pd\_data*: 48.2.
- pd\_data\_args*: [47.1](#), 48.2.
- pd\_data\_base*: 48.2.
- pd\_data\_size*: 48.2.
- pd\_data\_tab*: 48.2.
- pd\_dep\_var\_ind*: 47.2.
- pd\_dep\_var\_max*: [47.1](#), 47.2.
- pd\_eval\_string*: 47.2.
- pd\_handling*: [47.1](#), 48.1, 48.2.
- pd\_handling\_ind*: 47.2.
- pd\_inc*: 47.2, 47.3.
- pd\_ragged\_alloc*: [47.3](#), 48.1, 48.2.
- pd\_ragged\_realloc*: [47.3](#), 48.1, 48.2.
- pd\_rank\_ind*: 47.2.
- pd\_rank\_ind0*: 47.2.
- pd\_set\_baseinc*: [47.3](#).
- pd\_tab\_index*: 48.2.
- pd\_tag\_string*: 47.2.
- pd\_yield*: [47.1](#), 48.1, 48.2.
- pd\_yield\_ind*: 47.2.
- pf*: 19.1, 41.4, 42.1, 48.1.
- pf\_data*: 41.6, 42.2.
- pf\_data\_base*: 41.3, 41.4, 42.1.
- pf\_data\_inc*: 41.4, 42.1.
- pf\_data\_ind*: 41.4.
- pf\_data\_size*: 41.4, 42.1.
- pf\_data\_tab*: 41.3, 41.4, 42.1, 42.2.
- pf\_data\_table*: [41.3](#), 42.1, 42.2.
- pf\_decls**: [41.5](#).
- pf\_dep\_var\_ind*: 41.4.
- pf\_dep\_var\_max*: [41.2](#), 41.4, 47.1.
- pf\_eval\_name*: 41.4, 42.1.
- pf\_eval\_string*: 41.4.
- pf\_eval\_string\_length*: [41.2](#), 41.4, 42.1, 47.2, 48.1.
- pf\_max*: 41.4, 42.1.
- pf\_max\_indep\_params*: [41.2](#).
- pf\_max\_random*: [41.2](#).
- pf\_min*: 41.4, 42.1.
- pf\_mult*: 41.4, 42.1.
- pf\_name*: 41.4, 42.1.
- pf\_num\_dep\_var*: 41.4, 42.1.
- pf\_ragged\_alloc*: 41.6, 42.1, 42.2.
- pf\_ragged\_realloc*: [41.6](#), 42.1, 42.2.
- pf\_rank*: 41.4, 42.1.
- pf\_rank\_ind*: 41.4.
- pf\_rank\_ind0*: 41.4.
- pf\_spacing*: 41.4, 42.1.
- pf\_spacing\_linear*: [41.2](#), 48.1.
- pf\_spacing\_log*: [41.2](#), 48.1.
- pf\_spacing\_unknown*: [41.2](#).
- pf\_symbol\_string*: 41.4.
- pf\_tab\_index*: 41.3, 41.4, 41.6, 42.1, 42.2.
- pf\_table\_rank\_max*: [41.2](#), 41.4, 47.2.
- pf\_tag\_string*: 41.4.
- pf\_tag\_string\_length*: [41.2](#), 41.4, 42.1, 47.2, 48.1.
- pf\_unit\_string*: 41.4.
- pf\_unit\_string\_length*: [41.2](#), 41.4.
- pf\_units*: 41.4, 42.1.
- pf\_var*: 41.4, 42.1.
- pf\_var\_cos\_polar\_angle\_in*: [41.2](#).
- pf\_var\_energy\_in*: [41.2](#).
- pf\_var\_polar\_angle\_in*: [41.2](#).
- pf\_var\_t\_wall*: [41.2](#).
- pf\_var\_unknown*: [41.2](#).
- pf\_var\_vel.1\_in*: [41.2](#).
- pf\_var\_vel.2\_in*: [41.2](#).
- pf\_var\_vel.3\_in*: [41.2](#).
- pf\_var\_1st\_random\_number*: [41.2](#).
- pf\_var\_2nd\_random\_number*: [41.2](#).
- pf\_var\_3rd\_random\_number*: [41.2](#).
- pf\_version*: 41.4.
- pk*: 50.6, 50.7, 51.
- PK\_CAT\_MAX*: [50.6](#), 50.7.
- PK\_CAT\_MIN*: [50.6](#), 50.7.
- PK\_DIMEN*: [50.6](#).
- PK\_DUMMY*: [50.6](#).
- pk\_ncdecl*: 51.
- PK\_VAR*: [50.6](#).
- PK\_VARLOCAL*: [50.6](#).
- PK\_VARP*: [50.6](#).
- plasma*: 26.2, 27.1.

- plasma\_coords\_cartesian*: [20.1](#), [21.1](#).
- plasma\_coords\_cylindrical*: [20.1](#), [20.2](#), [20.3](#), [21.1](#).
- plasma\_e\_ion\_delta*: [26.2](#), [26.3](#), [27.1](#), [27.2](#).
- plasma\_e\_ion\_mult*: [26.2](#), [26.3](#), [27.1](#), [27.2](#).
- plasma\_ind*: [26.2](#), [27.1](#).
- plasma\_lookup*: [26.1](#), [27.2](#).
- plasma\_sector*: [26.1](#), [26.2](#), [26.3](#), [27.1](#).
- plasma\_temperature*: [26.2](#), [26.3](#), [27.1](#).
- plate*: [42.1](#), [48.1](#).
- pm*: [9.2](#), [18.2](#), [19.1](#), [42.1](#), [44.1](#).
- pm\_args*: [18](#).
- pm\_check*: [18](#).
- pm\_copy*: [18](#).
- pm\_decl**: [18](#).
- pm\_dummy*: [18](#).
- pm\_filename*: [18](#), [19.1](#).
- pm\_gen*: [18](#), [19.1](#).
- pm\_ignore*: [18.2](#).
- pm\_lookup*: [18](#), [19.3](#).
- pm\_materials*: [18](#), [19.1](#).
- pm\_name*: [18](#), [19.1](#).
- pm\_name.len*: [18.1](#), [18.2](#), [19.1](#).
- pm\_num*: [18](#), [18.2](#), [19.2](#), [43.4](#).
- pm\_pmi\_type*: [18](#), [19.1](#).
- pm\_product*: [18](#), [19.1](#).
- pm\_product\_max*: [18.1](#), [18.2](#).
- pm\_product\_num*: [18](#), [19.1](#).
- pm\_reagent*: [18](#), [19.1](#).
- pm\_sy*: [18](#), [19.1](#), [42.1](#).
- pm\_sy.len*: [18.1](#), [18.2](#), [19.1](#), [41.4](#), [42.1](#).
- pm\_type.len*: [18.1](#), [18.2](#), [19.1](#).
- pmi*: [8.3](#), [19.1](#).
- pmi\_bound\_product*: [18.2](#).
- pmi\_bound\_reagent*: [18.2](#).
- pmi\_filename*: [18](#), [18.2](#).
- pmi\_filename\_string*: [18.2](#).
- pmi\_generic*: [18](#), [18.2](#).
- pmi\_generic\_no*: [18](#), [19.1](#).
- pmi\_generic\_yes*: [18](#), [19.1](#).
- pmi\_handling*: [48.1](#), [48.2](#).
- pmi\_handling\_base*: [47.1](#), [47.2](#), [48.1](#).
- pmi\_handling\_delta*: [47.2](#), [48.1](#).
- pmi\_handling\_eval\_name*: [47.2](#), [48.1](#).
- pmi\_handling\_min*: [47.2](#), [48.1](#).
- pmi\_handling\_num\_rand*: [47.2](#), [48.1](#).
- pmi\_handling\_rank*: [47.2](#), [48.1](#).
- pmi\_handling\_size*: [47.2](#), [48.1](#).
- pmi\_handling\_spacing*: [47.2](#), [48.1](#).
- pmi\_handling\_tab*: [47.1](#), [47.2](#), [48.1](#).
- pmi\_handling\_tab\_index*: [47.1](#), [47.2](#), [48.1](#), [48.2](#).
- pmi\_handling\_var*: [47.2](#), [48.1](#).
- pmi\_handling\_var0*: [47.2](#), [48.1](#).
- pmi\_ind*: [18.2](#).
- pmi\_infile*: [54.1](#).
- pmi\_materials*: [18](#), [18.2](#).
- pmi\_materials\_string*: [18.2](#).
- pmi\_name*: [18](#), [18.2](#).
- pmi\_name\_string*: [18.2](#).
- pmi\_product*: [18](#), [18.2](#).
- pmi\_product\_ind*: [18.2](#).
- pmi\_product\_num*: [18](#), [18.2](#).
- pmi\_reagent*: [18](#), [18.2](#).
- pmi\_sy*: [18](#), [18.2](#).
- pmi\_symbol\_string*: [18.2](#).
- pmi\_type*: [18](#), [18.2](#).
- pmi\_type\_string*: [18.2](#).
- pmi\_version*: [18.2](#).
- pmi\_yield*: [48.2](#).
- pmi\_yield\_base*: [47.1](#), [47.2](#), [48.1](#).
- pmi\_yield\_delta*: [47.2](#), [48.1](#).
- pmi\_yield\_eval\_name*: [47.2](#), [48.1](#).
- pmi\_yield\_min*: [47.2](#), [48.1](#).
- pmi\_yield\_num\_rand*: [47.2](#), [48.1](#).
- pmi\_yield\_rank*: [47.2](#), [48.1](#).
- pmi\_yield\_size*: [47.2](#), [48.1](#).
- pmi\_yield\_spacing*: [47.2](#), [48.1](#).
- pmi\_yield\_tab*: [47.1](#), [47.2](#), [48.1](#).
- pmi\_yield\_tab\_index*: [47.1](#), [47.2](#), [48.1](#), [48.2](#).
- pmi\_yield\_var*: [47.2](#), [48.1](#).
- pmidata*: [30.6](#), [45.3](#), [46.1](#), [48.1](#).
- pmiifile*: [54.1](#).
- pmiiformat*: [38.2](#), [42.1](#), [42.2](#), [48.1](#).
- pointer*: [27.1](#), [50.2](#), [50.4](#).
- pointer\_*: [12.1](#).
- points\_ind*: [57.1](#).
- poly\_ind*: [57.1](#).
- pos\_*: [10](#).
- pr*: [43.4](#), [44.1](#).
- pr\_arrangement\_max*: [43.2](#), [43.4](#).
- pr\_background*: [43.1](#), [44.1](#).
- pr\_background\_args*: [43.1](#).
- pr\_background\_check*: [43.1](#).
- pr\_background\_copy*: [43.1](#).
- pr\_background\_decl**: [43.1](#).
- pr\_background\_dummy*: [43.1](#).
- pr\_background\_lookup*: [43.1](#), [44.3](#).
- pr\_background\_num*: [32.2](#), [32.3](#), [43.1](#), [43.3](#), [43.4](#), [44.2](#), [44.3](#).
- pr\_bk\_rc*: [43.1](#), [44.1](#).
- pr\_bkrc\_dim*: [43.4](#), [44.2](#).
- pr\_bkrc\_num*: [43.4](#), [44.2](#).
- pr\_bkrc\_prod*: [43.1](#), [44.1](#).
- pr\_bkrc\_reagent\_max*: [43.2](#), [43.4](#), [44.1](#).
- pr\_bkrc\_rg*: [43.1](#), [44.1](#).

- pr\_equiv\_ind*: 43.5.
- pr\_generic\_args*: 43.1.
- pr\_generic\_copy*: 43.1.
- pr\_generic\_decl**: 43.1.
- pr\_generic\_dummy*: 43.1.
- pr\_mat\_ref*: 43.1, 44.1.
- pr\_materials\_args*: 43.1.
- pr\_materials\_check*: 43.1.
- pr\_materials\_copy*: 43.1.
- pr\_materials\_decl**: 43.1.
- pr\_materials\_dummy*: 43.1.
- pr\_materials\_num*: 43.1, 43.4, 44.2.
- pr\_materials\_sub*: 44.2.
- pr\_max\_equiv*: 43.2, 43.5.
- pr\_num\_arrangements*: 43.1, 44.1.
- pr\_num\_change\_vars*: 43.3.
- pr\_num\_diag\_vars*: 43.3.
- pr\_pm\_case\_num*: 43.1, 44.1.
- pr\_pm\_cases*: 43.1, 44.1.
- pr\_pm\_num\_arrange*: 43.1, 44.1.
- pr\_pm\_prod*: 43.1, 44.1.
- pr\_pm\_prod\_mult*: 43.1.
- pr\_pm\_ref*: 43.1, 44.1.
- pr\_pmi*: 48.1, 48.2.
- pr\_pmi\_args*: 43.1.
- pr\_pmi\_check*: 43.1.
- pr\_pmi\_copy*: 43.1.
- pr\_pmi\_decl**: 43.1.
- pr\_pmi\_dummy*: 43.1.
- pr\_pmi\_max*: 43.2, 43.4.
- pr\_pmi\_num*: 8.1, 43.1, 43.4, 44.2.
- pr\_problem\_sp\_back*: 43.3, 44.3.
- pr\_problem\_sp\_test*: 43.3, 44.3.
- pr\_prod\_mult*: 43.1, 44.1.
- pr\_rc\_num*: 43.1, 44.1.
- pr\_reac*: 46.1, 46.2.
- pr\_reaction*: 43.1, 44.1.
- pr\_reaction\_args*: 43.1.
- pr\_reaction\_check*: 43.1.
- pr\_reaction\_copy*: 43.1.
- pr\_reaction\_decl**: 43.1.
- pr\_reaction\_dim*: 30.4, 43.4.
- pr\_reaction\_dummy*: 43.1.
- pr\_reaction\_lookup*: 43.1, 44.3.
- pr\_reaction\_max*: 43.2, 43.4.
- pr\_reaction\_num*: 8.1, 8.3, 43.1, 43.4, 44.2.
- pr\_tag\_string*: 43.4.
- pr\_tag\_string\_length*: 43.2, 43.4.
- pr\_test*: 8.1, 43.1, 44.1.
- pr\_test\_args*: 43.1.
- pr\_test\_check*: 8.1, 43.1.
- pr\_test\_copy*: 43.1.
- pr\_test\_decl**: 8.1, 43.1.
- pr\_test\_dummy*: 43.1.
- pr\_test\_lookup*: 8.3, 43.1, 44.3.
- pr\_test\_num*: 43.1, 43.4, 44.2, 44.3.
- pr\_ts\_bk*: 43.1, 44.1.
- pr\_ts\_prod*: 43.1, 44.1.
- pr\_ts\_rc*: 43.1, 44.1.
- pr\_var\_angle*: 43.3.
- pr\_var\_back\_index*: 43.3.
- pr\_var\_emitter\_v\_vector*: 43.3.
- pr\_var\_emitter\_v\_2*: 43.3.
- pr\_var\_emitter\_v\_3*: 43.3.
- pr\_var\_emitter\_vf\_Maxwell\_vector*: 43.3.
- pr\_var\_emitter\_vf\_Maxwell\_2*: 43.3.
- pr\_var\_emitter\_vf\_Maxwell\_3*: 43.3.
- pr\_var\_emitter\_vth\_Maxwell*: 43.3.
- pr\_var\_energy*: 43.3.
- pr\_var\_energy\_change*: 43.3.
- pr\_var\_energy\_in*: 43.3.
- pr\_var\_energy\_out*: 43.3.
- pr\_var\_mass*: 43.3.
- pr\_var\_mass\_change*: 43.3, 44.3.
- pr\_var\_mass\_in*: 43.3.
- pr\_var\_mass\_out*: 43.3.
- pr\_var\_momentum\_change\_vector*: 43.3.
- pr\_var\_momentum\_change\_2*: 43.3.
- pr\_var\_momentum\_change\_3*: 43.3.
- pr\_var\_momentum\_in\_vector*: 43.3.
- pr\_var\_momentum\_in\_2*: 43.3.
- pr\_var\_momentum\_in\_3*: 43.3.
- pr\_var\_momentum\_out\_vector*: 43.3.
- pr\_var\_momentum\_out\_2*: 43.3.
- pr\_var\_momentum\_out\_3*: 43.3.
- pr\_var\_momentum\_vector*: 43.3.
- pr\_var\_momentum\_2*: 43.3.
- pr\_var\_momentum\_3*: 43.3.
- pr\_var\_problem\_sp\_index*: 30.3, 43.3, 44.3.
- pr\_var\_unknown*: 43.3.
- pr\_var\_xz\_stress*: 43.3.
- pr\_var0\_list*: 31.1, 43.4, 44.2, 44.3, 46.1.
- pr\_var0\_list\_ind*: 43.4.
- pr\_var0\_num*: 43.4, 44.2.
- problem*: 44.4.
- problem\_arr\_ind*: 43.4.
- problem\_background\_ind*: 43.4.
- problem\_background\_reaction*: 43.1, 43.4.
- problem\_background\_sp*: 43.1, 43.4.
- problem\_bkrc\_ind*: 43.4.
- problem\_bkrc\_products*: 43.1, 43.4.
- problem\_bkrc\_reagents*: 43.1, 43.4.
- problem\_bkrc\_rg\_ind*: 43.4.
- problem\_infile*: 54.1.

*problem\_materials\_ref*: 43.1, 43.4.  
*problem\_materials\_ref\_ind*: 43.4.  
*problem\_materials\_sub*: 43.4.  
*problem\_materials\_sub\_ind*: 43.4.  
*problem\_num\_arrangements*: 43.1, 43.4.  
*problem\_pmi\_case\_num*: 43.1, 43.4.  
*problem\_pmi\_cases*: 43.1, 43.4.  
*problem\_pmi\_ind0*: 43.4.  
*problem\_pmi\_num\_arrange*: 43.1, 43.4.  
*problem\_pmi\_prod\_mult*: 43.1, 43.4.  
*problem\_pmi\_products*: 43.1, 43.4.  
*problem\_pmi\_ref*: 43.1, 43.4.  
*problem\_pmi\_ref\_ind*: 43.4.  
*problem\_pmi\_sub*: 43.4, 44.2.  
*problem\_pmi\_sub\_ind*: 43.4, 47.2.  
*problem\_prod\_mult*: 43.1, 43.4.  
*problem\_product\_ind*: 43.4.  
*problem\_rc*: 43.1, 43.4.  
*problem\_reaction\_ind*: 43.4, 45.2.  
*problem\_reaction\_ind0*: 43.4.  
*problem\_reaction\_num*: 43.1, 43.4.  
*problem\_species\_background*: 43.1, 43.4.  
*problem\_species\_ind*: 43.4, 43.5.  
*problem\_species\_test*: 43.1, 43.4.  
*problem\_test\_background*: 43.1, 43.4.  
*problem\_test\_ind*: 43.4.  
*problem\_test\_products*: 43.1, 43.4.  
*problem\_test\_reaction*: 43.1, 43.4.  
*problem\_test\_sp*: 43.1, 43.4.  
*problem\_version*: 43.4.  
*problemfile*: [54.1](#).  
*problemsetup*: 44.4.  
*process\_pk*: [50.7](#).  
*product\_ind*: 14.3.  
*ps*: 43.5, 44.4.  
*psp*: 43.3, 44.3.  
*pt*: 9.1.  
*pt\_args*: [8.1](#), 8.3, 12.1.  
*pt\_author*: [8.1](#), 8.3, 9.1.  
*pt\_author\_pm*: [8.3](#), 9.2.  
*pt\_author\_rc*: [8.3](#), 9.2.  
*pt\_author\_so*: [8.3](#), 9.2.  
*pt\_author1*: [8.1](#).  
*pt\_check*: [8.1](#).  
*pt\_copy*: [8.1](#), 12.1, 12.3.  
**pt\_decl**: [8.1](#).  
**pt\_decls**: [8.1](#).  
*pt\_dummy*: [8.1](#), 12.1.  
*pt\_geometry*: [8.2](#), 8.3.  
*pt\_init*: [8.3](#), 9.2, 12.3.  
*pt\_ion*: [8.2](#), 8.3.  
*pt\_loc*: [8.1](#), 8.3, 9.1.  
*pt\_loc1*: [8.1](#).  
*pt\_neutral*: [8.2](#), 8.3.  
*pt\_scatter\_thru*: [8.3](#), 9.2.  
*pt\_sp*: 1, [8.1](#), 8.3, 9.1.  
*pt\_specular*: [8.3](#), 9.2.  
*pt\_sp1*: [8.1](#).  
*pt\_t*: [8.1](#), 8.3, 9.1.  
*pt\_test*: [8.1](#), 8.3, 9.1.  
*pt\_test1*: [8.1](#).  
*pt\_thru\_face*: [8.3](#), 9.2.  
*pt\_track*: [8.3](#), 9.2.  
*pt\_type*: [8.1](#), 8.3, 9.1.  
*pt\_t1*: [8.1](#).  
*pt\_v*: [8.1](#), 8.3, 9.1.  
*pt\_v1*: [8.1](#).  
*pt\_w*: [8.1](#), 8.3, 9.1.  
*pt\_w1*: [8.1](#).  
*ptr*: 50.4, 51.  
*ptr\_arrayname*: 51.  
**ptr\_decl**: 50.2, [50.4](#).  
*ptr\_foo*: 51.  
*ptr\_ind*: 31.1.  
*ptr\_junk\_array*: 50.2.  
*ptr\_x*: 50.4.  
  
*ra*: 39.2, 40.1.  
*ra\_bool\_and\_label*: 39.2, 40.1.  
*ra\_bool\_not\_label*: 39.2, 40.1.  
*ra\_data\_filename*: 39.2, 40.1.  
*ra\_data\_filename\_length*: [39.1](#), 39.2.  
*ra\_dep\_var\_ind*: 39.2.  
*ra\_fit\_coef\_max*: [39.1](#), 39.4.  
*ra\_hier\_label*: 39.2, 40.1.  
*ra\_label\_max*: [39.1](#), 39.2.  
**ra\_localcommon**: [39.4](#).  
*ra\_mass*: 39.2, 40.1.  
*ra\_mass\_units*: 39.2, 40.1.  
*ra\_num\_bool\_and*: 39.2, 40.1.  
*ra\_num\_bool\_not*: 39.2, 40.1.  
*ra\_num\_hier*: 39.2, 40.1.  
*ra\_search\_label\_length*: [39.1](#), 39.2.  
*ra\_unit\_string\_length*: [39.1](#), 39.2.  
*ragged\_alloc*: 30.6, 45.3.  
*ran\_array*: 36.1.  
*ran\_c*: [36.2](#).  
*ran\_index*: 36.1.  
*ran\_k*: 36.1, [36.2](#).  
*ran\_s*: 32.3, 33.1, [36.2](#), 36.3.  
*ran\_temp*: 36.1.  
*random*: 22.2, [36.1](#), 36.3.  
*random\_cosdist*: 36.3.  
*random\_gauss*: 36.3.  
*random\_init*: 36.3.

- random\_isodist*: 36.3.
- range*: [50.2](#).
- range\_max*: 30.6, 37.5, 41.6, 45.3, 47.3, [50.2](#), 50.5.
- range\_max1*: [50.2](#).
- range\_min*: 30.6, 37.5, 41.6, 45.3, 47.3, [50.2](#), 50.3, 50.5.
- range\_min1*: [50.2](#).
- range\_size*: [50.2](#), 50.5, 52.2.
- range\_star*: [50.2](#).
- range\_stara*: [50.2](#).
- rangea*: [50.2](#).
- rank*: 48.1.
- rank\_*: 52.2.
- rank\_ind*: 31.1, 37.3, 38.1.
- rank\_ind0*: 37.3, 38.1.
- rank\_x*: 50.2.
- ratecalc*: 38.1.
- rc*: 9.2, 14.3, 15.1, 43.1, 44.1.
- rc\_args*: [14.1](#).
- rc\_check*: [14.1](#).
- rc\_copy*: [14.1](#).
- rc\_decl**: [14.1](#).
- rc\_dummy*: [14.1](#).
- rc\_emitter*: [14.1](#), 15.1.
- rc\_filename*: [14.1](#), 15.1.
- rc\_gen*: [14.1](#), 15.1.
- rc\_generic\_no*: [14.1](#), 15.1.
- rc\_generic\_yes*: [14.1](#), 15.1.
- rc\_lookup*: [14.1](#), 15.2.
- rc\_name*: [14.1](#), 15.1.
- rc\_name\_len*: [14.2](#), 14.3, 15.1.
- rc\_num*: 14.1, 14.3, 38.1.
- rc\_product*: [14.1](#), 15.1, 44.1.
- rc\_product\_max*: [14.2](#), 14.3, 15.1, 43.4.
- rc\_product\_num*: [14.1](#), 15.1.
- rc\_reaction\_type*: [14.1](#), 15.1.
- rc\_reagent*: 1, [14.1](#), 15.1.
- rc\_reagent\_max*: [14.2](#), 14.3, 15.1.
- rc\_reagent\_num*: [14.1](#), 15.1.
- rc\_sy*: [14.1](#), 15.1, 38.1.
- rc\_sy\_len*: [14.2](#), 14.3, 15.1, 37.3.
- rc\_type\_len*: [14.2](#), 14.3.
- rd*: 45.2, 46.1, 48.1.
- rd\_data*: 46.2.
- rd\_data\_args*: [45.1](#), 46.2.
- rd\_data\_base*: 46.2.
- rd\_data\_size*: 46.2.
- rd\_data\_tab*: 46.2.
- rd\_decls**: [45.3](#).
- rd\_dep\_var\_ind*: 45.2.
- rd\_dep\_var\_max*: [45.1](#), 45.2.
- rd\_eval\_string*: 45.2.
- rd\_handling*: [45.1](#), 46.1, 46.2.
- rd\_handling\_table\_ind*: 45.2.
- rd\_null\_alloc*: [45.3](#).
- rd\_ragged\_alloc*: [45.3](#), 46.1, 46.2.
- rd\_ragged\_realloc*: [45.3](#), 46.1, 46.2.
- rd\_rank\_ind*: 45.2.
- rd\_rank\_ind0*: 45.2.
- rd\_rate*: [45.1](#), 46.1, 46.2.
- rd\_set\_baseinc*: [45.3](#).
- rd\_tab\_index*: 46.2.
- rd\_table\_ind*: 45.2.
- rd\_tag\_string*: 45.2.
- reac*: 8.3.
- reac\_so*: 31.1.
- react\_begin*: [39.4](#).
- react\_done*: [39.4](#).
- reaction*: 15.1.
- reaction\_emitter*: 14.1, 14.3.
- reaction\_filename*: 14.1, 14.3.
- reaction\_filename\_string*: 14.3.
- reaction\_generic*: 14.1, 14.3.
- reaction\_handling*: 1, 46.1, 46.2.
- reaction\_handling\_base*: 45.1, 45.2, 46.1.
- reaction\_handling\_delta*: 45.2, 46.1.
- reaction\_handling\_eval\_name*: 45.2, 46.1.
- reaction\_handling\_min*: 45.2, 46.1.
- reaction\_handling\_num\_rand*: 45.2, 46.1.
- reaction\_handling\_rank*: 45.2, 46.1.
- reaction\_handling\_size*: 45.2, 46.1.
- reaction\_handling\_spacing*: 45.2, 46.1.
- reaction\_handling\_tab*: 45.1, 45.2, 46.1.
- reaction\_handling\_tab\_index*: 45.1, 45.2, 46.1.
- reaction\_handling\_var*: 45.2, 46.1.
- reaction\_handling\_var0*: 31.1, 45.2, 46.1.
- reaction\_ind*: 1, 14.3.
- reaction\_infile*: [54.1](#).
- reaction\_name*: 14.1, 14.3.
- reaction\_name\_string*: 14.3.
- reaction\_product*: 14.1, 14.3.
- reaction\_product\_num*: 14.1, 14.3.
- reaction\_rate*: 46.1, 46.2.
- reaction\_rate\_base*: 45.1, 45.2, 46.1.
- reaction\_rate\_delta*: 45.2, 46.1.
- reaction\_rate\_eval\_name*: 45.2, 46.1.
- reaction\_rate\_min*: 45.2, 46.1.
- reaction\_rate\_num\_rand*: 45.2, 46.1.
- reaction\_rate\_rank*: 45.2, 46.1.
- reaction\_rate\_size*: 45.2, 46.1.
- reaction\_rate\_spacing*: 45.2, 46.1.
- reaction\_rate\_tab*: 45.1, 45.2, 46.1.
- reaction\_rate\_tab\_index*: 45.1, 45.2, 46.1.
- reaction\_rate\_var*: 45.2, 46.1.

*reaction\_reagent*: 1, 14.1, 14.3.  
*reaction\_reagent\_num*: 14.1, 14.3.  
*reaction\_sy*: 14.1, 14.3.  
*reaction\_symbol\_string*: 14.3.  
*reaction\_type*: 14.1, 14.3.  
*reaction\_type\_string*: 14.3.  
*reaction\_version*: 14.3.  
*reactionfile*: [54.1](#).  
*read\_int\_soft\_fail*: [49](#).  
*read\_integer*: [49](#).  
*read\_real*: [49](#).  
*read\_real\_soft\_fail*: [49](#).  
*read\_string*: [49](#).  
*read\_text*: [49.2](#).  
*readbackground*: 23.3.  
*readfilenames*: 55.  
*reagent\_ind*: 1, 14.3.  
*real\_alloc*: [50.4](#).  
*real\_free*: [50.4](#).  
*real\_realloc*: [50.4](#).  
*recyc*: 26.3, 27.2.  
*register*: [50.6](#).  
*rf*: [54.2](#).  
*rn\_args*: 12.1, [36.1](#), 36.3.  
*rn\_array*: [36.1](#).  
*rn\_copy*: 12.1, [36.1](#).  
*rn\_cos\_next*: [36.3](#).  
**rn\_decl**: 12.1, [36.1](#).  
**rn\_decls**: [36.1](#).  
*rn\_dummy*: 12.1, [36.1](#).  
*rn\_gauss\_next*: [36.3](#).  
*rn\_index*: [36.1](#).  
*rn\_init*: [36.3](#).  
*rn\_iso\_next*: [36.3](#).  
*rn\_next*: [36.3](#).  
*rn\_seed\_args*: [36.3](#).  
*rn\_seed\_copy*: [36.3](#).  
*rn\_seed\_copy1*: [36.3](#).  
**rn\_seed\_decl**: [36.1](#), [36.3](#).  
  
*sa*: 34.2, 35.1.  
*sample\_sources*: 23.2.  
*sc*: 26.2, 26.3, 27.1.  
*sc\_args*: [26.1](#).  
*sc\_check*: [26.1](#), 27.2.  
*sc\_compare*: [26.1](#), 27.2.  
*sc\_copy*: [26.1](#).  
**sc\_decl**: [26.1](#).  
*sc\_diag\_angle*: [26.1](#), 27.1.  
*sc\_diag\_check*: [26.1](#), 27.2.  
*sc\_diag\_energy*: [26.1](#), 27.1.  
*sc\_diag\_ind*: 26.2.  
*sc\_diag\_max\_bins*: 26.2, 27.1, 29.1.

*sc\_diag\_name\_len*: [26.1](#), 26.2.  
*sc\_diag\_name\_string*: 26.2.  
*sc\_diag\_size*: 26.2, 27.1.  
*sc\_diag\_spacing\_linear*: [26.1](#).  
*sc\_diag\_spacing\_log*: [26.1](#).  
*sc\_diag\_spacing\_unknown*: [26.1](#).  
*sc\_diag\_unknown*: [26.1](#).  
*sc\_diagnostic*: [26.1](#), 27.2.  
*sc\_diagnostic\_grps*: 26.1, 26.2, 27.1.  
*sc\_dummy*: [26.1](#).  
*sc\_exit*: [26.1](#), 26.3.  
*sc\_exit\_check*: [26.1](#), 27.2.  
*sc\_exit\_num*: 26.1, 26.2, 26.3, 27.1.  
*sc\_neg*: 26.2, [26.3](#).  
*sc\_plasma*: [26.1](#), 26.3, 27.2.  
*sc\_plasma\_check*: [26.1](#), 27.2.  
*sc\_plasma\_decl*: [26.1](#).  
*sc\_plasma\_num*: 26.1, 26.2, 26.3, 27.1.  
*sc\_pos*: 26.2, [26.3](#).  
*sc\_target*: [26.1](#), 26.3.  
*sc\_target\_check*: [26.1](#), 27.2.  
*sc\_target\_decl*: [26.1](#).  
*sc\_target\_num*: 26.1, 26.2, 26.3, 27.1.  
*sc\_type\_max*: [26.1](#), 26.2.  
*sc\_unknown*: [26.1](#).  
*sc\_vacuum*: [26.1](#), 26.3.  
*sc\_vacuum\_check*: [26.1](#).  
*sc\_vacuum\_num*: 26.1, 26.2, 26.3, 27.1.  
*sc\_wall*: [26.1](#), 26.3, 27.2.  
*sc\_wall\_check*: [26.1](#), 27.2.  
*sc\_wall\_decl*: [26.1](#).  
*sc\_wall\_num*: 26.1, 26.2, 26.3, 27.1.  
*scal\_ind*: 31.1.  
*scoring\_data*: 30.3, 31.2.  
*scoring\_data\_max*: [30.3](#).  
*se*: 29.1.  
*search\_label*: 39.2.  
*sect*: 26.1, 27.2.  
*sector*: 26.1, 27.1, 27.2.  
*sector\_*: 10.  
*sector\_ind*: 26.2.  
*sector\_neg\_pos\_ind*: 26.2.  
*sector\_next\_*: 10.  
*sector\_points*: 26.2, 27.1.  
*sector\_strata\_segment*: 26.1, 26.2, 27.1.  
*sector\_surface*: 26.1, 26.2, 27.1.  
*sector\_type\_ind*: 26.2.  
*sector\_type\_pointer*: 26.2, 26.3, 27.1.  
*sector\_zone*: 26.1, 26.2, 27.1.  
*sectors*: 26.2, 27.1.  
*seed*: [36.3](#).  
*seed\_ind*: 33.1.

- seg*: 26.1, 27.2.
- set\_base*: 37.5, 41.6.
- set\_inc*: 37.5, 41.6.
- set\_prob\_alias*: 23.2.
- set\_var\_list*: 30.3, 31.1.
- single\_precision**: 36.1.
- size*: 50.2.
- slice1*: 50.3.
- so*: 22.4, 23.1, 44.1.
- so\_args*: 22.1.
- so\_base*: 22.1, 23.1.
- so\_check*: 22.1.
- so\_chkpt*: 22.1, 23.1.
- so\_copy*: 22.1.
- so\_decl**: 22.1.
- so\_direct*: 22.1, 23.3.
- so\_direct\_delta*: 22.4.
- so\_direct\_mult*: 22.1.
- so\_dummy*: 22.1.
- so\_geom*: 22.1, 23.1.
- so\_grps*: 12.1, 22.1, 22.4, 23.2, 32.3.
- so\_line*: 22.1.
- so\_name*: 22.1, 22.3, 23.2.
- so\_name\_ind*: 22.4.
- so\_name\_len*: 22.1, 22.4.
- so\_nflights*: 22.1, 23.1.
- so\_nseg*: 22.1, 23.1.
- so\_param\_ind*: 22.4.
- so\_param\_max*: 22.1, 22.4.
- so\_param\_puff\_temp*: 22.1.
- so\_param\_unknown*: 22.1.
- so\_params*: 22.1, 23.1, 27.1.
- so\_plate*: 22.1, 22.3.
- so\_point*: 22.1.
- so\_puff*: 22.1, 22.3.
- so\_random*: 22.1, 22.3, 23.3.
- so\_recomb*: 22.1, 22.3.
- so\_rel\_wt\_max*: 22.3, 22.4, 23.3.
- so\_rel\_wt\_min*: 22.3, 22.4, 23.3.
- so\_restart*: 22.3, 22.4, 23.3.
- so\_root\_sp*: 22.1, 23.1.
- so\_sampling*: 22.3, 22.4, 23.3.
- so\_scale*: 22.1, 23.1.
- so\_seed\_len*: 22.2, 22.4.
- so\_seed\_spacing*: 22.3, 22.4, 23.3.
- so\_seed\_string*: 22.3, 22.4, 23.3.
- so\_seed\_string\_ind*: 22.4.
- so\_seg\_check*: 22.1.
- so\_seg\_tot*: 22.1, 22.4, 23.2.
- so\_set\_run\_flags*: 22.3, 23.3.
- so\_spaced\_seeds*: 22.3, 22.4, 23.3, 33.1.
- so\_species*: 22.1, 23.1.
- so\_surface*: 22.1.
- so\_tot\_curr*: 22.1, 23.1.
- so\_type*: 8.3, 22.1, 23.1.
- so\_type\_ind*: 22.4.
- so\_type\_num*: 8.1, 8.3, 22.1, 22.4, 30.4.
- so\_volume*: 22.1.
- so\_wt\_norm*: 22.1, 23.1, 23.2, 23.3.
- so\_wt\_norm\_max*: 22.3, 22.4, 23.3.
- so\_wt\_norm\_min*: 22.3, 22.4, 23.3.
- source*: 8.3, 9.2, 50.7.
- source\_*: 12.1.
- source\_base\_ptr*: 22.1, 22.4.
- source\_current*: 22.4, 23.2.
- source\_geometry*: 22.1, 22.4.
- source\_grp\_ind*: 22.4.
- source\_name*: 22.1, 22.4.
- source\_num\_checkpoints*: 22.1, 22.4.
- source\_num\_flights*: 22.1, 22.4.
- source\_num\_segments*: 22.1, 22.4.
- source\_parameters*: 22.1, 22.4.
- source\_root\_species*: 22.1, 22.4.
- source\_scale\_factor*: 22.1, 22.4.
- source\_seg\_ind*: 22.4.
- source\_segment\_prob\_alias*: 22.4, 23.2.
- source\_segment\_ptr*: 22.4, 23.2.
- source\_segment\_ptr\_alias*: 22.4, 23.2.
- source\_segment\_rel\_wt*: 22.4, 23.1, 23.2, 23.3.
- source\_species*: 22.1, 22.4.
- source\_total\_current*: 22.1, 22.4.
- source\_type*: 22.1, 22.4.
- source\_weight\_norm*: 22.1, 22.4.
- sp*: 6.3, 7.1, 19.1, 43.1, 44.1.
- SP*: 49.1.
- sp\_args*: 6.1.
- sp\_check*: 6.1, 8.1.
- sp\_copy*: 6.1.
- sp\_count*: 6.1, 7.1.
- sp\_decl**: 6.1.
- sp\_dummy*: 6.1.
- sp\_el*: 1, 6.1, 7.1.
- sp\_generic*: 6.1, 7.1.
- sp\_lookup*: 6.1, 7.2, 8.3.
- sp\_m*: 6.1, 7.1, 8.3.
- sp\_multiplicity*: 6.1, 7.1.
- sp\_name*: 6.1, 7.1.
- sp\_name\_len*: 6.2, 6.3, 7.1.
- sp\_ncomp*: 6.1, 7.1.
- sp\_ncomp\_max*: 6.2, 6.3, 7.1.
- sp\_num*: 6.1, 6.3, 43.4.
- sp\_sy*: 6.1, 7.1.
- sp\_sy\_len*: 6.2, 6.3, 7.1.
- sp\_z*: 6.1, 7.1, 8.3.

- species*: 7.1.
- species\_*: 8.1.
- species\_comp\_ind*: 6.3.
- species\_count*: 6.1, 6.3.
- species\_el*: 6.1, 6.3.
- species\_generic*: 6.1, 6.3.
- species\_ind*: 6.3.
- species\_infile*: [54.1](#).
- species\_m*: 6.1, 6.3.
- species\_multiplicity*: 6.1, 6.3.
- species\_name*: 6.1, 6.3.
- species\_name\_string*: 6.3.
- species\_ncomp*: 6.1, 6.3.
- species\_sy*: 6.1, 6.3.
- species\_symbol\_string*: 6.3.
- species\_version*: 6.3.
- species\_z*: 6.1, 6.3.
- speciesfile*: [54.1](#).
- sqrt*: 20.2, 20.3, 22.1.
- src*: 55.
- st\_decls**: 49.
- stack\_*: 12.1.
- start\_*: 50.7.
- start\_common*: [50.7](#).
- start\_module*: [50.7](#).
- start\_mpibroadcast*: [50.7](#).
- start\_mpireceive*: [50.7](#).
- start\_mpisend*: [50.7](#).
- start\_ncdecl*: [50.7](#).
- start\_ncdef*: [50.7](#).
- start\_ncread*: [50.7](#).
- start\_ncwrite*: [50.7](#).
- start\_pvmget*: [50.7](#).
- start\_pvmput*: [50.7](#).
- stat\_comp\_fff*: 35.1.
- stat\_comp\_fin*: 34.2.
- stat\_comp\_flt*: 34.2.
- stat\_comp\_frag*: 34.2.
- stat\_dim\_fff*: 35.1.
- stat\_dim\_fin*: 34.2.
- stat\_dim\_flt*: 34.2.
- stat\_dim\_frag*: 34.2.
- stat\_fff*: 35.1.
- stat\_fin*: 34.2, 35.1.
- stat\_fin\_ind*: 34.2.
- stat\_flt*: 34.2.
- stat\_flt\_ind*: 34.2.
- stat\_frag*: 34.2.
- stat\_frag\_ind*: 34.2.
- stat\_mem\_inc\_fff*: 35.2.
- stat\_mem\_inc\_fin*: [34.1](#).
- stat\_mem\_inc\_flt*: [34.1](#).
- stat\_mem\_inc\_frag*: [34.1](#).
- stat\_moments\_ind*: 34.2.
- stat\_pf\_dim\_fff*: 35.1.
- stat\_pf\_dim\_fin*: 34.2.
- stat\_pf\_dim\_flt*: 34.2.
- stat\_pf\_dim\_frag*: 34.2.
- stat\_pf\_fin\_ind*: 34.2.
- stat\_pf\_flt\_ind*: 34.2.
- stat\_pf\_frag\_ind*: 34.2.
- stat\_ps\_dim\_fff*: 35.1.
- stat\_ps\_dim\_fin*: 34.2.
- stat\_ps\_dim\_flt*: 34.2.
- stat\_ps\_dim\_frag*: 34.2.
- stat\_ps\_fin\_ind*: 34.2.
- stat\_ps\_flt\_ind*: 34.2.
- stat\_ps\_frag\_ind*: 34.2.
- stat\_ptr2full\_fff*: 35.1.
- stat\_ptr2full\_fin*: 34.2.
- stat\_ptr2full\_flt*: 34.2.
- stat\_ptr2full\_frag*: 34.2.
- stat\_ptr2short\_fff*: 35.1.
- stat\_ptr2short\_fin*: 34.2.
- stat\_ptr2short\_flt*: 34.2.
- stat\_ptr2short\_frag*: 34.2.
- stat\_size\_fff*: 35.1.
- stat\_size\_fin*: 34.2.
- stat\_size\_flt*: 34.2.
- stat\_size\_frag*: 34.2.
- stat\_wt\_dim\_fff*: 35.1.
- stat\_wt\_dim\_fin*: 34.2.
- stat\_wt\_dim\_frag*: 34.2.
- stat\_wt\_fff*: 35.1.
- stat\_wt\_fin*: 34.2.
- stat\_wt\_fin\_ind*: 34.2.
- stat\_wt\_frag*: 34.2, 35.1.
- stat\_wt\_frag\_ind*: 34.2.
- stat\_wt\_tot\_fff*: 35.1.
- stat\_wt\_tot\_fin*: 34.2.
- stat\_wt\_tot\_flt*: 34.2.
- stat\_wt\_tot\_frag*: 34.2.
- strata*: 26.1, 26.2, 27.1.
- stratum*: 26.1, 27.2.
- string*: 4.3, 6.3, 14.3, 16.3, 18.2, 24.3, 30.4, 32.3, 37.3, 41.4, 43.4.
- string\_length*: [49](#), 49.1.
- string\_lookup*: 4.1, 6.1, 14.1, 16.1, 18, 26.1, 28.1, [49](#).
- subs*: 30.6, 45.1, 45.3, 46.2, 47.1, 47.3, 48.2.
- surface*: 27.1.
- surface\_*: 10.
- surface\_reflect*: 8.3.
- surface\_sectors*: 27.1.
- surface\_specular*: 8.3.

*sy*: 4.1, 6.1, 8.3, 9.2, 12.3, 13.2, 14.1, 16.1, 18.

*sym*: 20.2, 20.3.

*symbol*: 5.2, 7.2, 15.2, 17.3, 19.3.

*t\_sector*: 27.2.

*tab\_index*: 48.1.

*table\_calc*: 38.1.

*table\_eval*: 38.1.

*table\_external*: 38.1.

*table\_ind*: 33.1.

*tag*: 50.7.

*tag\_string*: 37.3.

*tally*: 31.1.

*tally\_base*: 30.4, 31.1, 32.2.

*tally\_cv\_action*: 30.4, 31.1.

*tally\_cv\_ind*: 30.4.

*tally\_cv\_num\_partners*: 30.4, 31.1.

*tally\_cv\_partner*: 31.1.

*tally\_cv\_partner\_ind*: 30.4.

*tally\_cv\_partners*: 30.4.

*tally\_cv\_ptr*: 30.4, 31.1.

*tally\_cv\_ptr\_ind*: 30.4.

*tally\_cv\_scaler\_ind*: 30.4.

*tally\_cv\_scalers*: 30.4, 31.1.

*tally\_cv\_type*: 30.4, 31.1.

*tally\_dep\_var*: 30.4, 31.1.

*tally\_dep\_var\_dim*: 30.3, 30.4, 31.1, 31.2, 32.2, 33.2.

*tally\_est\_ind*: 30.4.

*tally\_est\_reaction*: 30.4, 31.1.

*tally\_est\_test*: 30.4, 31.1.

*tally\_geometry*: 30.4, 31.1.

*tally\_geometry\_ptr*: 30.4, 31.1.

*tally\_ind*: 30.4, 31.1.

*tally\_indep\_var*: 30.4, 31.1.

*tally\_index\_ind*: 30.4.

*tally\_infile*: 54.1.

*tally\_name*: 30.4, 31.1.

*tally\_name\_string*: 30.4.

*tally\_num\_conversions*: 30.4, 31.1.

*tally\_rank*: 30.4, 31.1, 33.2.

*tally\_rank\_ind*: 30.4.

*tally\_reac\_ind*: 30.4.

*tally\_size*: 30.4, 31.1, 32.3, 35.1.

*tally\_tab\_index*: 30.4, 31.1, 32.2, 33.2.

*tally\_tag\_string*: 30.4.

*tally\_type*: 30.1, 30.4, 31.1.

*tally\_type\_base*: 30.4, 31.1.

*tally\_type\_ind*: 30.4.

*tally\_type\_num*: 30.4, 31.1.

*tally\_var\_list*: 30.4.

*tally\_version*: 30.4.

*tallyfile*: 54.1.

*tallysetup*: 31.1.

*target*: 26.2, 27.1.

*target\_e\_ion\_delta*: 26.2, 26.3, 27.1.

*target\_e\_ion\_mult*: 26.2, 26.3, 27.1.

*target\_ind*: 26.2, 27.1.

*target\_lookup*: 26.1, 27.2.

*target\_material*: 26.2, 26.3, 27.1.

*target\_recyc\_coef*: 26.2, 26.3, 27.1.

*target\_sector*: 26.1, 26.2, 26.3, 27.1.

*target\_temperature*: 26.2, 26.3, 27.1.

*temp*: 26.3, 27.2.

*TEMPCOUNT*: 50.7.

*temperature*: 39.4.

*test*: 43.3, 44.3.

*test\_*: 8.1.

*testfile*: 54.1.

*time\_*: 8.1.

*tl*: 30.4, 31.1.

*tl\_args*: 30.1.

*tl\_check*: 30.1.

*tl\_copy*: 30.1.

*tl\_cv\_divide\_number*: 30.3, 31.1.

*tl\_cv\_max\_conversions*: 30.3, 30.4, 31.1.

*tl\_cv\_max\_local\_data*: 30.3.

*tl\_cv\_max\_params*: 30.3.

*tl\_cv\_max\_partners*: 30.3, 30.4, 31.1.

*tl\_cv\_max\_scalers*: 30.3, 30.4, 31.1.

*tl\_cv\_output*: 30.3, 31.1.

*tl\_cv\_Pa\_per\_mTorr*: 30.3.

*tl\_cv\_partner\_unknown*: 30.3.

*tl\_cv\_pos\_1*: 30.3.

*tl\_cv\_pos\_2*: 30.3.

*tl\_cv\_pos\_3*: 30.3.

*tl\_cv\_post*: 30.3, 31.1.

*tl\_cv\_problem\_sp\_mass*: 30.3.

*tl\_cv\_scale*: 30.3, 31.1.

*tl\_cv\_scaler\_unknown*: 30.3.

*tl\_cv\_test\_mass*: 30.3.

*tl\_cv\_three\_halves*: 30.3.

*tl\_cv\_to\_external\_coords*: 30.3, 31.1.

*tl\_cv\_to\_internal\_coords*: 30.3, 31.1.

*tl\_cv\_track*: 30.3, 31.1.

*tl\_cv\_unknown*: 30.3.

*tl\_cv\_volume*: 30.3.

*tl\_cv\_zone\_pos\_1*: 30.3.

*tl\_cv\_zone\_pos\_2*: 30.3.

*tl\_cv\_zone\_pos\_3*: 30.3.

**tl\_decl**: 30.1.

**tl\_decls**: 30.5.

*tl\_dep\_var\_scalar*: 30.3, 31.1.

*tl\_dep\_var\_vector*: 30.3, 31.1.

*tl\_dummy*: 30.1.

*tl\_est\_collision*: 30.2, 31.1.

- tl\_est\_max*: [30.2](#), [30.4](#).
- tl\_est\_post\_process*: [30.2](#), [31.1](#).
- tl\_est\_track*: [30.2](#), [31.1](#).
- tl\_est\_unknown*: [30.2](#), [31.1](#).
- tl\_geom\_detector*: [30.3](#), [31.1](#).
- tl\_geom\_global*: [30.3](#), [31.1](#).
- tl\_geom\_surface*: [30.3](#), [31.1](#).
- tl\_geom\_unknown*: [30.3](#).
- tl\_geom\_volume*: [30.3](#), [31.1](#).
- tl\_index*: [31.1](#).
- tl\_index\_angle\_bin*: [30.3](#).
- tl\_index\_detector*: [30.3](#).
- tl\_index\_diagnostic*: [30.3](#).
- tl\_index\_energy\_bin*: [30.3](#).
- tl\_index\_material*: [30.3](#).
- tl\_index\_max*: [30.3](#), [30.4](#).
- tl\_index\_plasma\_zone*: [30.3](#).
- tl\_index\_pmi*: [30.3](#).
- tl\_index\_problem\_sp*: [30.3](#).
- tl\_index\_reaction*: [30.3](#).
- tl\_index\_sector*: [30.3](#).
- tl\_index\_source\_group*: [30.3](#).
- tl\_index\_strata*: [30.3](#).
- tl\_index\_strata\_segment*: [30.3](#).
- tl\_index\_test*: [30.3](#).
- tl\_index\_test\_author*: [30.3](#).
- tl\_index\_unknown*: [30.3](#).
- tl\_index\_wavelength\_bin*: [30.3](#).
- tl\_index\_zone*: [30.3](#).
- tl\_index\_zone\_ind\_1*: [30.3](#).
- tl\_index\_zone\_ind\_2*: [30.3](#).
- tl\_name\_length*: [30.3](#), [30.4](#).
- tl\_num*: [30.1](#), [30.4](#), [31.1](#).
- tl\_rank\_max*: [30.2](#), [30.4](#), [33.2](#).
- tl\_scoring\_index*: [30.3](#), [31.2](#).
- tl\_set\_base\_size*: [30.6](#).
- tl\_set\_baseinc*: [30.6](#).
- tl\_tag\_length*: [30.3](#), [30.4](#).
- tl\_type\_est*: [31.1](#).
- tl\_type\_max*: [30.1](#), [30.2](#), [30.4](#).
- tl\_type\_reaction*: [30.2](#), [31.1](#).
- tl\_type\_sector*: [30.2](#), [31.1](#).
- tl\_type\_test*: [30.2](#), [31.1](#).
- tl\_type\_undefined*: [30.2](#).
- tmax*: [8.3](#), [9.2](#).
- trim*: [49.1](#).
- TRUE*: [23.3](#), [33.1](#), [35.1](#).
- type*: [8.3](#), [9.2](#), [24.4](#), [27.1](#), [50.2](#), [50.6](#), [51](#).
- type\_*: [8.1](#).
- type\_ind*: [31.1](#).
- type\_x*: [50.2](#).
- u*: [39.4](#).
- unit\_string*: [37.3](#).
- up*: [39.4](#).
- use*: [50.7](#).
- v\_ext\_to\_int*: [20.2](#).
- v\_int\_to\_ext*: [20.3](#).
- v\_t*: [20.2](#), [20.3](#).
- vacuum*: [26.2](#), [27.1](#).
- vacuum\_ind*: [26.2](#), [27.1](#).
- vacuum\_sector*: [26.2](#), [26.3](#), [27.1](#).
- var*: [48.1](#), [50.7](#).
- var\_alloc*: [37.5](#), [41.6](#), [45.3](#), [47.3](#), [50.4](#), [51](#).
- var\_beg*: [50.2](#).
- var\_def*: [52.2](#), [53](#).
- var\_free*: [50.4](#), [51](#).
- var\_inq*: [52.2](#), [53](#).
- var\_read*: [52.2](#), [53](#).
- var\_realloc*: [37.5](#), [41.6](#), [45.3](#), [47.3](#), [50.4](#), [50.5](#), [51](#).
- var\_realloca*: [26.3](#), [50.5](#), [51](#).
- var\_reallocb*: [50.5](#), [51](#).
- var\_reallocc*: [50.5](#), [51](#).
- var\_type*: [50.2](#).
- var\_type\_star*: [50.2](#).
- var\_typea*: [50.2](#), [50.4](#).
- var\_write*: [52.2](#), [53](#).
- varname*: [51](#), [53](#).
- vc\_copy*: [8.1](#), [10](#), [20.2](#), [20.3](#).
- vector*: [20.4](#), [24.3](#), [26.2](#), [28.3](#).
- velocity\_*: [8.1](#).
- view\_ind*: [29.1](#).
- w\_sector*: [27.2](#).
- wall*: [26.2](#), [27.1](#).
- wall\_ind*: [26.2](#), [27.1](#).
- wall\_lookup*: [26.1](#), [27.2](#).
- wall\_material*: [26.2](#), [26.3](#), [27.1](#).
- wall\_recyc\_coef*: [26.2](#), [26.3](#), [27.1](#).
- wall\_sector*: [26.1](#), [26.2](#), [26.3](#), [27.1](#).
- wall\_temperature*: [26.2](#), [26.3](#), [27.1](#).
- walls*: [26.2](#).
- web*: [22.2](#), [23.3](#), [27.1](#), [31.1](#), [38.1](#), [42.1](#), [44.4](#), [48.1](#), [55](#).
- weight\_*: [8.1](#).
- x*: [4.1](#), [6.1](#), [14.1](#), [16.1](#), [18](#), [20.1](#), [22.1](#), [24.1](#), [26.1](#), [28.1](#), [30.1](#), [36.3](#), [43.1](#).
- x\_base*: [41.6](#), [47.3](#).
- x\_inc*: [41.6](#).
- x\_size*: [41.6](#), [47.3](#).
- x\_tab*: [41.6](#), [47.3](#).
- xs*: [15.1](#), [37.3](#), [38.1](#), [46.1](#).
- xs\_data*: [38.2](#).
- xs\_data\_base*: [37.2](#), [37.3](#), [38.1](#).
- xs\_data\_inc*: [37.3](#), [38.1](#).

*xs\_data\_ind*: 37.3.  
*xs\_data\_size*: 37.3, 38.1.  
*xs\_data\_tab*: 37.2, 37.3, 38.1, 38.2.  
*xs\_data\_table*: 37.2, 38.2.  
**xs\_decls**: 37.4.  
*xs\_dep\_var\_max*: 37.1, 37.3, 39.2, 40.1, 45.1.  
*xs\_eval\_name*: 37.3, 38.1.  
*xs\_eval\_name\_length*: 37.1, 37.3, 39.4, 45.2.  
*xs\_max*: 37.3, 38.1.  
*xs\_max\_indep\_params*: 37.1.  
*xs\_max\_random*: 37.1.  
*xs\_min*: 37.3, 38.1.  
*xs\_mult*: 37.3, 38.1.  
*xs\_name*: 37.3, 38.1.  
*xs\_num\_dep\_var*: 37.3, 38.1.  
*xs\_ragged\_alloc*: 37.5, 38.2.  
*xs\_ragged\_realloc*: 37.5, 38.2.  
*xs\_rank*: 37.3, 38.1.  
*xs\_spacing*: 37.3, 38.1.  
*xs\_spacing\_linear*: 37.1, 46.1.  
*xs\_spacing\_log*: 37.1, 46.1.  
*xs\_spacing\_unknown*: 37.1.  
*xs\_symbol\_string*: 37.3.  
*xs\_tab\_index*: 37.2, 37.3, 38.1, 38.2, 46.2.  
*xs\_table\_rank\_max*: 37.1, 37.3, 45.2.  
*xs\_tag\_string\_length*: 37.1, 37.3, 45.2.  
*xs\_unit\_string\_length*: 37.1, 37.3.  
*xs\_units*: 37.3, 38.1.  
*xs\_var*: 37.3, 38.1.  
*xs\_var\_density*: 37.1.  
*xs\_var\_energy*: 37.1.  
*xs\_var\_m\_back*: 37.1.  
*xs\_var\_m\_emitter*: 37.1.  
*xs\_var\_m\_test*: 37.1.  
*xs\_var\_sp\_energy*: 37.1.  
*xs\_var\_sp\_temperature*: 37.1.  
*xs\_var\_temperature*: 37.1.  
*xs\_var\_unknown*: 37.1.  
*xs\_var\_v\_flow\_1*: 37.1.  
*xs\_var\_v\_flow\_2*: 37.1.  
*xs\_var\_v\_flow\_3*: 37.1.  
*xs\_var\_v\_flowb\_1*: 37.1.  
*xs\_var\_v\_flowb\_2*: 37.1.  
*xs\_var\_v\_flowb\_3*: 37.1.  
*xs\_var\_v\_test\_1*: 37.1.  
*xs\_var\_v\_test\_2*: 37.1.  
*xs\_var\_v\_test\_3*: 37.1.  
*xs\_var\_zone*: 37.1.  
*xs\_var\_1st\_random\_number*: 37.1.  
*xsection*: 46.1.  
*xsection\_version*: 37.3.  
*xxx\_base*: 46.1, 48.1.  
  
*xz\_ind*: 57.1.  
  
*zero*: 8.3, 20.2, 20.3, 26.3.  
*zi\_ix*: 24.2.  
*zi\_iy*: 24.2.  
*zi\_iz*: 24.2.  
*zi\_ptr*: 24.2, 25.1.  
*zn*: 24.3, 25.1, 27.1.  
*zn\_args*: 24.1.  
*zn\_check*: 24.1.  
*zn\_copy*: 24.1.  
**zn\_decl**: 24.1.  
*zn\_dummy*: 24.1.  
*zn\_exit*: 24.2.  
*zn\_index*: 24.1, 25.1, 25.2, 27.1, 33.1.  
*zn\_index\_max*: 24.2, 24.3, 25.1, 25.2.  
*zn\_num*: 24.1, 24.3, 25.2, 28.3.  
*zn\_plasma*: 20.4, 24.2, 25.1.  
*zn\_pointer*: 20.1, 24.1, 24.4, 25.1.  
*zn\_solid*: 24.2, 25.1.  
*zn\_type*: 24.1, 24.4, 25.1.  
*zn\_type\_max*: 24.1, 24.2, 24.3.  
*zn\_type\_set*: 24.4.  
*zn\_undefined*: 24.2.  
*zn\_vacuum*: 24.2, 25.1.  
*zn\_volume*: 24.1, 25.1.  
*zone*: 24.4, 25.1, 25.2, 26.1, 27.1, 27.2, 29.1.  
*zone\_*: 10.  
*zone\_center*: 24.3, 25.2.  
*zone\_exit*: 8.2.  
*zone\_ind*: 24.3.  
*zone\_index*: 24.1, 24.2, 24.3, 26.2, 33.1.  
*zone\_index\_ind*: 24.3.  
*zone\_index\_max*: 24.3, 25.2.  
*zone\_index\_min*: 24.3, 25.2.  
*zone\_max*: 24.3, 25.2.  
*zone\_min*: 24.3, 25.2.  
*zone\_next\_*: 10.  
*zone\_pointer*: 24.1, 24.3.  
*zone\_type*: 24.1, 24.3.  
*zone\_type\_ind*: 24.3.  
*zone\_type\_num*: 20.4, 24.3, 24.4.  
*zone\_version*: 24.3.  
*zone\_volume*: 24.1, 24.3.

**COMMAND LINE:** "fweave -f -i! -W[ -ykw700 -ytw40000 -j -n/  
/u/dstotler/degas2/src/classes.web".

**WEB FILE:** "/u/dstotler/degas2/src/classes.web".

**CHANGE FILE:** (none).

**GLOBAL LANGUAGE:** FORTRAN.